Software Requirements Specification

for

URECAMatch

Version 1.0

Prepared by IAN TAN, ROBIN TEOH, NICHOLAS NG,

EDWIN TAN QI LUN, BENJAMIN NG

SCEF GRP 4

18/10/2026

Table of Contents

1. Introduction **4**
- 1.1 Purpose 4
- **1.2 Document Conventions** **4**
- **1.3 Intended Audience and Reading Suggestions** **5**
- **1.4 Product Scope** **5**
- **1.5 References** **6**

2. Overall Description **8**
- 2.1 Product Perspective 8
- 2.2 Product Functions 9
- 2.3 User Classes and Characteristics 12
- 2.4 Operating Environment 13
- 2.5 Design and Implementation Constraints 14
- 2.6 User Documentation 15
- 2.7 Assumptions and Dependencies 15

3. External Interface Requirements **17**

4. System Features **60**
- 6.1 Database Requirements 85
- 6.2 Legal and Data-Protection Requirements 85
- 6.3 Internationalisation Requirements 85
- 6.4 Reuse Objectives 86
- 6.5 Operational Requirements 86

Revision History

Name	Date	Reason For Changes	Version
Robin	2/9/2026	LAB 1 Feedback (Data Dictionary, FR/NFR Identifiers)	0.1
	15/9/2026	LAB 2 (Data Dictionary, FR/NFR, Class Diagram, Dialog Map)	0.2
	17/10/2026	LAB 3	

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the requirements and specifications for URECAMatch (“Version 1.0”), a web-based portal designed to host the application and allocation process of the Undergraduate Research Experience on Campus (URECA) programme at Nanyang Technological University (NTU).

1.2 Document Conventions

This section describes the conventions used throughout this Software Requirements Specification (SRS).

1.2.1 Typography

- Headings are set in Times New Roman.
- Body text, tables and captions are set in Arial.
- Italic text denotes a note or an example.
- Monospaced text denotes identifiers, such as field names, API paths and requirement codes.

1.2.2 Requirement Identification

- Each functional requirement is assigned a unique identifier in the form REQ-<FEATURE>-<n>, for example REQ-ROUND-1.
- Each use case is assigned a unique identifier in the form #<actor group>-<n>, for example #1-1.
- Identifiers are never reused. If a requirement is withdrawn, its identifier is retired.
- Requirement identifiers are used in the source code and test suite, to support traceability from this SRS to the implementation.

1.2.3 Requirement Statements and Priority

- The words "shall", "should" and "may" indicate a mandatory, a recommended and an optional requirement respectively.
- Priority is assigned at the feature level. Every requirement under a feature inherits the priority of that feature.

1.2.4 Figures and Tables

- Figures and tables are numbered by section, for example Figure 3.1 and Table 3.2.
- Each figure and table has a caption, and is referred to by its number in the text.

1.2.5 Document Versioning

- Each revision of this document is recorded in the Revision History, with its version number, date, author and the reason for the change.

1.3 Intended Audience and Reading Suggestions

This document is written for the Group 4 developers and testers, the course teaching team who assess it, and the URECA office staff who would run the system. Section 2 describes the product, its users and its constraints. Section 3 specifies the external interfaces. Section 4 gives each system feature, its use case and its requirement identifiers. Section 5 gives the non-functional requirements. The appendices hold the glossary, the analysis models and the system architecture.

Developers and testers should read Sections 2, 4 and 5 in that order, and consult Section 3 for the screens. Teaching staff should begin with Sections 1 and 2. URECA office staff should read 2.2 and 2.3, then the Admin features in Section 4.

1.4 Product Scope

URECAMatch is a web portal that runs the URECA application and allocation process at NTU. Today, projects are listed on the NTU website as only a title, project ID and supervisor, in a list of over 1,000 entries with no search function and no description, prerequisites, time commitment or number of open slots. The only way for a student to apply is to e-mail the professor, so students cannot see how many others have applied or rank their choices, and one student may collect several offers while others get none. Supervisors can post several projects, but they must manage all their applicants by e-mail, which is tedious and gives them no shared view or tool to compare or rank applicants. Admins have no control over, or information about, the application process.



List of URECA Project Proposals

--

ADM ASE CCEB CEE CoB (NBS) EEE IGS MAE MSE NIE RSIS SBS SCBE SCSE SPMS SSS SoH WKW SCI All

School	Project ID	Type	Title	Supervisor
ADM	ADM26026	URECA	Personalized AI Assistant for Hand-Drawn Animation: Local ComfyUI Workflow for Style-Consistent Production	Asst Prof Ahmad Ismail Saleh
ADM	ADM26013	URECA	Creative Critical Writing on Film/Media # 1	Mr Benjamin Alexander Slater
ADM	ADM26014	URECA	Creative Critical Writing on Film/Media # 2	Mr Benjamin Alexander Slater
ADM	ADM26015	URECA	Research based Story/Script Development # 1	Mr Benjamin Alexander Slater
ADM	ADM26016	URECA	Research-based Story/Script Development # 2	Mr Benjamin Alexander Slater
ADM	ADM26011	URECA	Visualising Cultural Heritage in VR	Assoc Prof Benjamin Seide
ADM	ADM26021	URECA	Visualising Cultural Heritage in Cinematic Virtual Reality	Assoc Prof Benjamin Seide
ADM	ADM26031	URECA	Visualising Cultural Heritage in VR - 2	Assoc Prof Benjamin Seide
ADM	ADM26032	URECA	Visualising Cultural Heritage in VR - 3	Assoc Prof Benjamin Seide
ADM	ADM26005	URECA	"The Frenzy of Orlando"animation for the communication of epic poem in a transcultural framework	Assoc Prof Davide Benvenuti
ADM	ADM26006	URECA	Fashion and costume design for animation	Assoc Prof Davide Benvenuti
ADM	ADM26007	URECA	Webcomic and serialisation for public health communication. A study on the Asian best practices and how those can be applied in the Singapore context	Assoc Prof Davide Benvenuti
ADM	ADM26017	URECA	Filmmaking and AI: Exploring the Future of Creative Production 1.	Asst Prof Ella Raidel
ADM	ADM26018	URECA	Filmmaking and AI: Exploring the Future of Creative Production 2.	Asst Prof Ella Raidel
ADM	ADM26019	URECA	Vertical Filmmaking: New Formats for New Audiences	Asst Prof Ella Raidel
ADM	ADM26020	URECA	Social Media Filmmaking: Creative Formats and Audience Engagement	Asst Prof Ella Raidel
ADM	ADM26027	URECA	Social Media Filmmaking: Creative Formats and Audience Engagement II.	Asst Prof Ella Raidel
ADM	ADM26033	URECA	Attack on Titan-Critical Analysis	Prof Hans-Martin Rall
ADM	ADM26022	URECA	Agentic AI Pipelines for Immersive Media	Assoc Prof Ina Conradi Chavez
ADM	ADM26023	URECA	ComfyUI Pipelines for AI-Generated Moving Image	Assoc Prof Ina Conradi Chavez
ADM	ADM26028	URECA	Agentic AI Pipelines for Immersive Media Part 2	Assoc Prof Ina Conradi Chavez
ADM	ADM26030	URECA	Agentic AI Pipelines for Immersive Media: Style-Adaptive Agents Trained on an Authored Film Corpus	Assoc Prof Ina Conradi Chavez
ADM	ADM26034	URECA	Agentic Spaces: Interaction Design for Generative Immersive Installations	Assoc Prof Ina Conradi Chavez
ADM	ADM26002	URECA	Exploring AI Capacity in Colour Design & Selection	Assoc Prof Ng Woon Lam
ADM	ADM26029	URECA	Between Accuracy and Interpretation: Colour Theory in Film Scanning	Assoc Prof Ng Woon Lam
ADM	ADM26035	URECA	Human-in-the-Loop 3D Reconstruction from Image	Asst Prof Ruan Lingyan

Figure 1.1 Current URECA Application System

URECAMatch replaces this with one system. Supervisors publish project listings. Students browse, search and filter them, apply with a statement and documents, and rank their applications. Supervisors rank their applicants. The system matches students to projects in sequential rounds, using both sets of rankings. Supervisors may decline a proposed match in an approval window. The system then publishes the results as offers, which students accept or decline. Students may also propose a project to a supervisor. Admins configure cycles and rounds, assign students manually, manage accounts and read the audit log.

1.5 References

The following documents are referred to in this SRS.

[1] SC2006 Group 4, Use Case Document.

[2] SC2006 Group 4, Lab Deliverables 1 to 4, Nanyang Technological University, submitted to the course and stored in the GitHub repository softwarelab3/2006-SCEF-A4

[3] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE, 1998.

[4] Nanyang Technological University, Undergraduate Research Experience on Campus (URECA) programme web pages, accessed October 2026.

[5] Nanyang Technological University, Content of Courses (course catalogue), accessed October 2026.

2. Overall Description

2.1 Product Perspective

URECAMatch is a new, self-contained web-based product. It is not a follow-on member of a product family, nor is it a component of a larger software system. It replaces the existing manual process for allocating students to URECA research projects, in which projects are published as a list of titles on the NTU website and students approach supervisors individually by e-mail. That process provides no project detail, no search or filtering capability, no visibility of remaining slots, and no means for either party to express ranked preferences. Section 1.4 states the objectives of its replacement.

The system operates independently of NTU institutional systems and neither reads from nor writes to them (Figure 2.1). It depends on four external services. Registration is restricted to NTU e-mail domains, and verification codes are sent to the user's NTU address. An e-mail delivery service sends verification codes and notifications. Cloud services host the application and provide its database, user authentication and file storage. An AI match service scores applicants for supervisors when it is available; when it is not, the system scores them with its own formula. The system itself advances each matching round and times out unanswered offers as dates pass. Supporting documents such as academic transcripts are uploaded by students and held in the system's own document storage rather than retrieved from institutional records. Integration with NTU single sign-on is desirable for a production deployment but is outside the scope of this release. Each interface is specified in Section 3.

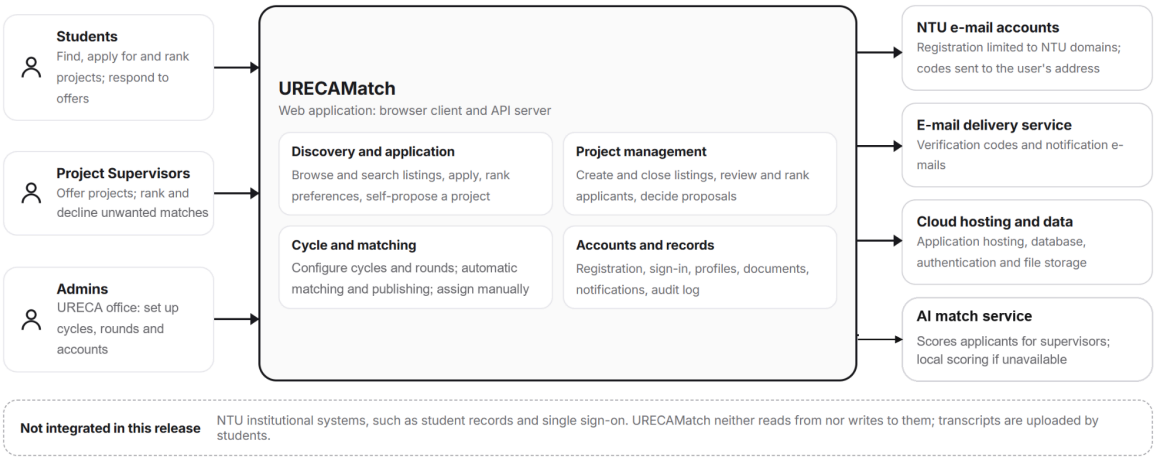


Figure 2.1 System overview

Allocation is organised into application cycles, normally one per academic year (Figure 2.2). Each cycle opens with a pre-application period, in which students may browse listings but not apply, followed by up to three matching rounds. In each round, students apply to projects and rank their applications during the application window; supervisors review and rank their applicants during the review window; the system then runs the matching process by itself when the review window closes. During the approval window that follows, supervisors may decline students matched to their projects, and matches not declined are accepted. The system publishes the results by itself when the

approval window ends. Students respond to their offers by the end of the next application window that has not yet closed. Students who remain unmatched and eligible may take part in the next round, so that the slots released by declined or expired offers can still be filled within the same cycle.

	Pre-application listings visible, no applying	**Application window** 1 to 14 days	**Review window** 1 to 7 days	**Approval window** 1 to 7 days, default 2	**Results and offers** until the next application window ends
Student	Completes profile; browses projects	Applies with a statement; ranks applications; may propose a project	Waits; may still view applications	Waits for the results	Receives offer or outcome; accepts or declines the offer
Project Supervisor	Publishes listings	Reviews applicants; decides proposals	Ranks applicants for each listing	Declines unwanted matches; the rest are accepted	Sees which offers are accepted
Admin	Creates the cycle; configures up to three rounds and their windows	Monitors applications on the dashboard	Monitors rankings	Reviews the matches	Assigns unmatched students in exceptional cases
System	Restricts applying until Round 1 opens	Enforces the window, profile and slot rules	Closes ranking; runs stable matching when the window ends	Accepts matches not declined; publishes results when the window ends	Times out unanswered offers daily

Up to 3 rounds Each round repeats the application to offer-response phases, starting 2 days after the previous approval window ends by default. Unmatched eligible students may carry their applications into the next round; students who accept, decline or let an offer time out take no further part in the cycle.

Figure 2.2 Application cycle and responsibilities

2.2 Product Functions

This section summarises the major functions that URECAMatch performs and the functions it allows each class of user to perform. Functions are grouped by the user role that initiates them. Detailed descriptions of each function, including its priority, stimulus and response sequences, and associated functional requirements, are provided in Section 4.

2.2.1 Use Case Diagram

The use case diagram in Figure 2.3 shows the actors that interact with URECAMatch and the use cases available to each. Student, Project Supervisor and Admin are specialisations of the generalised User actor, which owns the use cases common to all roles. Two system actors also take part: an e-mail delivery service, which receives verification codes and notification e-mails for delivery, and an AI service, which receives applicant details and supporting documents and returns a score for each applicant, and receives a draft application and returns tips. Each use case corresponds to the system feature of the same name in Section 4, and its full description is provided in the Use Case Descriptions document (Reference 1).

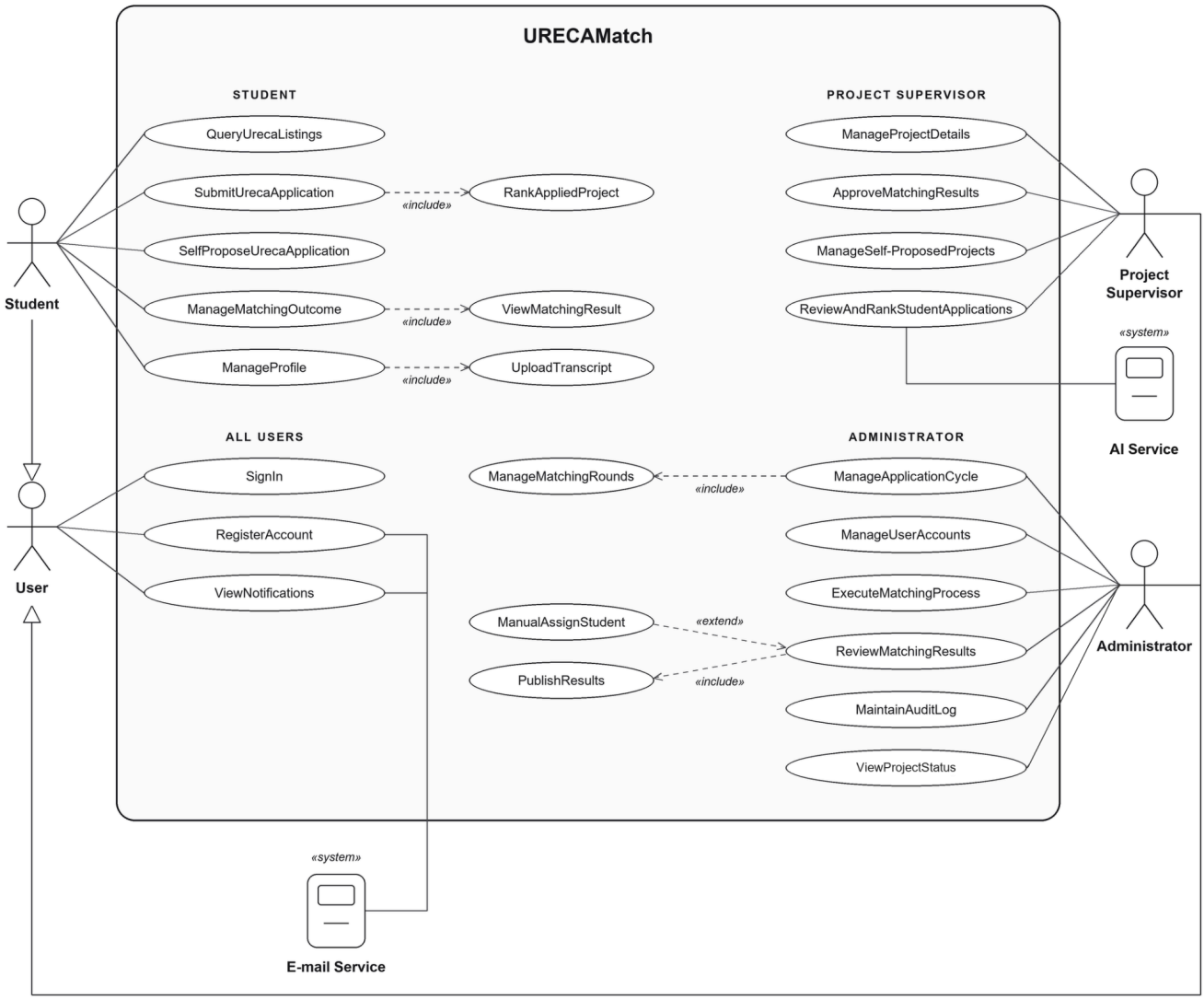


Figure 2.3 Use case diagram

2.2.2 Major Product Functions

URECAMatch allows its users to perform the following functions:

- **All users**
  - Register an account as a Student or Project Supervisor and verify it with a code sent to an NTU e-mail address; Admin accounts are created by an Admin
  - Sign in, sign out, and view notifications
- **Students**
  - Complete and maintain a profile, including completed modules, research interests and supporting documents
  - Browse, search and filter project listings by keyword, school, topic and supervisor
  - Apply to listings with a statement of interest and supporting documents, and withdraw an application while it is under review

  - Rank applications in order of preference
  - Check a draft application and receive tips before submitting it
  - See recommended projects, each with a match score
  - Propose a project directly to a chosen supervisor
  - Withdraw a proposal until the supervisor answers it
  - View published results and accept or decline a match offer
- **Project Supervisors**
  - Create, publish, update and close project listings, or save them as drafts
  - Review applicants and their supporting documents, and rank applicants by preference, optionally guided by an AI score, and set how much each factor counts in a project's AI score
  - Decline matches proposed for their projects during the approval window; matches not declined are accepted
  - Review student proposals and accept or decline them
- **Admins**
  - Create and change application cycles (a cycle ends on its closing date)
  - Configure up to three sequential matching rounds and whether unmatched applications carry over
  - Set the length of each round's review and approval windows, and review the matching results
  - Reassign a match, assign an unmatched student manually, or unassign a student, each with a recorded justification
  - Create, update and deactivate user accounts
  - View the audit log of all administrative operations
  - View every project and the students it holds, read only
- **System**
  - Notify users in the application and by e-mail when an event concerns them
  - Run matching when a review window closes, publish results when an approval window ends, and time out unanswered offers

2.2.3 Class Diagram

The class diagram in Figure 2.4 shows the principal entity classes maintained by URECAMatch, their attributes, and the relationships between them. User is specialised into Student, ProjectSupervisor and Admin, and Application into ListingApplication and SelfProposedApplication. An ApplicationCycle comprises one to three MatchingRounds; each round collects listing applications and applicant rankings and produces match results, each of which allocates one student to one project listing. Notifications, verification codes and audit log entries are associated with the user they concern or the user who initiated them.

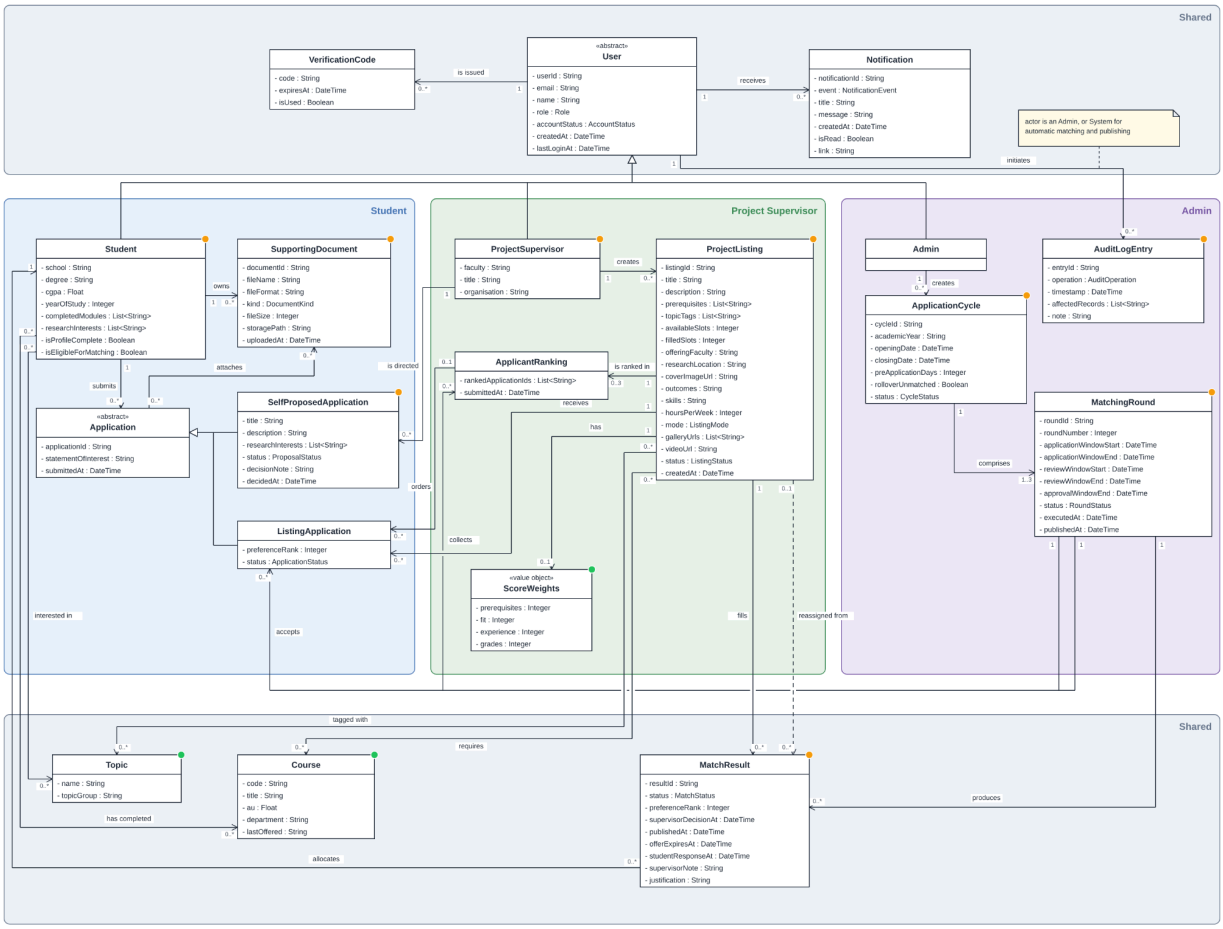


Figure 2.4 Class diagram

2.3 User Classes and Characteristics

Users of the platform fall into three classes, which differ in number, frequency of use, privilege and the functions they need (Figure 2.5).

Students are undergraduates of NTU seeking a URECA research placement, typically in their second or third year of study. They are the largest user class. All are familiar with web applications, but their technical expertise varies and none can be assumed to have prior experience of a preference-based allocation system. Usage is concentrated and infrequent: a student interacts with the system heavily during the application window of each matching round and when results are published, and rarely at other times. Students hold the lowest privilege level, with access restricted to their own profile, applications and match results. They use the discovery, application, ranking and match response functions, and no administrative function.

Project Supervisors are academic staff who offer research projects and select students for them. They are experienced professionals with high educational attainment but no assumed familiarity with the system, and their available time is limited. Usage is periodic rather than continuous, consisting of listing projects before a cycle opens and of applicant review, ranking and declining unwanted matches within each round. Applicant review must therefore be achievable in a single sitting without

recourse to external tools, which is the basis of the requirement that documents be viewable within the system. Supervisors may access only the projects they own and the applicants to those projects.

Administrators are staff of the URECA office responsible for overseeing the allocation process. They are the smallest user class, are trained in the operation of the system, and use it throughout a cycle rather than at particular points. They hold the highest privilege level and are the only class able to configure application cycles and matching rounds, override an allocation manually and manage user accounts. Matching and publication run automatically when a round's windows close. The audit and override functions pertain exclusively to this class.

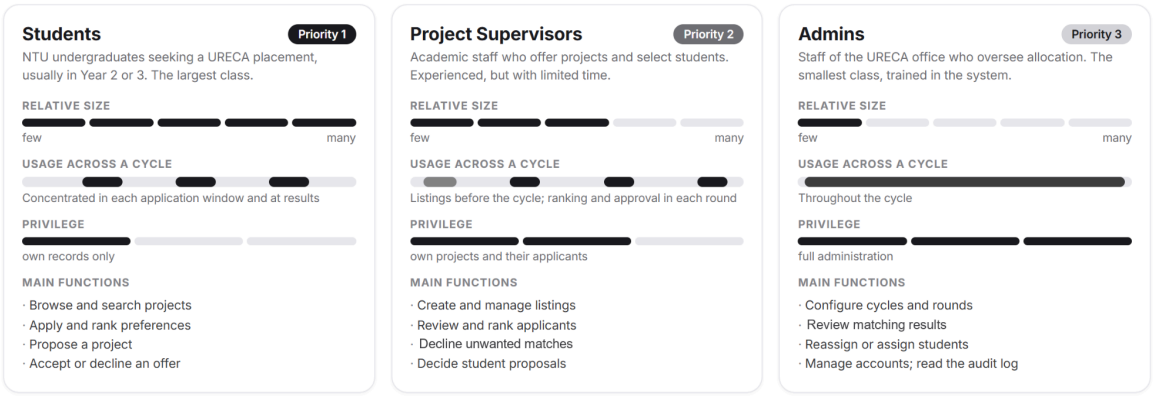


Figure 2.5 User classes and characteristics

Students are the most important user class to satisfy. They are the most numerous, they are the group whose experience of the existing process the system exists to improve, and they have the least tolerance for a system that is difficult to use, since they have no obligation to persist with one. Project Supervisors are the second most important. Their participation is essential, as a matching round cannot produce a valid allocation without supervisor rankings, but they are a smaller group with a direct interest in the outcome and are correspondingly more likely to complete a task that is not effortless. Administrators are the least important class to satisfy in terms of interface quality. They are few in number, are trained in the system and use it as part of their duties, so their functions may prioritise correctness, completeness and auditability over ease of use.

2.4 Operating Environment

URECAMatch is a web application, and no software is installed on the user's computer. Users access it through a current version of Chrome, Edge, Safari or Firefox on a desktop or laptop computer running any operating system those browsers support, in a browser window at least 1200 px wide. Tablets and smartphones are not supported in this release.

The web client, built with Next.js 16, and the API server, built with Python 3.12 and FastAPI, are hosted on Vercel in its Singapore region (sin1). The database, user authentication and file storage are provided by Supabase, a managed PostgreSQL service, and e-mail is delivered through Resend. The system must coexist with the browser extensions users may have installed and with NTU's e-mail service, through which verification codes and notifications are received. The supported environment and every external interface are specified in Section 3.

2.5 Design and Implementation Constraints

The following constraints limit the options available to the developers:

- **Technology** The web client is implemented in Next.js with TypeScript, the API server in Python with FastAPI, the sign-in animation with three.js (WebGL) and the motion library (a plain graphic is shown without WebGL), persistence, authentication and file storage on Supabase, and hosting on Vercel. These were chosen to match the team's existing expertise.
- **Architecture** The system follows a layered Boundary-Control-Entity architecture. Every business rule is enforced in the API server, not only in the browser; all data is accessed through a single Store interface; and the generated API contract is the single definition shared by the web client and the server.
- **Hosting** The API runs as serverless functions, so persistent state must be held in the database rather than in server memory, and each request must complete within the hosting provider's time limit.
- **Service limits** The free tiers of Supabase, Vercel and Resend limit database size, function execution time and the number of e-mails that can be sent each day.
- **Data protection** Personal data is handled in accordance with Singapore's Personal Data Protection Act (Section 5.3.3). Student documents are stored privately and released only to authorised users through short-lived links.
- **Security** Registration is restricted to NTU e-mail domains, passwords are handled only by the authentication service, and service keys are never sent to the browser.
- **Language** The interface is in English, and dates and times are shown in Singapore time.
- **Process** The product is delivered within one semester by a team of five. All changes are made through reviewed pull requests, and every change must pass automated linting, tests, type checks and a production build before it is merged. Code and tests are tagged with the requirement identifiers they implement.

2.6 User Documentation

No separate user manual is delivered. Users are guided within the interface itself: each screen has a clear title, and screens that need it carry a short description, form fields carry hints, empty lists explain what will appear and how to proceed, and every error message states how to correct the problem (Section 3.1.1).

For developers and maintainers, the following documentation is delivered with the source code and maintained alongside it in Markdown:

- **Setup guide** How to install, configure and run the web client and the API server, (docs/SETUP.md).

- **Contributing guide** Branching, review and testing practices for the team (docs/CONTRIBUTING.md).
- **Architecture** The layers, components and design patterns of the system (docs/architecture.md).
- **API specification** The OpenAPI document generated from the code (shared/openapi.json), browsable at /docs on a running API server.
- **Traceability matrix** The mapping from each requirement to its endpoint, screen and test (docs/traceability.md).

2.7 Assumptions and Dependencies

The requirements in this document rest on the following assumptions. If any of them proves incorrect, the affected requirements must be reviewed.

- **A-1** Every student and supervisor has an active NTU e-mail account that can receive messages from the system.
- **A-2** Supervisors rank their applicants before each review window closes. Applicants who are not ranked cannot be matched to that supervisor's project.
- **A-3** Documents uploaded by students are genuine. The system does not verify transcripts against NTU records.
- **A-4** At most one application cycle is active at a time, and each cycle is of a size similar to the current URECA programme: about 1,300 project listings from about 480 supervisors, based on the currently published URECA project list.
- **A-5** Users access the system from a supported desktop browser with a reliable internet connection.

The project also depends on the following external factors:

- **D-1** Supabase remains available for the database, user authentication and file storage.
- **D-2** Vercel remains available for hosting the application and running its scheduled jobs.
- **D-3** Resend delivers e-mail promptly, and NTU's e-mail service does not filter it as spam. The notification latency in NFR 5.1.2 depends on both.
- **D-4** Google's AI service stays available for AI scores and application tips. If it does not, scores come from the system's own formula, tips come from fixed checks, and nothing else is affected.
- **D-5** The open-source frameworks on which the system is built, including Next.js, React, FastAPI and Pydantic, remain supported. A major upgrade of any of them may require changes to the code.

3. External Interface Requirements

3.1 User Interfaces

URECAMatch is used through a web browser by three groups of users: Students, Project Supervisors and Admins. This subsection specifies the logical characteristics of the interface between the system and these users: the conventions that apply to every screen, the visual style, the shared components, the way results and errors are reported, accessibility, and the design of each screen. The visual style and components are defined once in a shared stylesheet and component library, so that every screen looks and behaves the same; the flow between screens is specified by the dialog map in Appendix B.

3.1.1 General Conventions

Each user signs in once and is then shown only the functions of their role. A sidebar on every signed-in screen carries the navigation for that role, the user's name and role, and the sign-out control (Figure 3.1). After sign-in, Students arrive at Browse projects, Project Supervisors at My projects and Admins at the Dashboard. Counts of unread notifications and of items awaiting the user, such as proposals or match approvals, appear as badges beside the relevant menu item, so that pending work is visible from any screen.

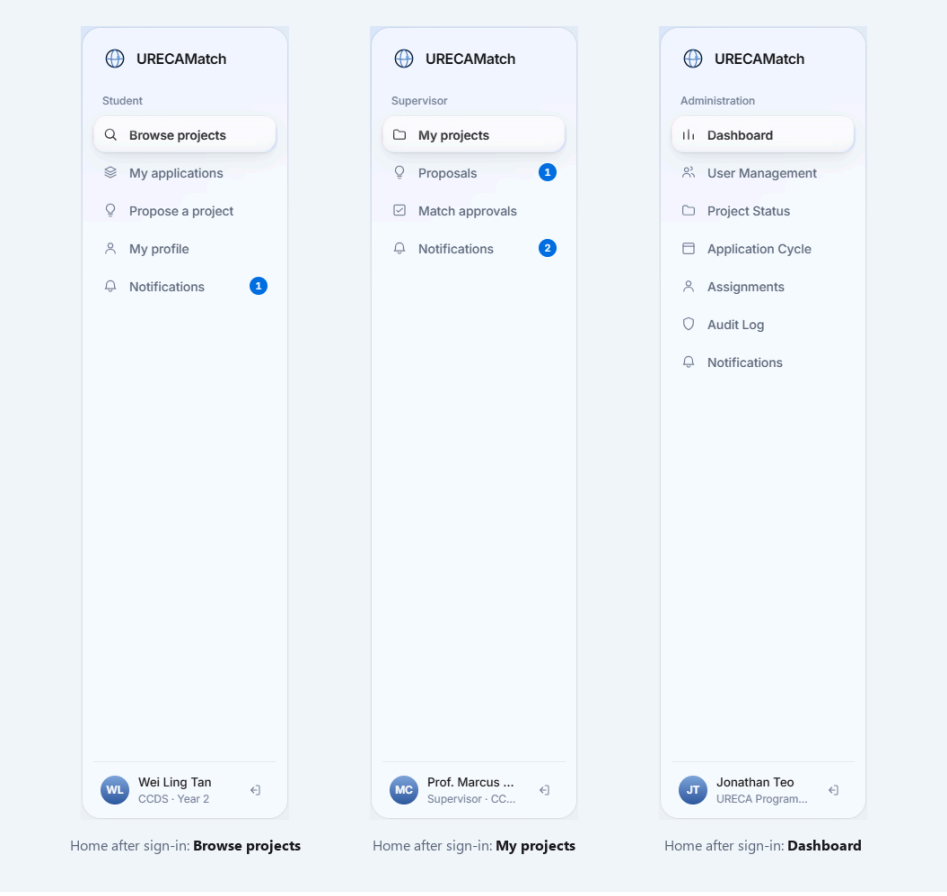


Figure 3.1 Navigation by role

Every screen follows the same structure (Figure 3.2): a page header with the screen title, the cycle status capsule on the screens where the current round matters (it shows the round, the time left in its application window and, on hover, every round's dates), a back link on screens reached from another screen, and the content of the screen. Where a list can be long, search and filters sit directly above it. The interface is in English. Dates are shown as day and month, for example 10 Sep, and times are shown in Singapore time and labelled SGT, so that every deadline is unambiguous.

The system has no separate help pages. Guidance is given where it is needed instead: form fields carry short hints, checklists show what is left to do before an action is allowed, empty lists explain what will appear and how to proceed, and every error message states how to correct the problem (3.1.6).

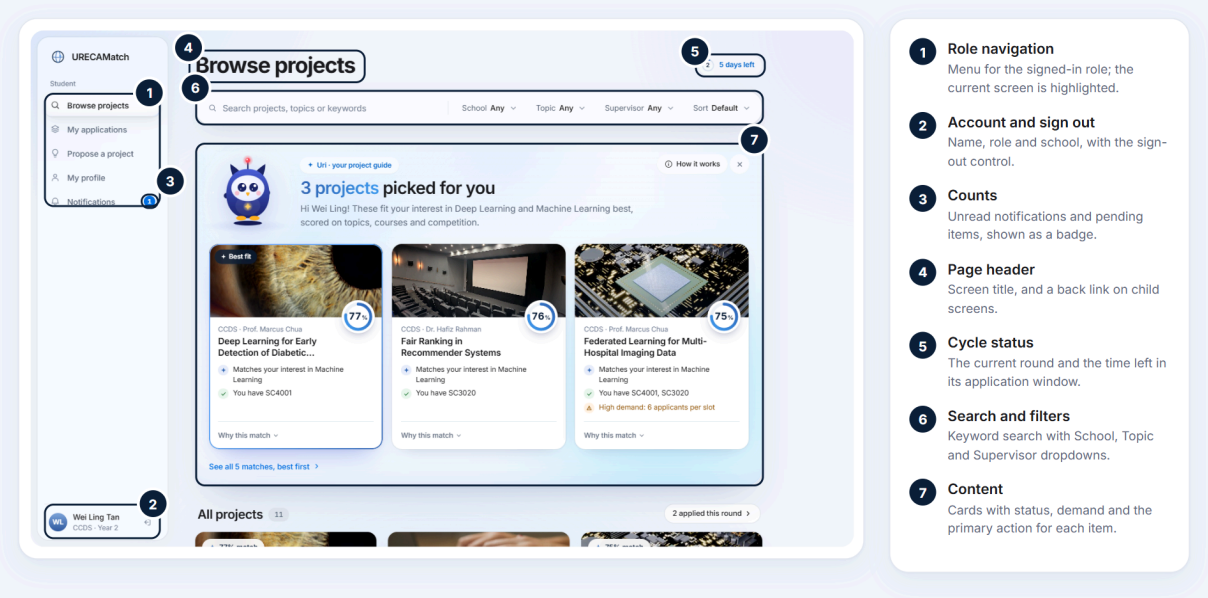


Figure 3.2 Anatomy of a screen

3.1.2 Colour

The palette is calm and cool, so that colour carries meaning rather than decoration (Figure 3.3). A pale ice-blue canvas and white cards draw the structure of every screen, and dark navy ink is used for text and for the logo. A single accent blue marks what the user can act on: buttons, links, the keyboard focus ring and count badges. Three semantic colours express state: green for success and approval, amber for caution, waiting and deadlines, and red for errors, refusals and destructive actions. Each semantic colour has a darker ink shade, used for text, and a light soft shade, used for backgrounds, and text placed on a soft shade always uses the ink shade of the same colour. The interface uses a light colour scheme only.

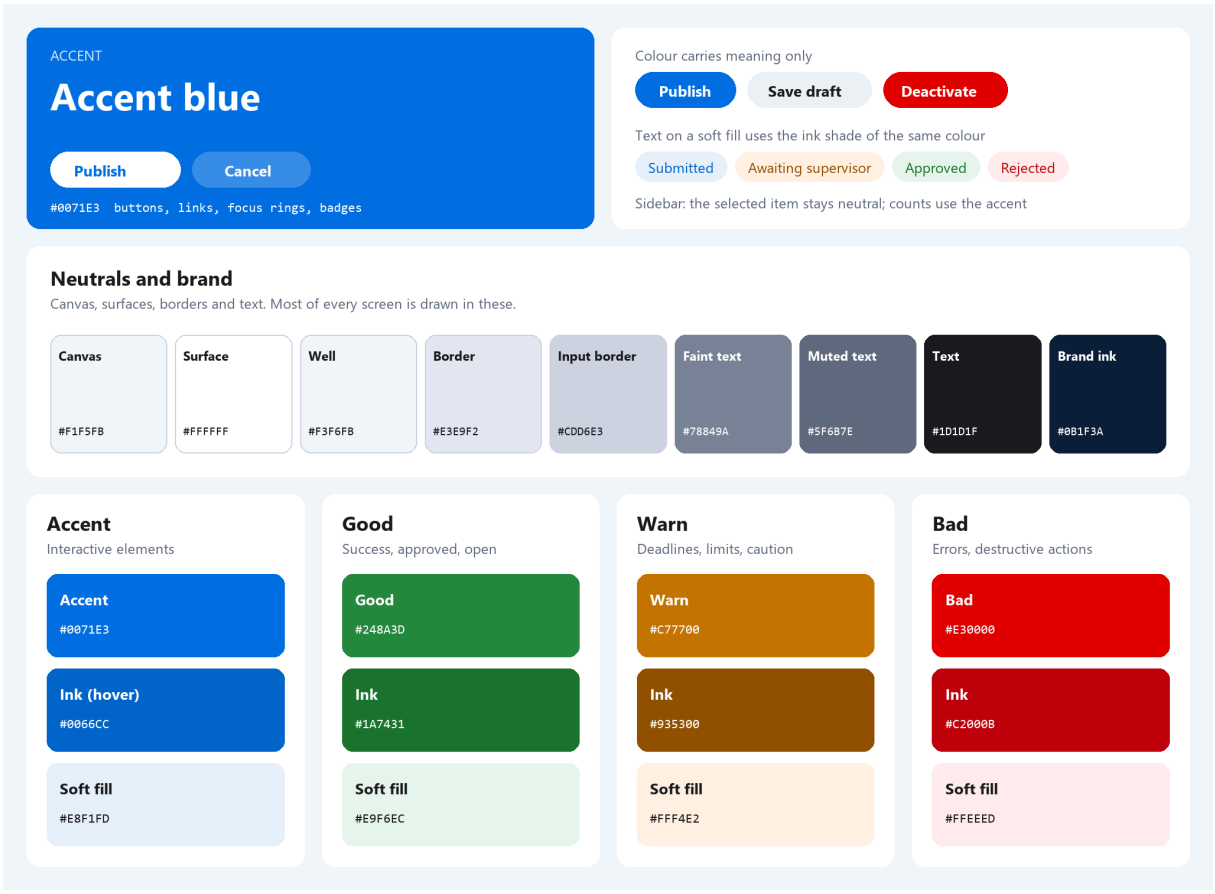


Figure 3.3 Colour palette

3.1.3 Typography

All text is set in one typeface, Inter, and hierarchy is expressed through size and weight rather than through different typefaces (Figure 3.4). Body text is 14 px with a line height of 1.5, page titles are 28 px bold, and no text is smaller than 11 px. Numbers that are read in columns or compared, such as dates, CGPA and slot counts, use tabular numerals so that their digits align. Identifiers such as module codes are shown in the system monospace font.

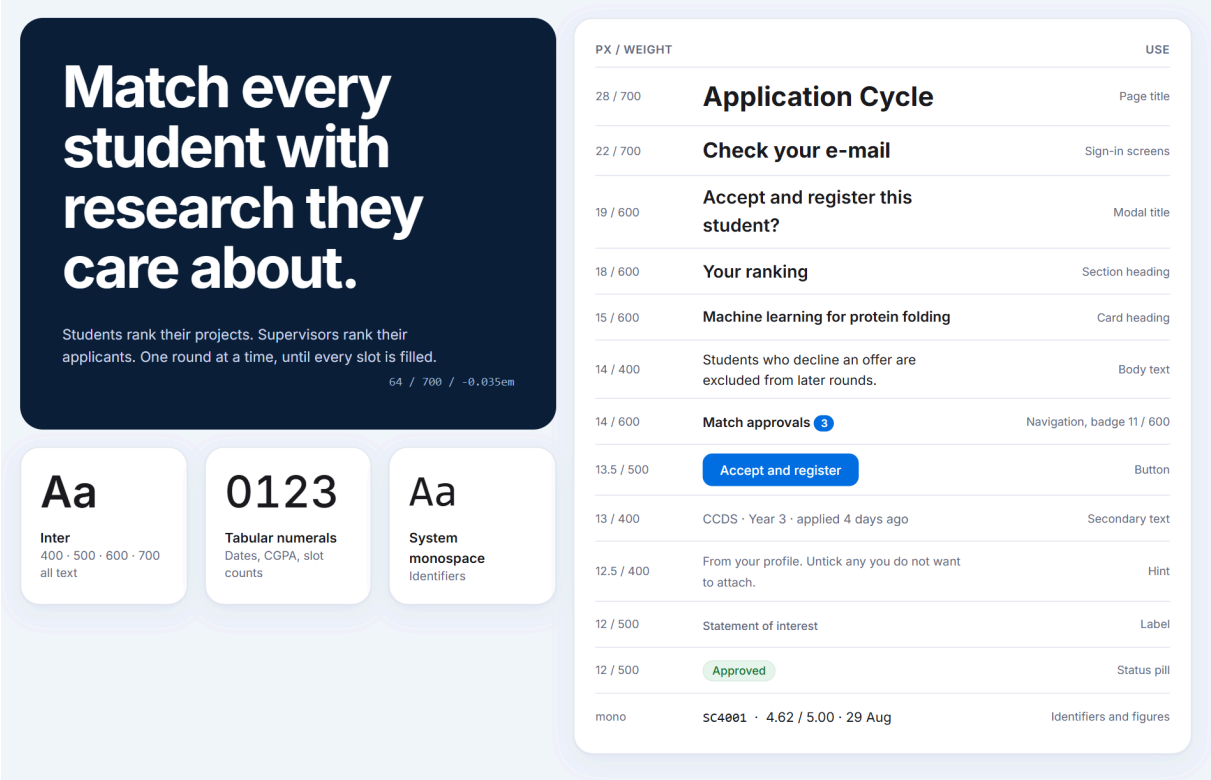


Figure 3.4 Typography

3.1.4 Layout, Shape and Interaction

All signed-in screens share one shell (Figure 3.5). A 240 px sidebar floats 12 px from the window edge, and the content region beside it is at most 1080 px wide. The interface is designed for a browser window at least 1200 px wide and aligns to a 4 px spacing grid. Corners are rounded according to the type of element, from 8 px for small icon controls to 20 px for modals and the sidebar, so that elements of the same kind always share the same shape. Each page eases in when it opens, in about 0.3 seconds.

Two overlays are used, each for one purpose. A drawer, 460 px wide, slides in from the right to show supporting detail without leaving the current screen, such as an applicant's profile and documents while a Supervisor ranks applicants. A modal, 520 px wide and centred, asks the user to confirm an action; a document preview uses a wider modal so that a PDF can be read. Both dim the screen behind them and close when the user presses Escape or clicks outside them. Buttons come in three heights, 44, 36 and 28 px, and every control offers a target of at least 28 by 28 px.

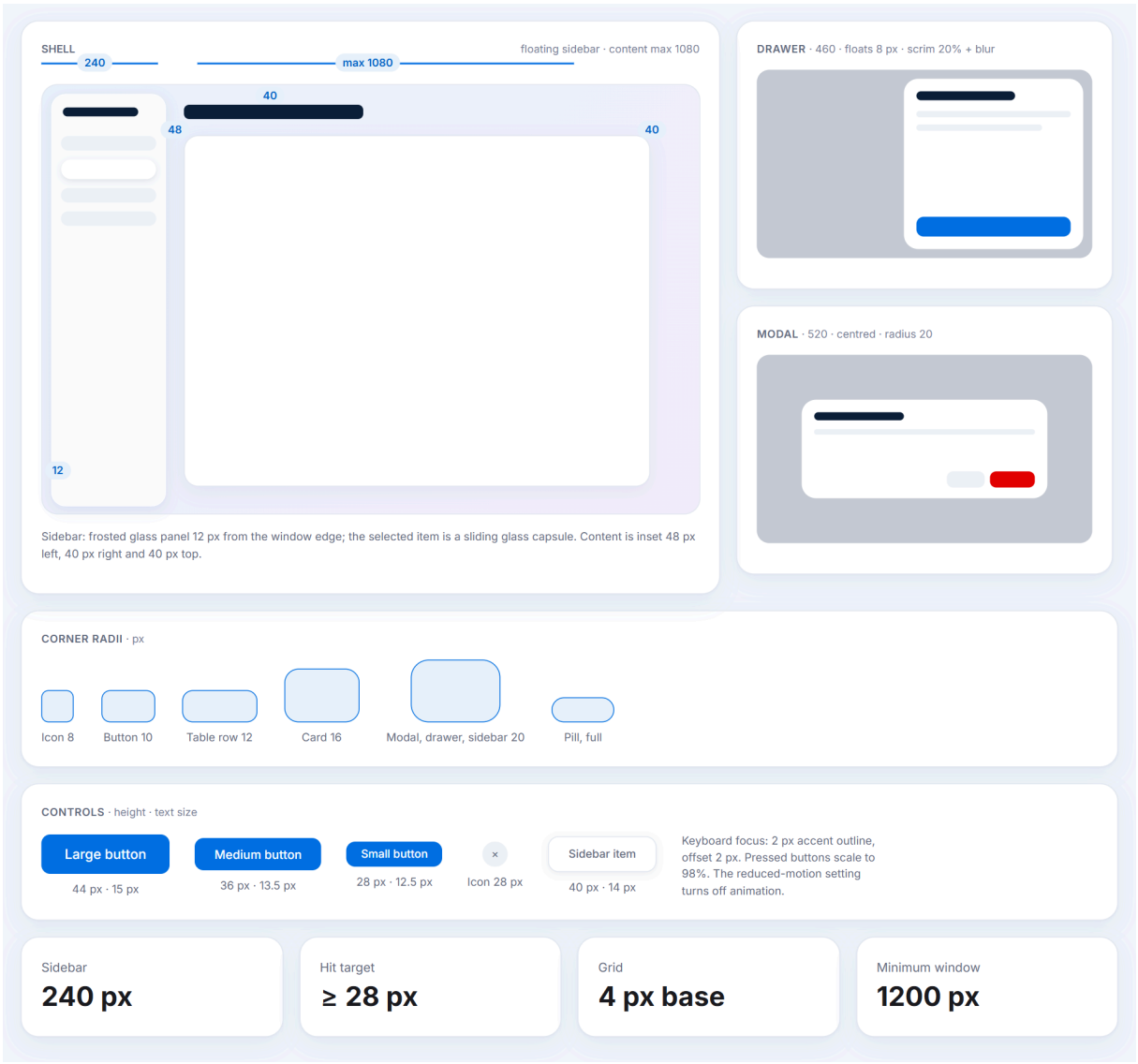


Figure 3.5 Layout, shape and controls

3.1.5 Common Components

Every screen is assembled from one shared library of components (Figure 3.6). A rule about appearance or behaviour, such as how a field error is shown or how a status is coloured, is therefore implemented once and applies to every screen that uses the component. The library provides buttons in five styles, text inputs and searchable pickers, tag inputs for topics and prerequisites, dropdown filters, status pills, deadline and rank badges, the cycle status capsule, toggles and steppers, file upload, statistic cards and progress bars, loading skeletons, the drawer and modal, and an in-browser PDF viewer. A new pattern is added to the library before it is used on a screen, so that the interface stays consistent as it grows.

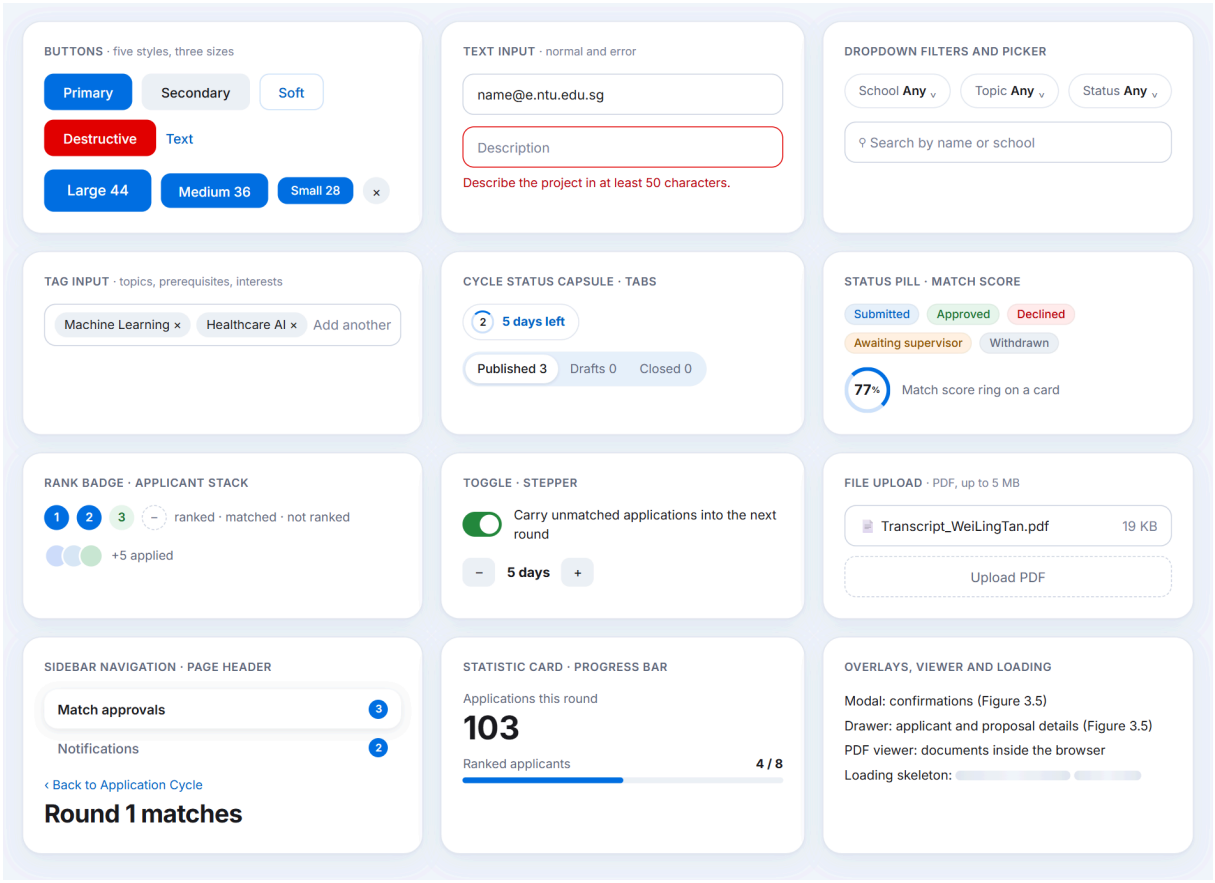


Figure 3.6 Common components

3.1.6 Messages and Status Labels

Every action a user takes produces a visible result, and every message is written in plain English, states what happened and, for a failure, says how to put it right (NFR 5.4.2). Messages take five forms (Figure 3.7). When the server rejects an input, the field at fault is outlined in red and the message appears directly below it. Notices that stay in the page appear as a card as wide as its text, with a coloured border: blue for guidance on the next step, green for success, amber for a deadline or limit, and red for a failure that is not tied to one field. Brief confirmations, such as "Saved.", appear as a floating message at the top of the content; they disappear by themselves after a few seconds, can be closed, and only one shows at a time. Before any action that cannot be undone, such as closing a listing, withdrawing an application, accepting or declining an offer or a proposal or deactivating a user, a modal states the consequence and asks for confirmation. A list with no items explains why and offers the next step. When a session expires after 30 minutes without activity, the user is returned to the sign-in page with the reason.

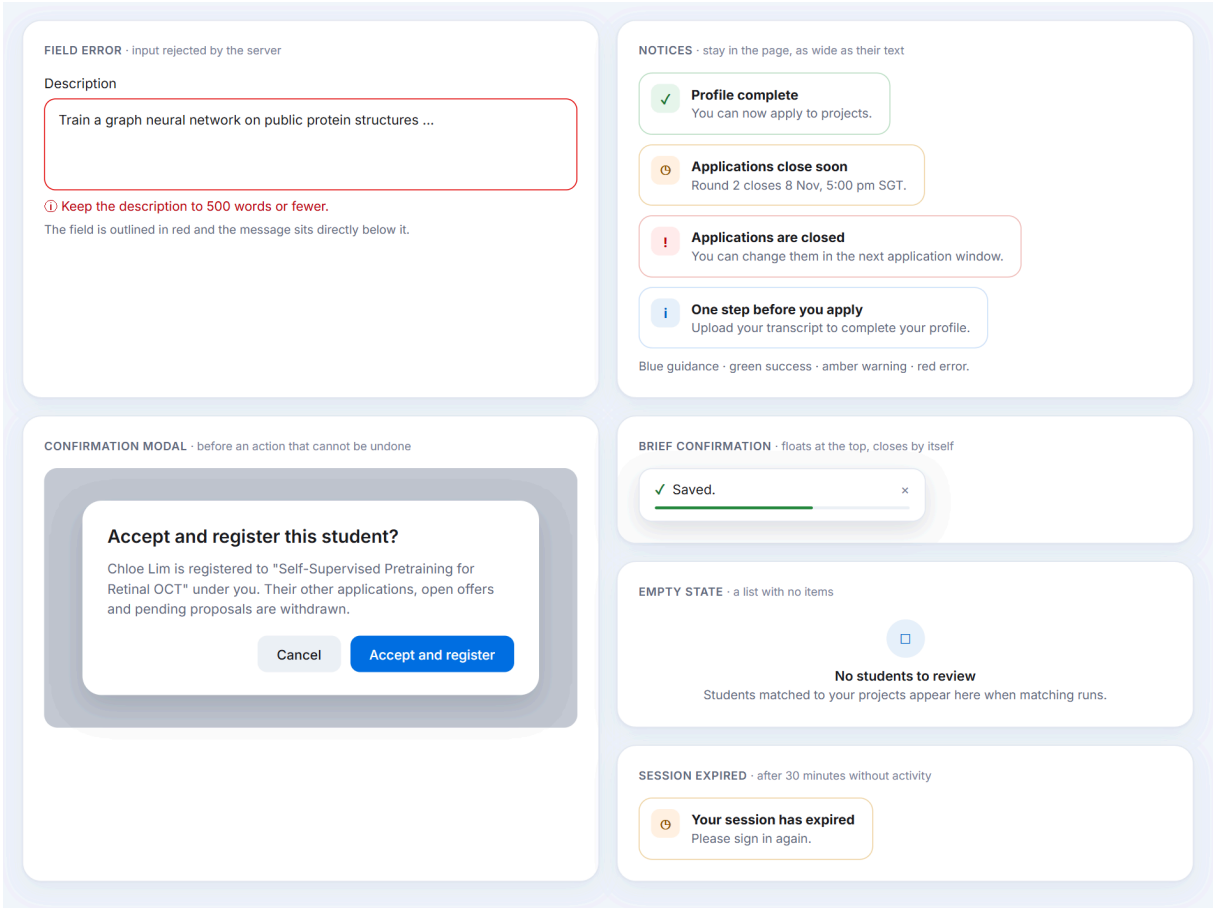


Figure 3.7 Messages and feedback

The status of every record, whether an application, match, proposal, listing, round or account, is shown with one fixed label and colour on every screen (Figure 3.8). The colour follows the meaning of the status: blue for in progress, green for success, amber for waiting or caution, red for ended or refused, and grey for inactive. A status therefore reads the same wherever it appears, and the label shown to users is always the same for the same stored value.

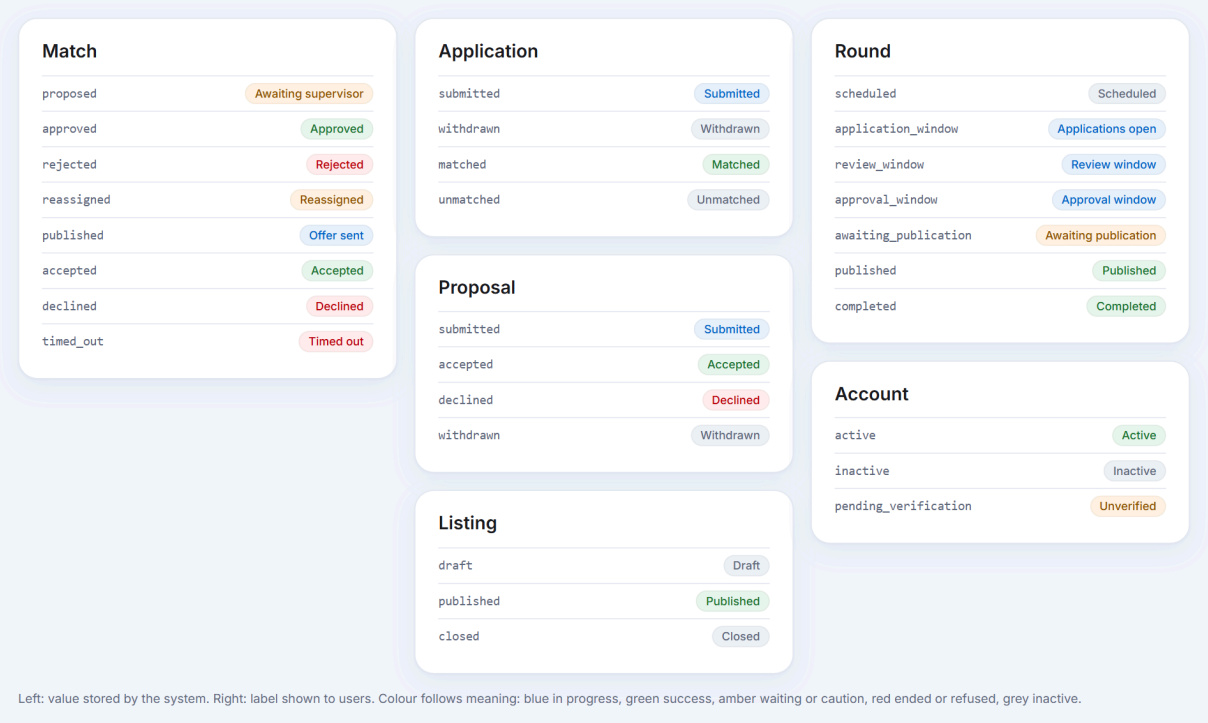


Figure 3.8 Status labels

3.1.7 Accessibility

The interface is designed to WCAG 2.2 Level AA. Text meets the contrast ratio of 4.5:1 against its background, with the exception of the faint grey used for hints and placeholders, which is to be darkened to meet it. Colour is never the only means of conveying information: every coloured status also carries a text label and every error also carries an icon. Every control can be reached and operated with the keyboard and shows a visible focus outline; forms submit with Enter, overlays close with Escape, and the dropdown menus follow the listbox pattern with arrow-key navigation. Icon-only controls carry a text label for screen readers, and the page language is declared as English. When the operating system requests reduced motion, animations and transitions are turned off.

3.1.8 Screen Designs

Figures 3.9 to 3.34 show every screen of the system, grouped by user role and captured at a window size of 1440 by 900 px. Each screen is named as in the dialog map (Appendix B), which specifies how the user moves between screens. Table 3.1 links each screen to its route and to the system feature in section 4 that it implements, so that every screen can be traced to its requirements.

Table 3.1 Screen index

Figure	Screen and route	User	Section 4 feature
3.9	SignInPage /sign-in	All	4.20 SignIn

Figure	Screen and route	User	Section 4 feature
3.10	CreateAccountPage /create-account	All	4.19 RegisterAccount
3.11	EmailVerificationPage /verify	All	4.19 RegisterAccount
3.12	CompleteProfilePage /profile?complete=1	Student	4.14 ManageProfile
3.13	BrowseProjectPage /browse	Student	4.9 QueryUrecaListings
3.14	ProjectDetailPage /projects/{id}	Student	4.9 QueryUrecaListings
3.15	ApplyProjectPage /apply/{id}	Student	4.10 SubmitUrecaApplication
3.16	ProposeProjectPage /propose	Student	4.12 SelfProposeUrecaApplication
3.17	ProposalSuccessPage /propose (after sending)	Student	4.12 SelfProposeUrecaApplication
3.18	MyApplicationPage /my-applications	Student	4.11 RankAppliedProjects 4.13 ManageMatchingOutcome
3.19	MyProfilePage /profile	Student	4.14 ManageProfile
3.20	NotificationPage /notifications	All	4.21 ViewNotifications
3.21	SupervisorProjectsPage /projects	Supervisor	4.15 ManageProjectDetails
3.22	CreateProjectPage /projects/new	Supervisor	4.15 ManageProjectDetails
3.23	EditProjectPage /projects/{id}/edit	Supervisor	4.15 ManageProjectDetails
3.24	ApplicantAndRankingsPage /projects/{id}/applicants	Supervisor	4.16 ReviewAndRankStudentApplications
3.25	StudentProposalsPage /proposals	Supervisor	4.18 ManageSelf-ProposedProjects

Figure	Screen and route	User	Section 4 feature
3.26	StudentProposalsPage, confirmation /proposals	Supervisor	4.18 ManageSelf-ProposedProjects
3.27	MatchApprovalsPage /approvals	Supervisor	4.17 ApproveMatchingResults
3.28	AdminDashboardPage /dashboard	Admin	4.1 to 4.8, overview
3.29	ManageUsersPage /users	Admin	4.8 ManageUserAccounts
3.30	MatchingRoundsPage /rounds	Admin	4.1 ManageApplicationCycle 4.2 ManageMatchingRounds 4.3 ExecuteMatchingProcess
3.31	MatchingAwaitPublishPage /rounds/{id}/matches	Admin	4.4 ReviewMatchingResults 4.5 PublishResults
3.32	AssignmentsPage /assignments	Admin	4.6 ManualAssignStudent
3.33	AuditOverridesPage /audit	Admin	4.7 MaintainAuditLog
3.34	ProjectStatusPage /listings	Admin	4.6 ManualAssignStudent

All users

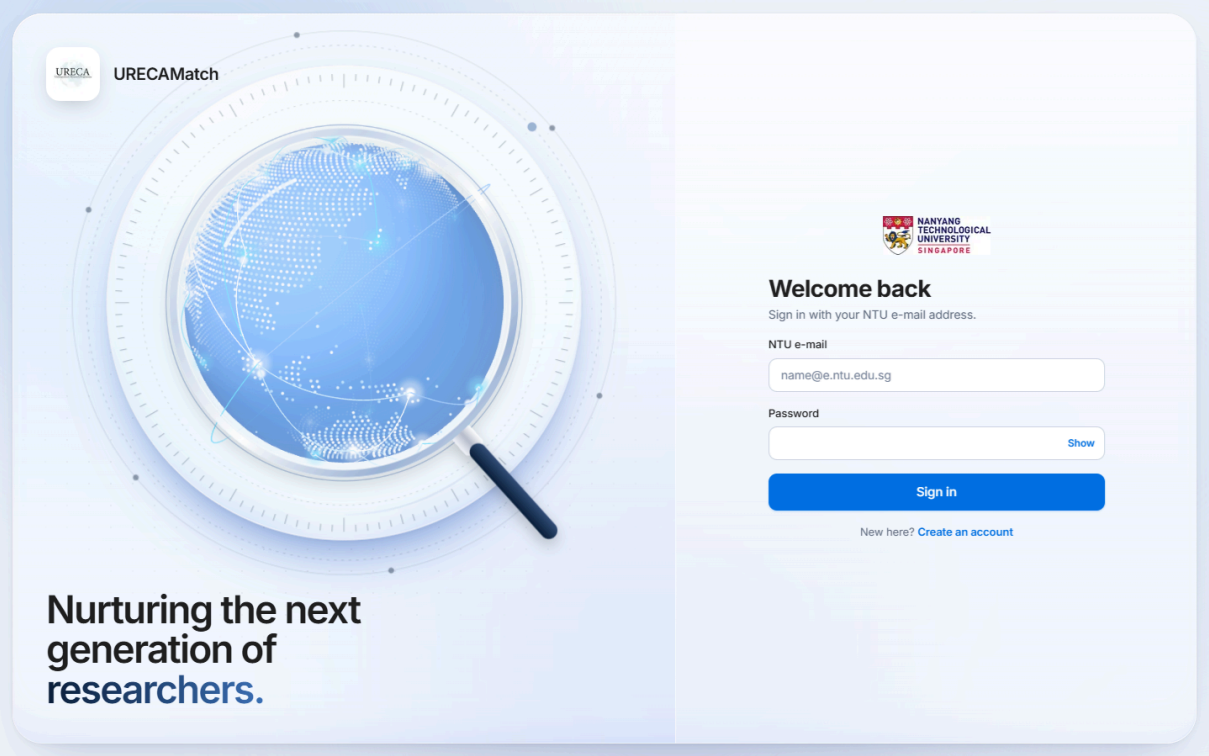


Figure 3.9 SignInPage

Sign-in with NTU e-mail and password, beside a short introduction and a headline. The password can be revealed. Links to account creation.

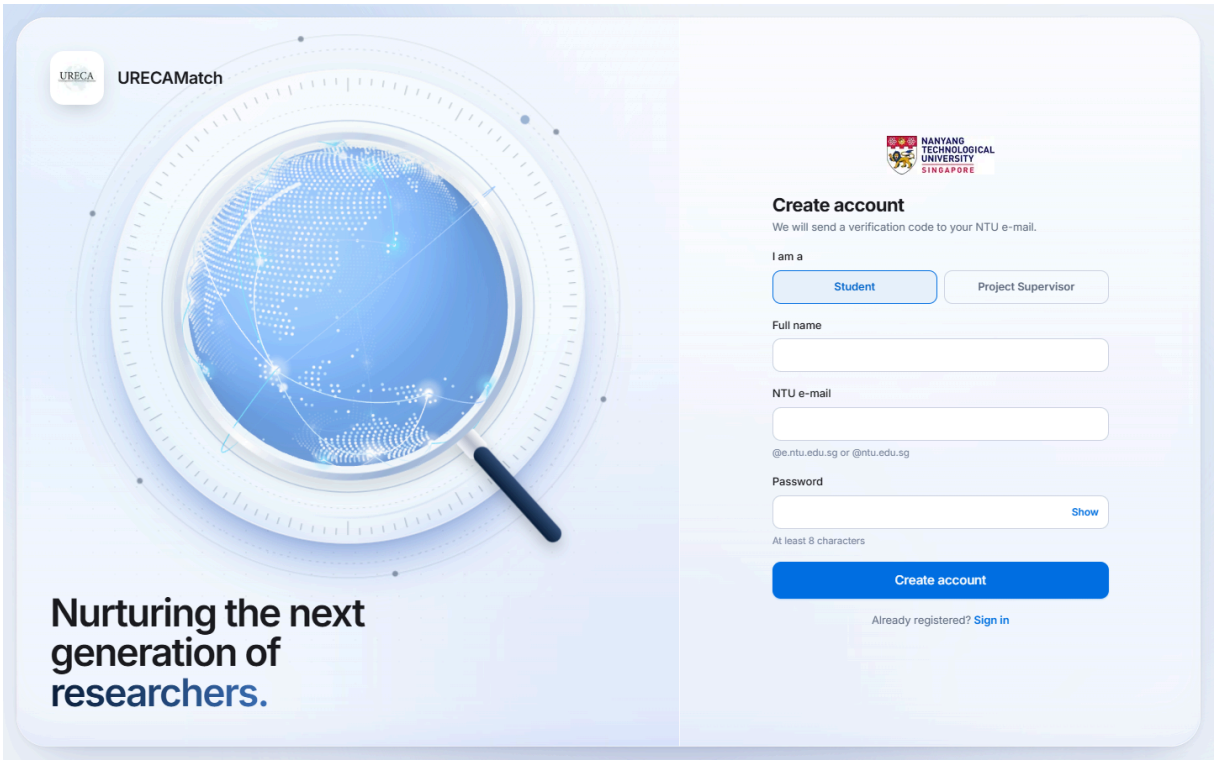


Figure 3.10 CreateAccountPage

Choice of Student or Project Supervisor, name, NTU e-mail and a password of at least 8 characters.

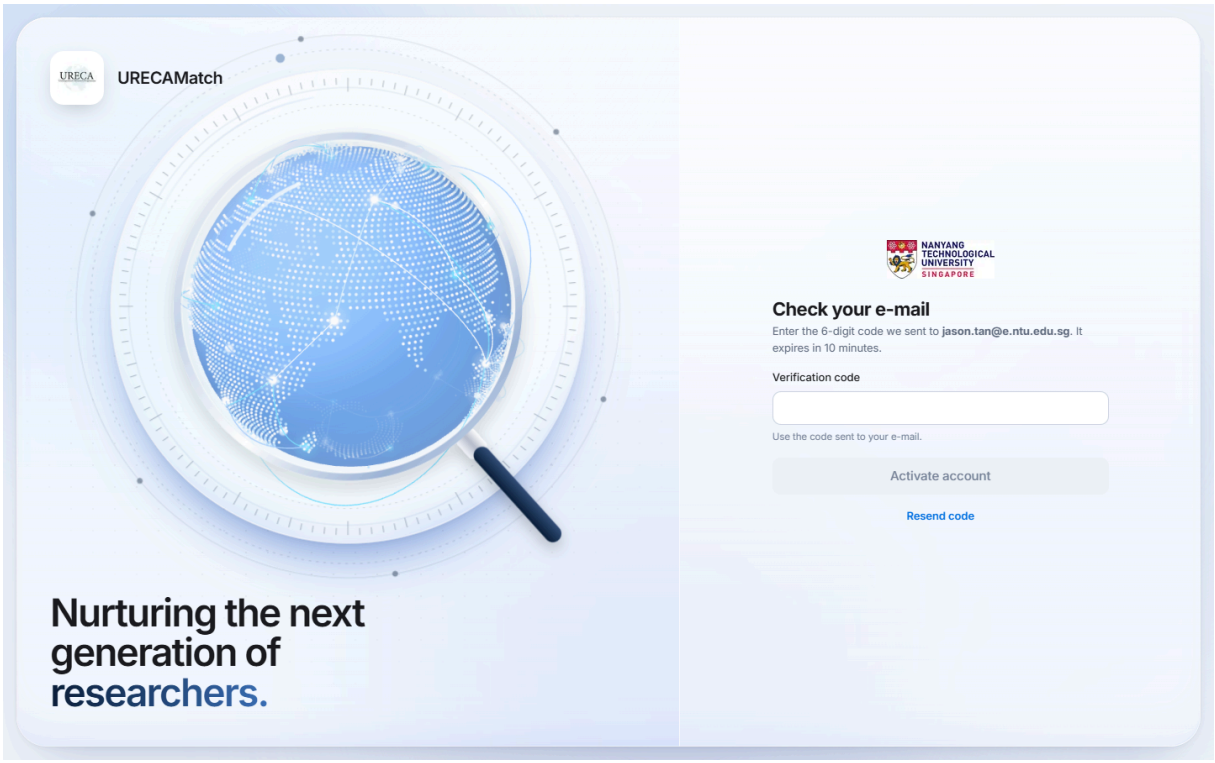


Figure 3.11 EmailVerificationPage

Entry of the 6-digit code sent by e-mail, which expires after 10 minutes. A new code can be requested.

Student

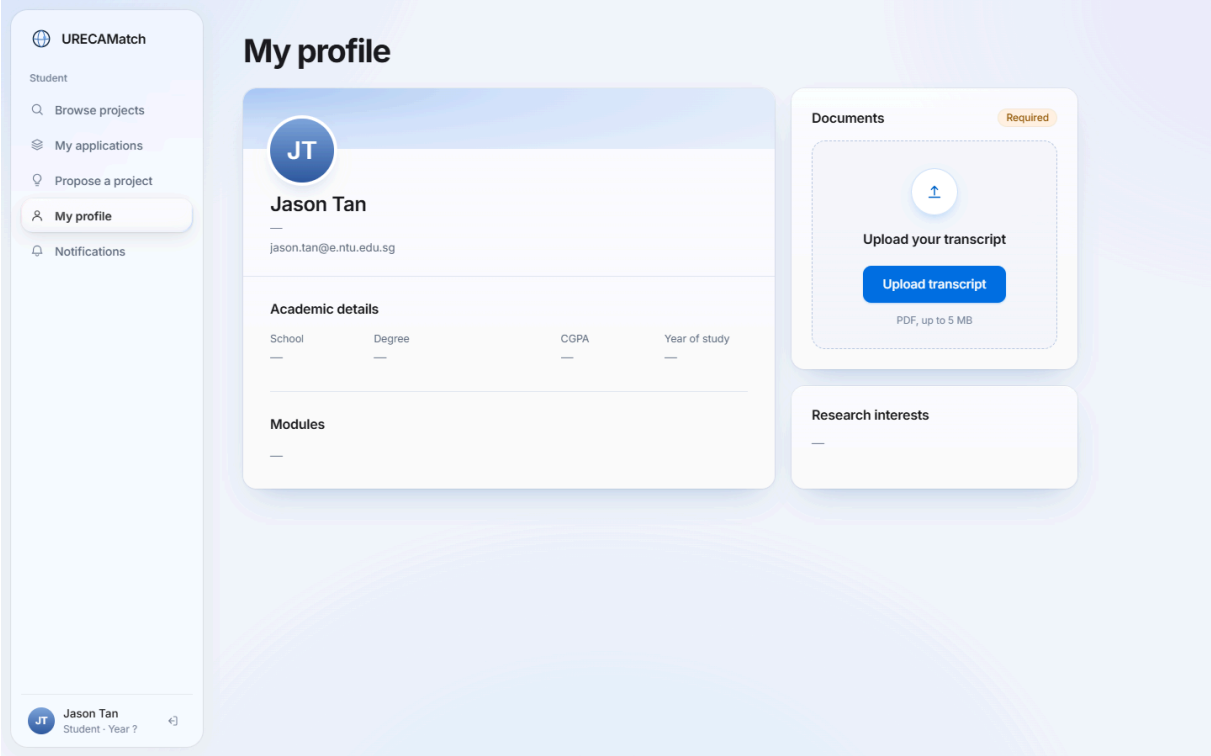


Figure 3.12 CompleteProfilePage

Shown after first sign-in, and in place of any other screen, until the profile is complete: the Student uploads a transcript, the school, degree, CGPA, year of study and modules fill in from it for checking, and research interests are added.

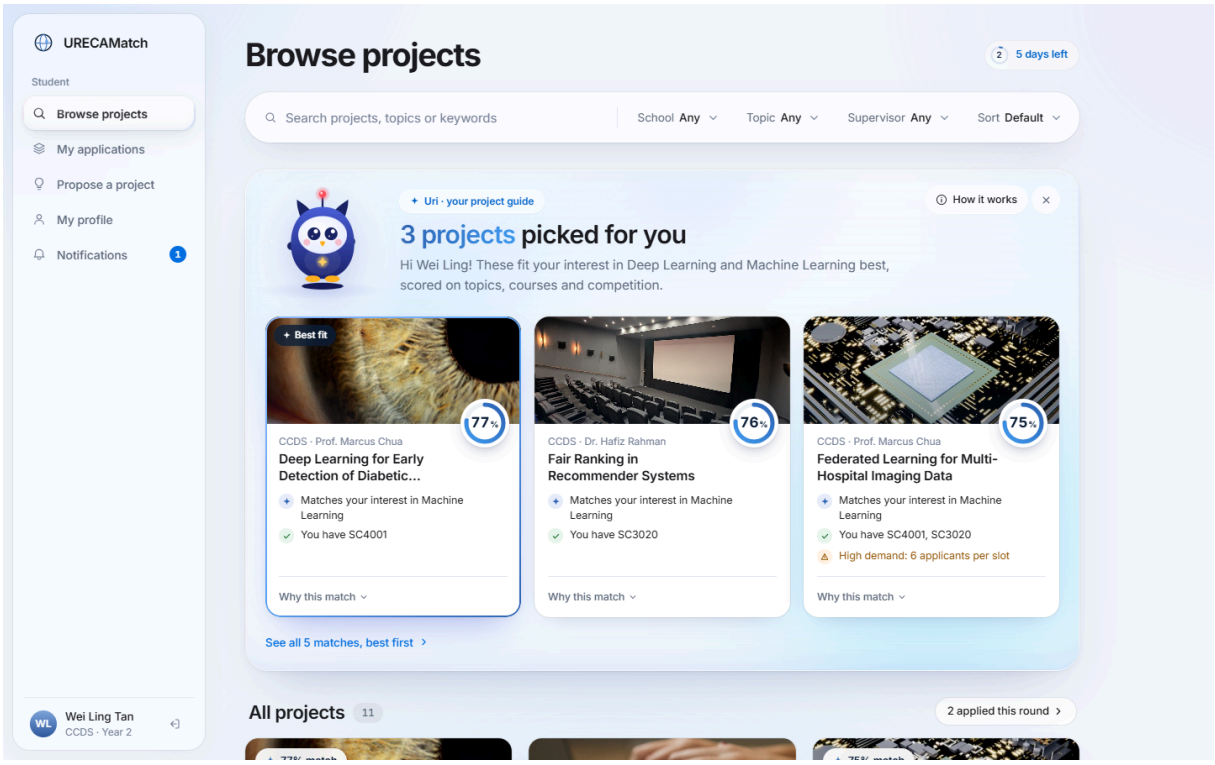


Figure 3.13 BrowseProjectPage

Search, with School, Topic and Supervisor dropdowns and a sort. A panel of picks from the Uri project guide shows each pick's match score and the reasons for it. Each card shows open slots, the number of applicants, a High demand label and its match score. The cycle status capsule is shown top right. While applications are not open, cards are shown locked with the reason.

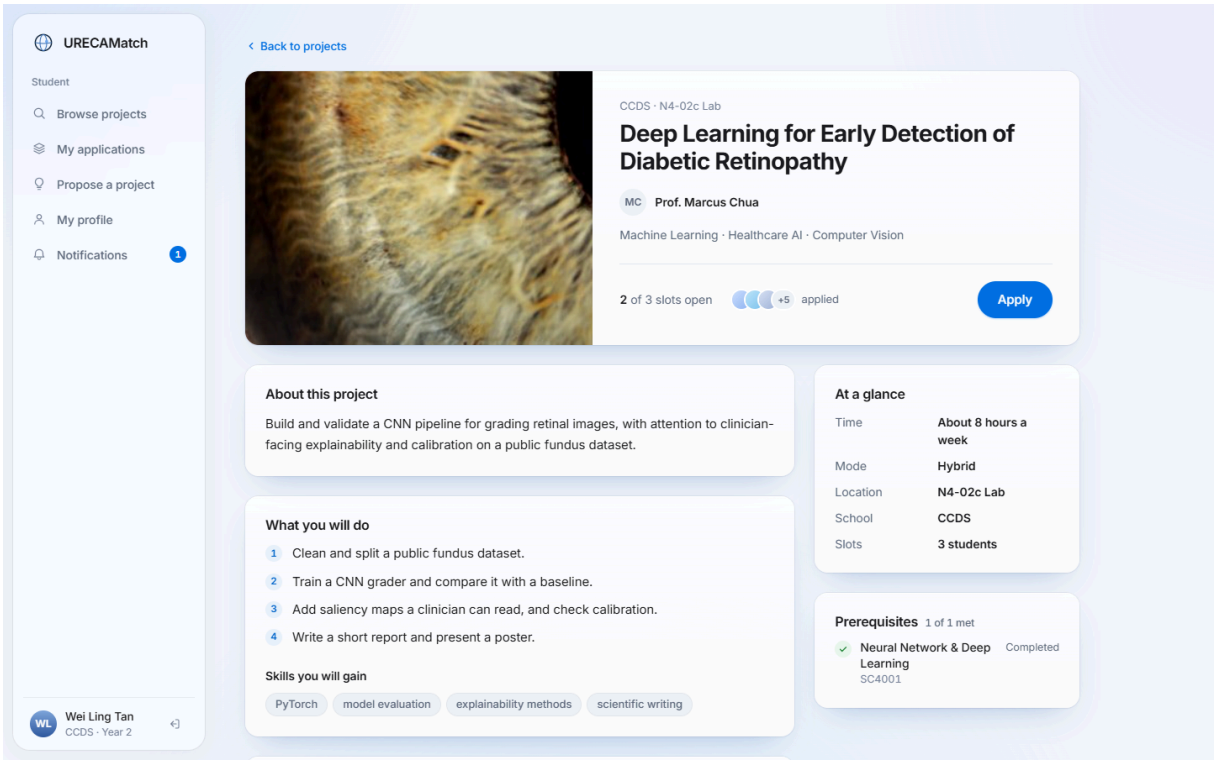


Figure 3.14 ProjectDetailPage

Full description, tasks and outputs, an at-a-glance panel (hours per week, mode, location, slots), photos and a video, prerequisites checked against the student's completed modules (each shows the course title and Completed or Missing), slots and the Apply action.

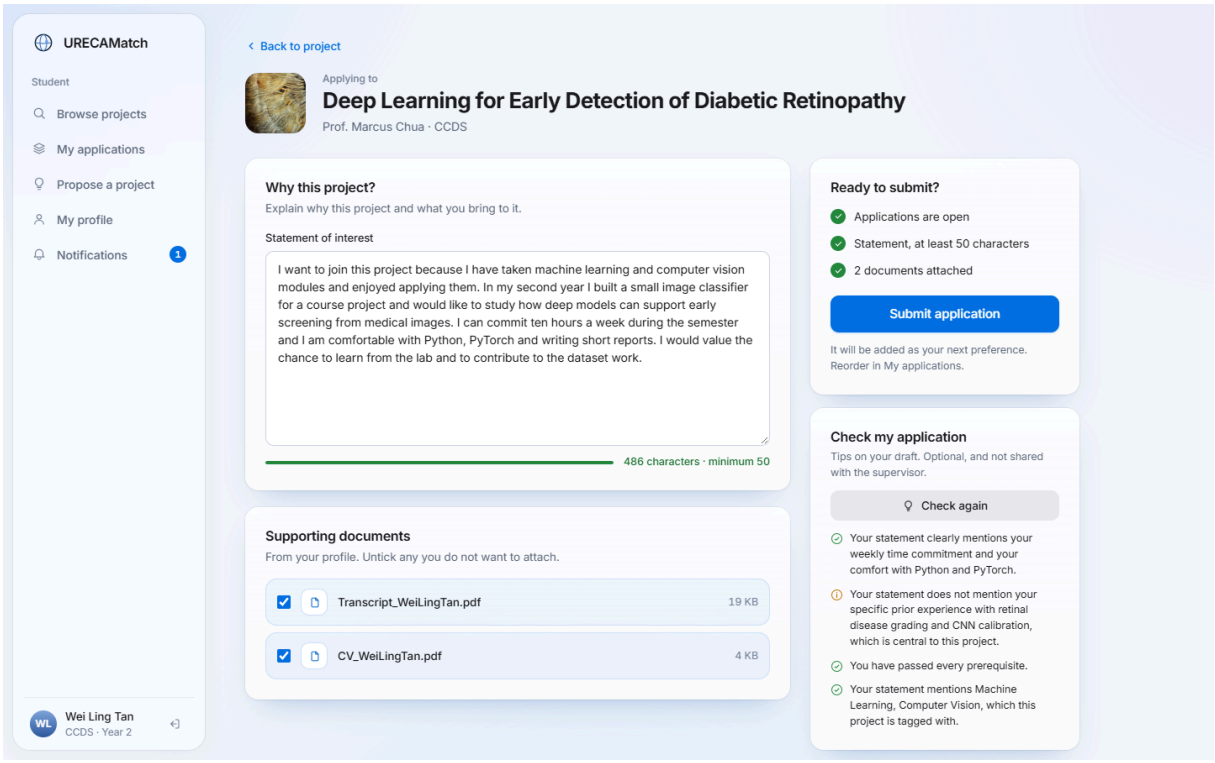


Figure 3.15 ApplyProjectPage

A project header, the statement of interest with a bar toward the 50-character minimum, documents attached from the profile, and a "Ready to submit?" checklist that holds the Submit button. A "Check my application" panel shows tips on the draft; it never gives a score and saves nothing.

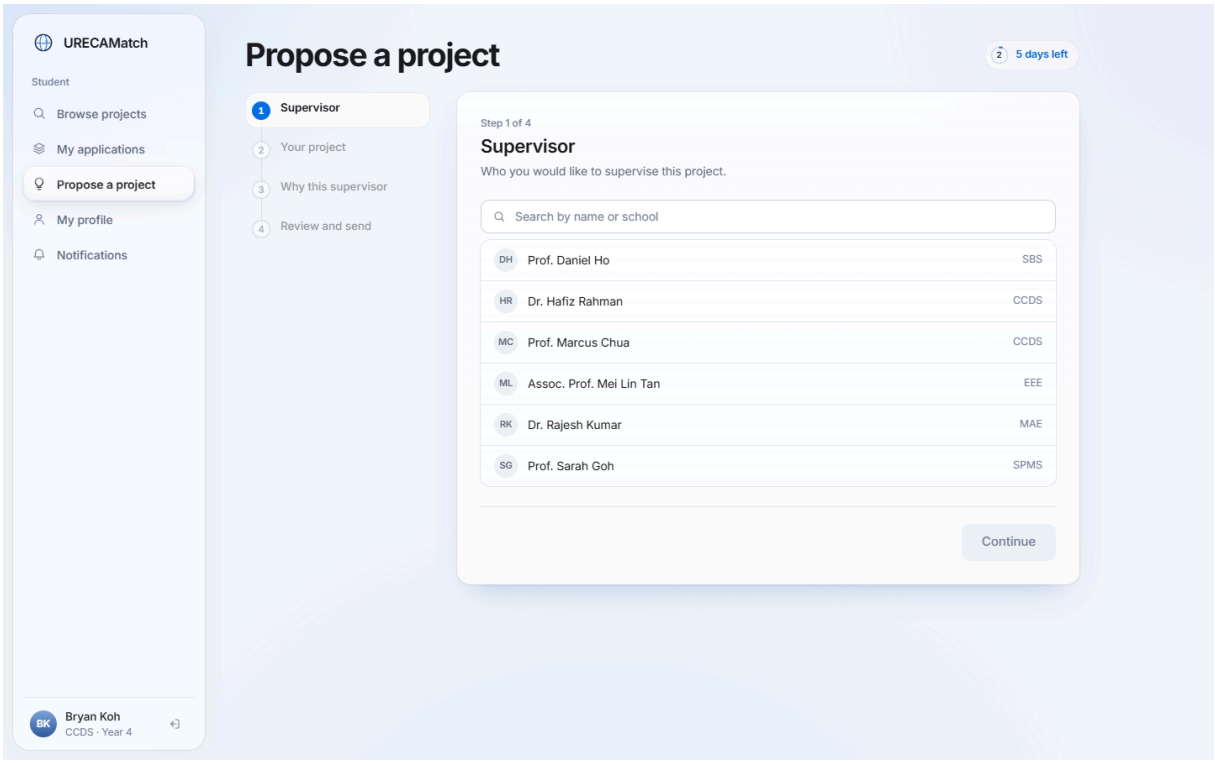


Figure 3.16 ProposeProjectPage

A guided flow in four steps: Supervisor, Your project, Why this supervisor, and Review and send. A step rail on the left shows the answers so far and lets the Student reopen a finished step.

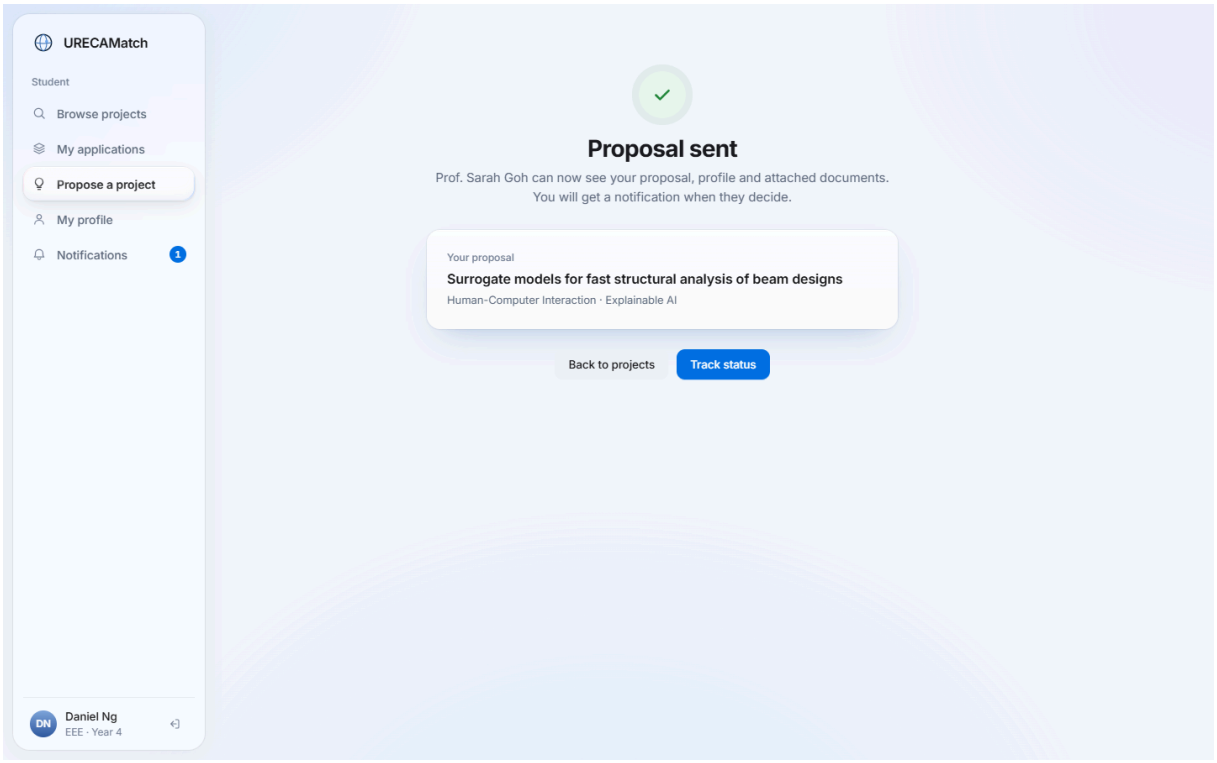


Figure 3.17 ProposalSuccessPage

Confirmation that the proposal was sent, naming the supervisor, with links back to projects and to track its status.

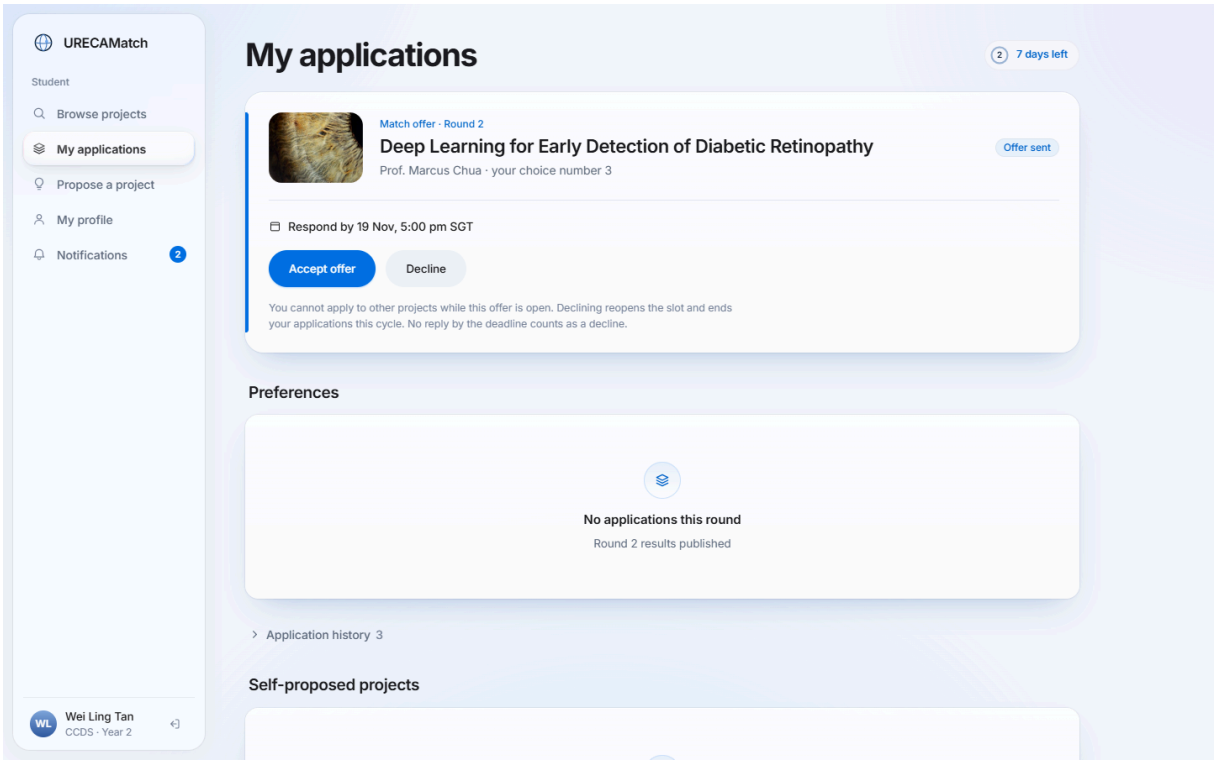


Figure 3.18 MyApplicationPage

Match offer with its answer-by date and Accept and Decline, the ranked applications (move up or down, withdraw), proposals with their status and, if declined, the reason, and earlier results under Application history. A Student who holds a project sees no preferences.

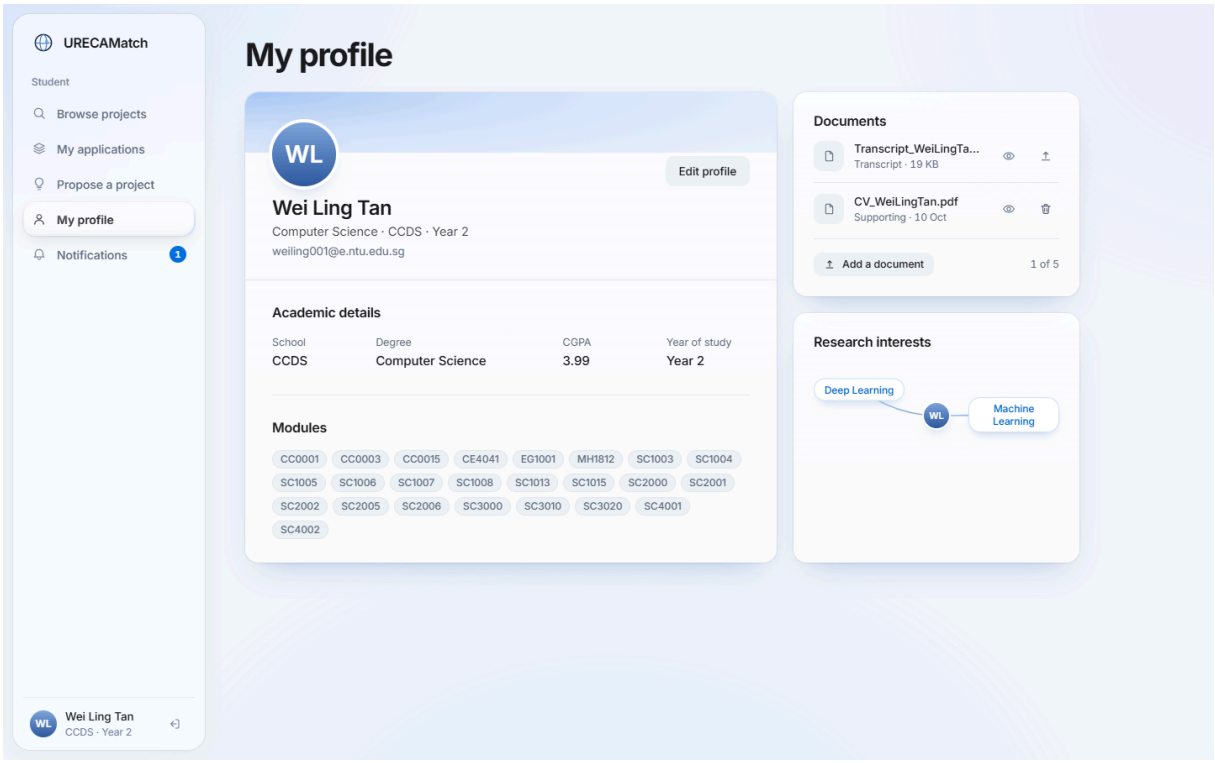


Figure 3.19 MyProfilePage

A profile card with academic details and modules, research interests drawn as a network, and documents with view, replace and delete. Edit profile opens the fields in place.

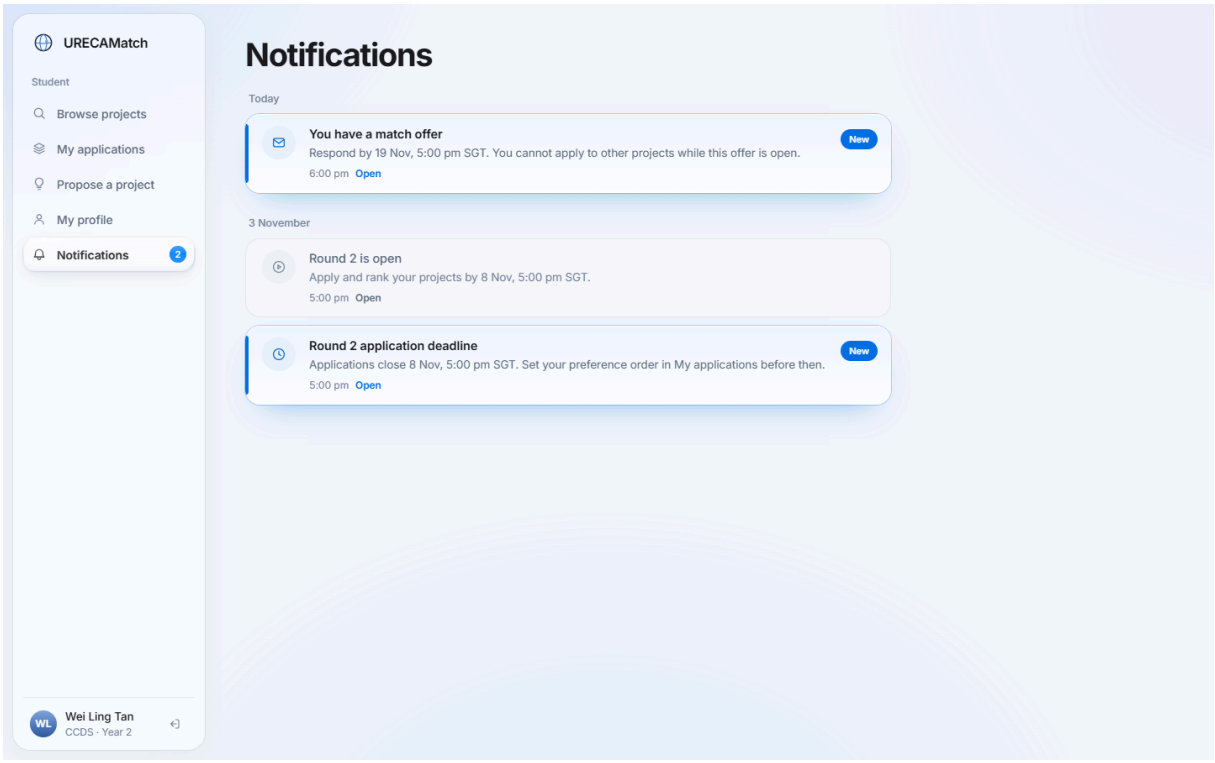


Figure 3.20 NotificationPage

Notifications grouped by day, each with an icon for its kind of event. Unread items are highlighted; opening one marks it read.

Project Supervisor

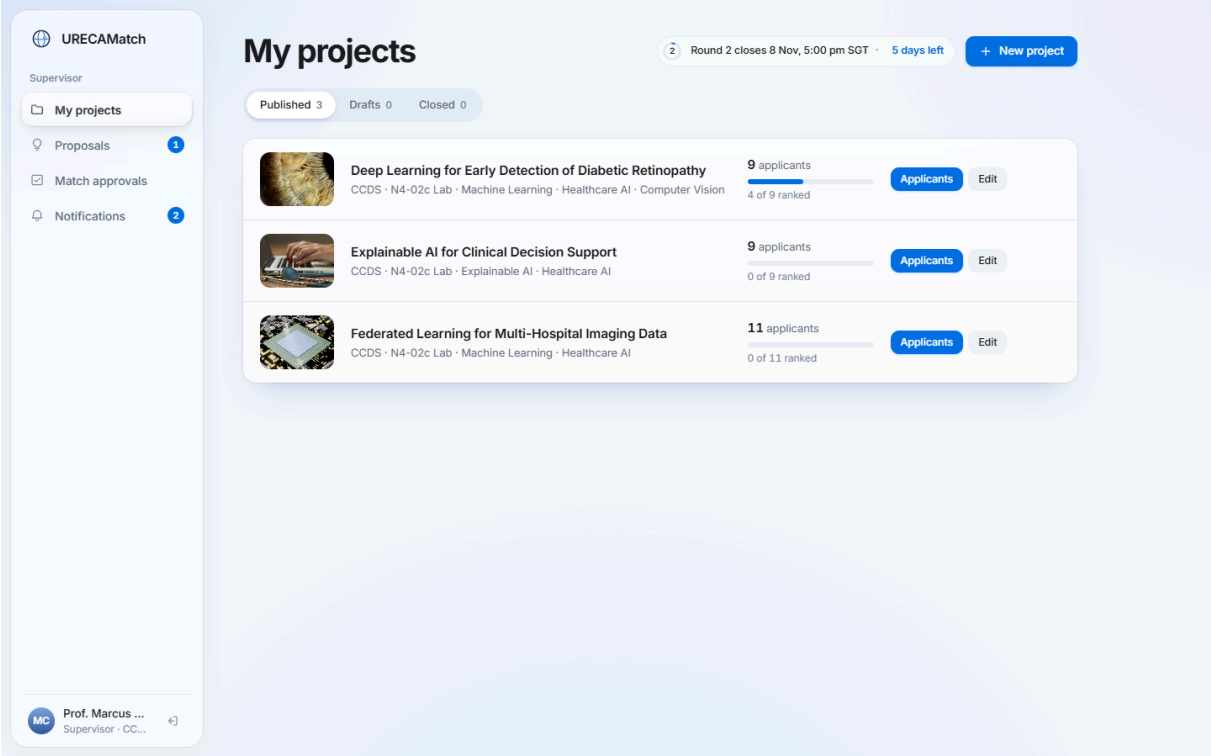


Figure 3.21 SupervisorProjectsPage

Published, Drafts and Closed tabs and the status of the current window. Each row shows the project's cover, its applicant count, how many applicants are ranked, and the Applicants and Edit actions; closing a project asks for confirmation first.

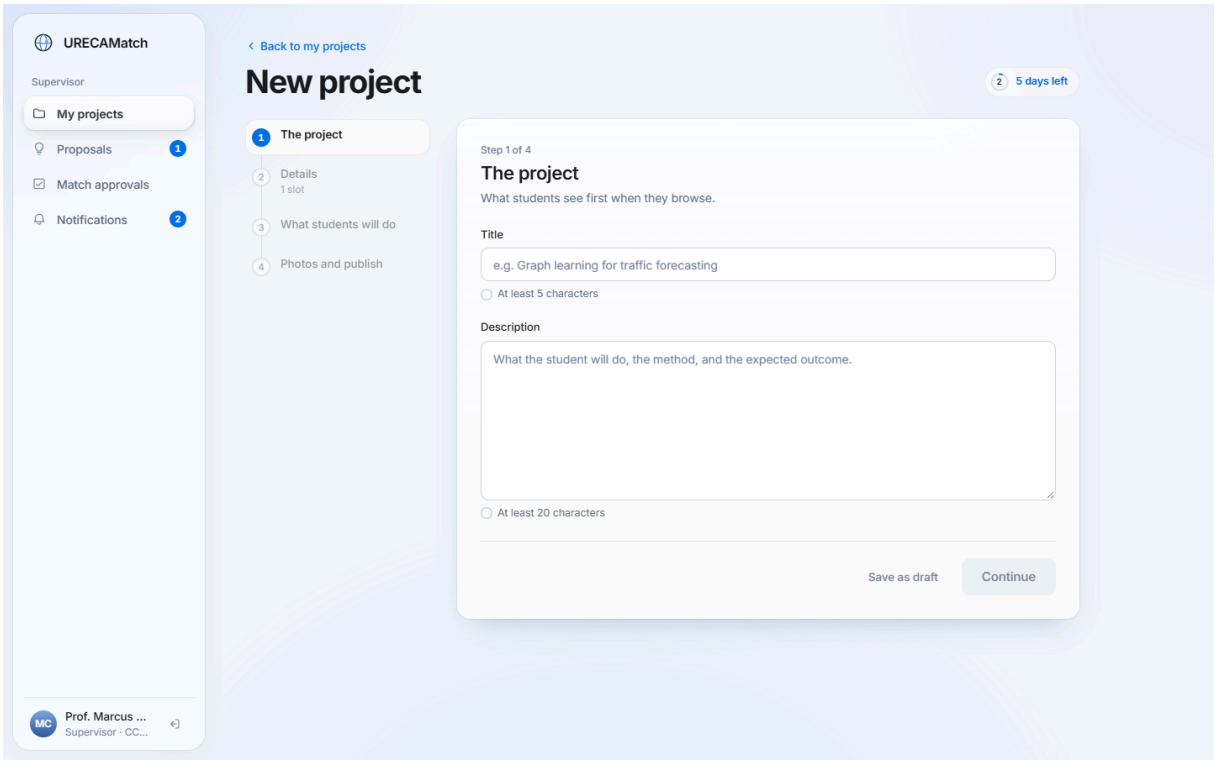


Figure 3.22 CreateProjectPage

A guided flow in four steps (the project, details, what students will do, photos and publish), with Save as draft on every step.

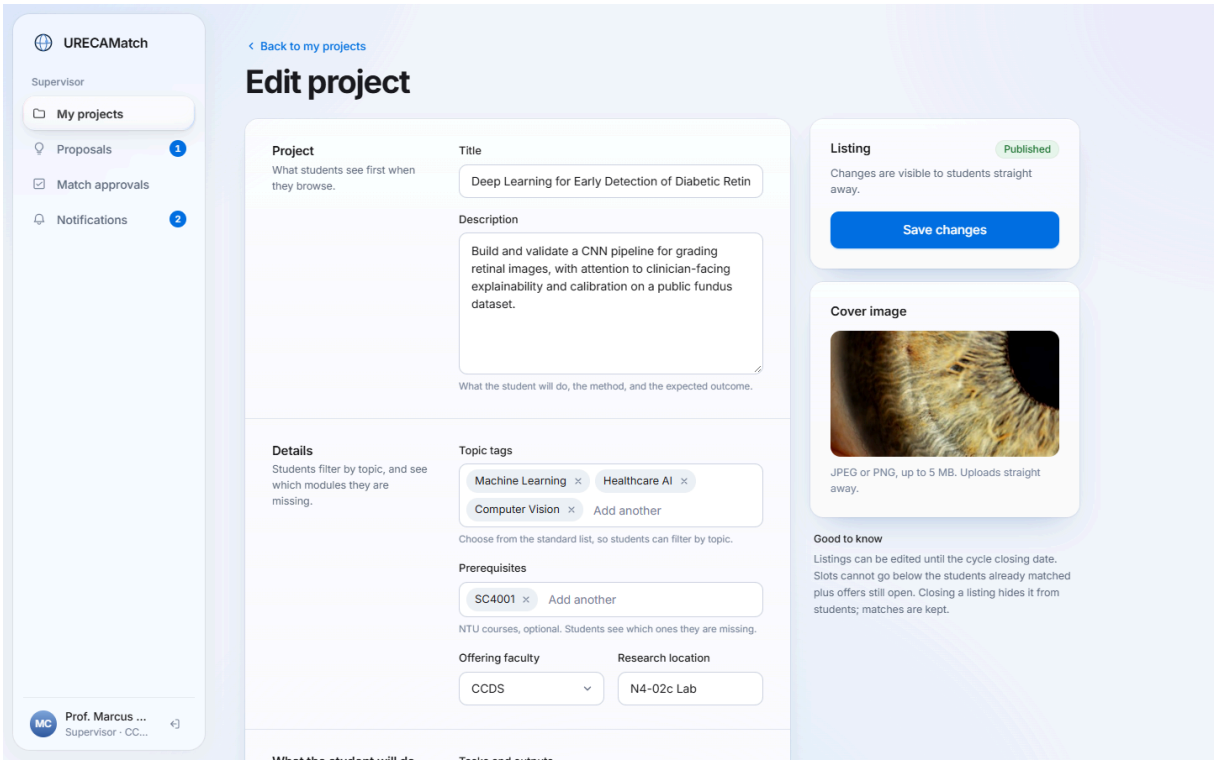


Figure 3.23 EditProjectPage

The same fields as CreateProjectPage, prefilled. Slots cannot go below the students who accepted plus the offers still open, and a closed project cannot be edited.

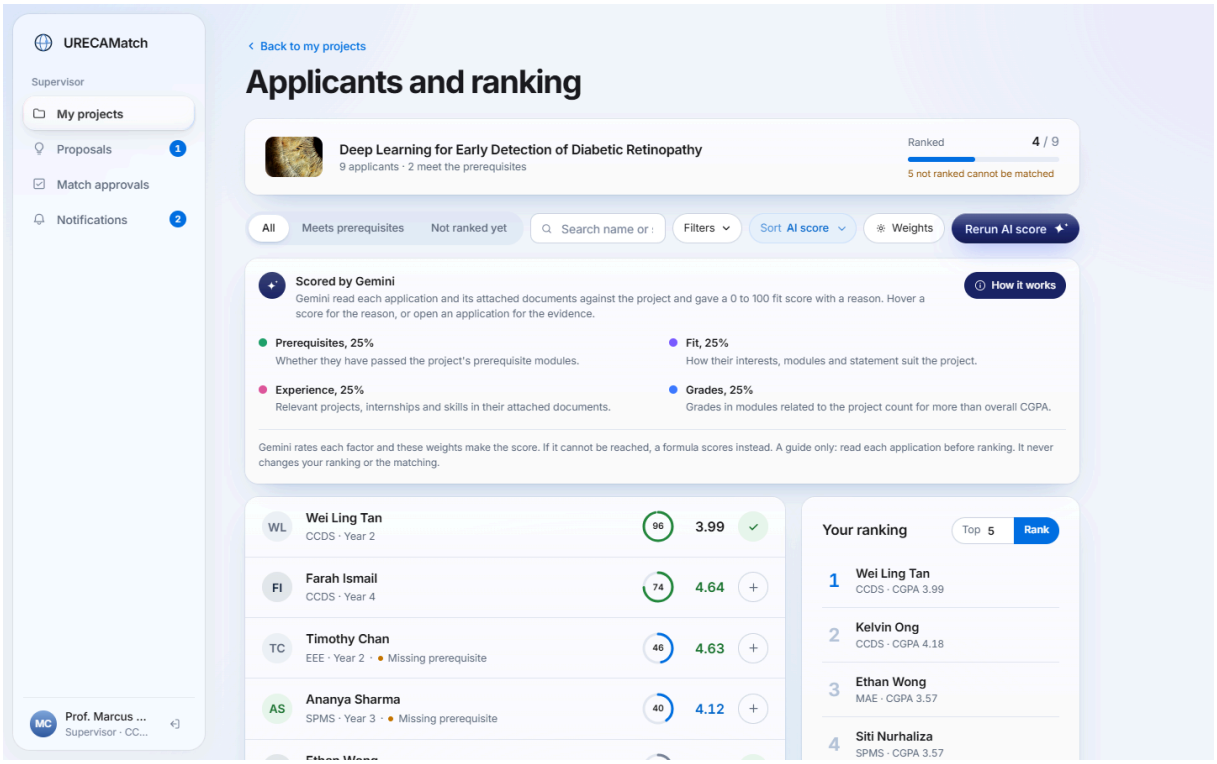


Figure 3.24 ApplicantAndRankingsPage

Applicants searched, filtered by CGPA, school, year and prerequisites, and sorted by CGPA, AI score or name, beside the supervisor's ranking. "AI score" rates each applicant on four factors and shows the reason and the quotes behind each score. The Weights panel sets how much each factor counts for the project, and students never see it. Rank top N fills the ranking from the list as shown. The drawer shows one application and its documents.

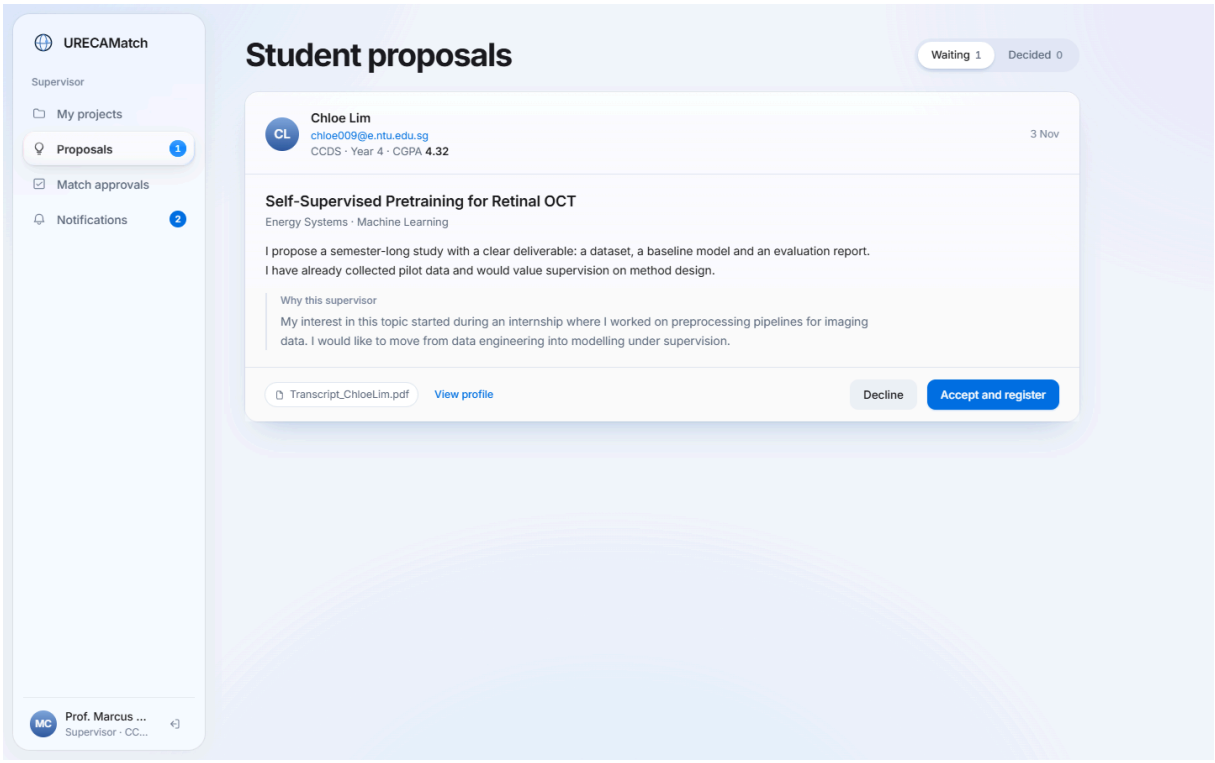


Figure 3.25 StudentProposalsPage

Incoming proposals, in Waiting and Decided tabs. The drawer shows the student's profile and attached documents. Declining asks for a reason of at least 10 characters, which the Student reads.

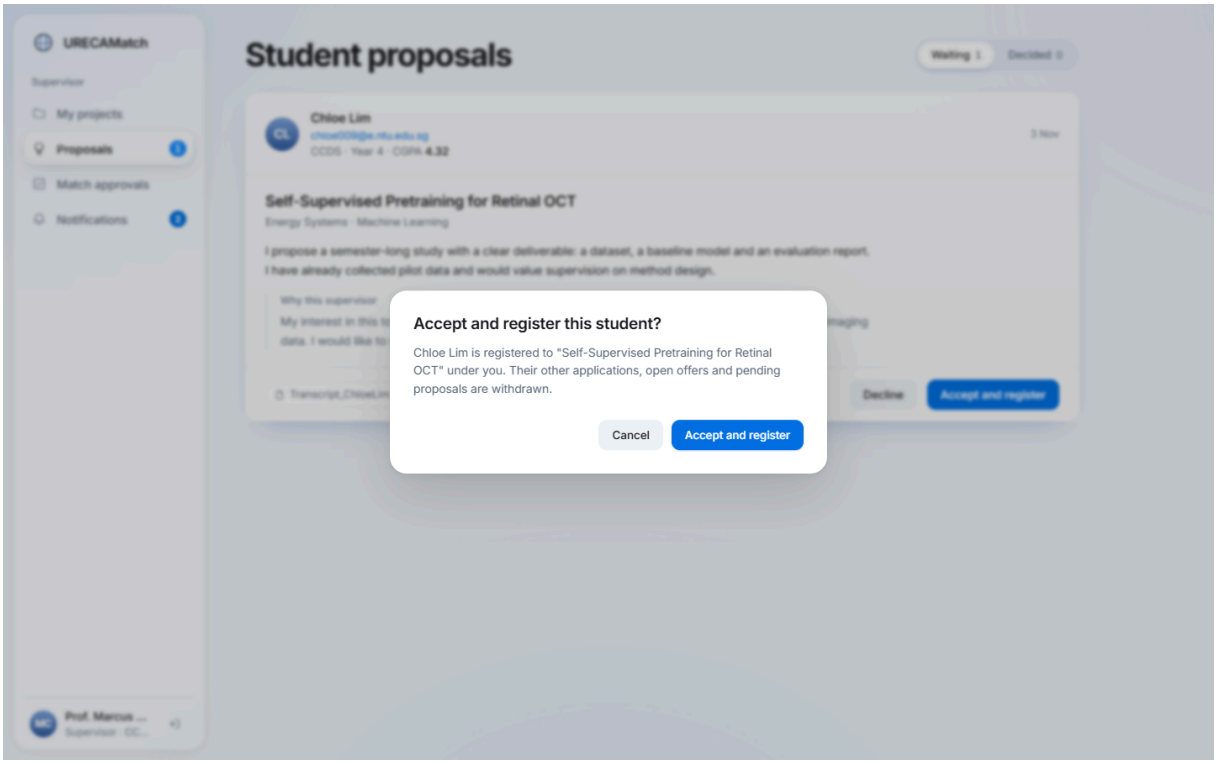


Figure 3.26 StudentProposalsPage, confirmation

Confirmation modal before accepting a proposal, stating its consequence.

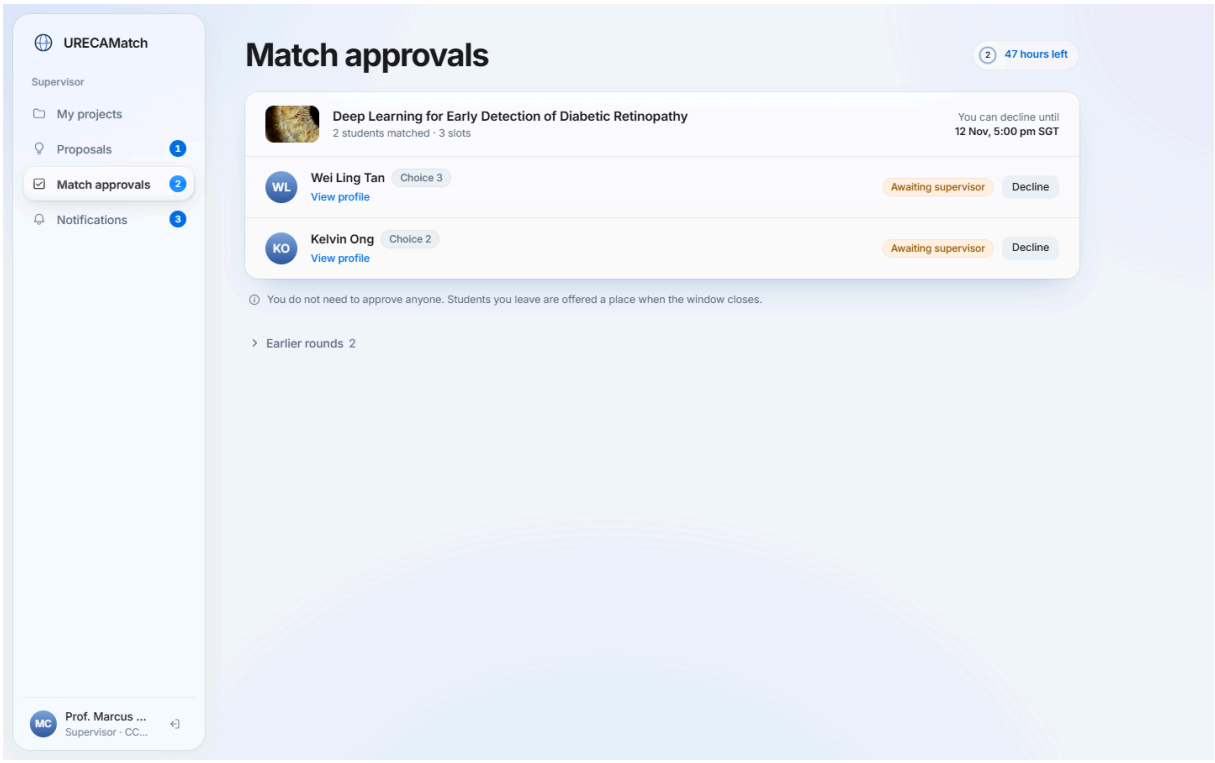


Figure 3.27 MatchApprovalsPage

Matched students grouped by project, with earlier rounds folded away. While an approval window is open, each match can be declined, with an optional reason; matches not declined are accepted when the window ends. View profile opens the student's application in a drawer.

Admin

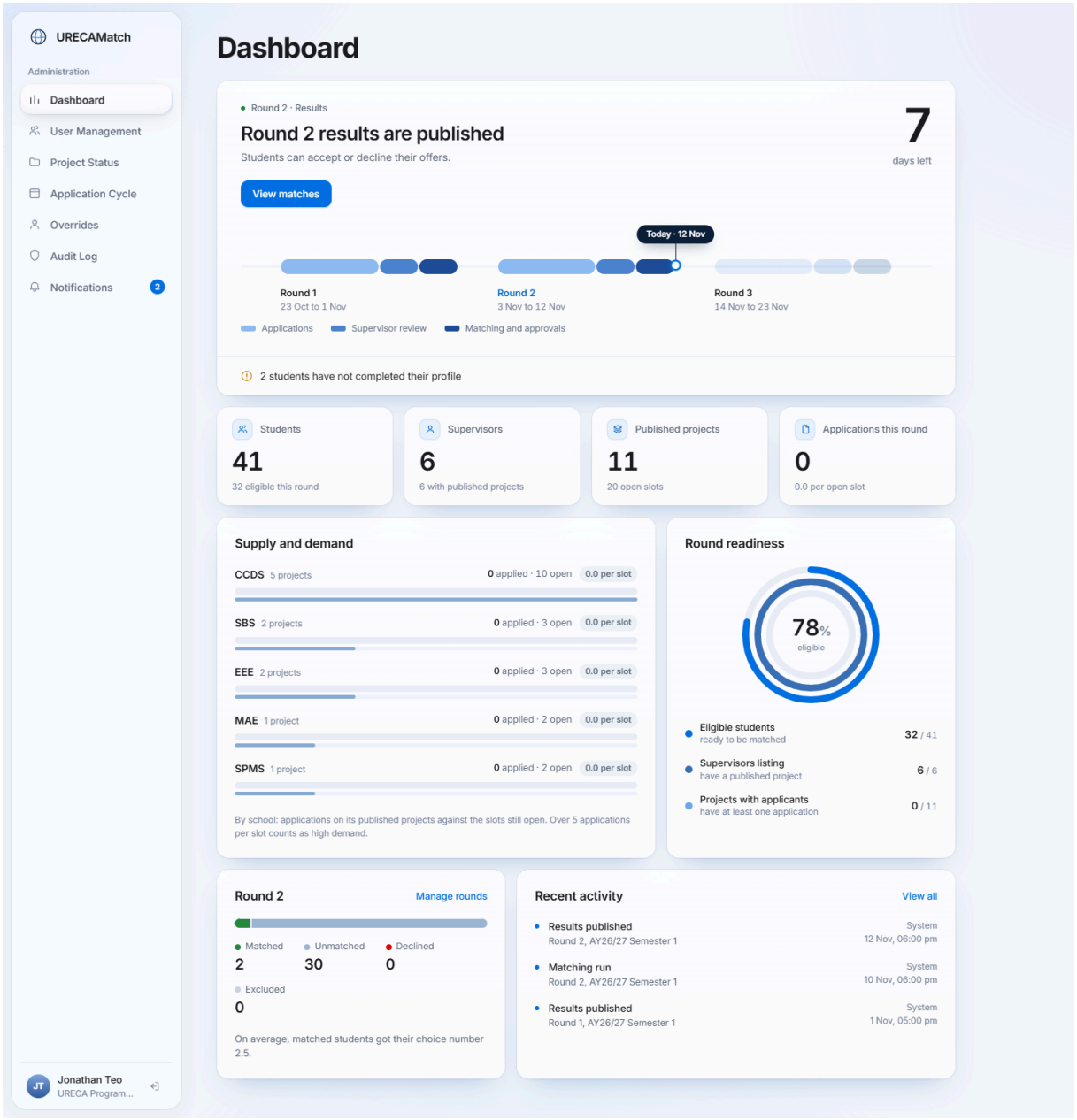


Figure 3.28 AdminDashboardPage

A "right now" panel (round state, time left and what needs attention), the cycle on one date axis, counts of students, supervisors, listings and applications, supply and demand per school, round readiness, the round's matching result, and the three latest audit entries.

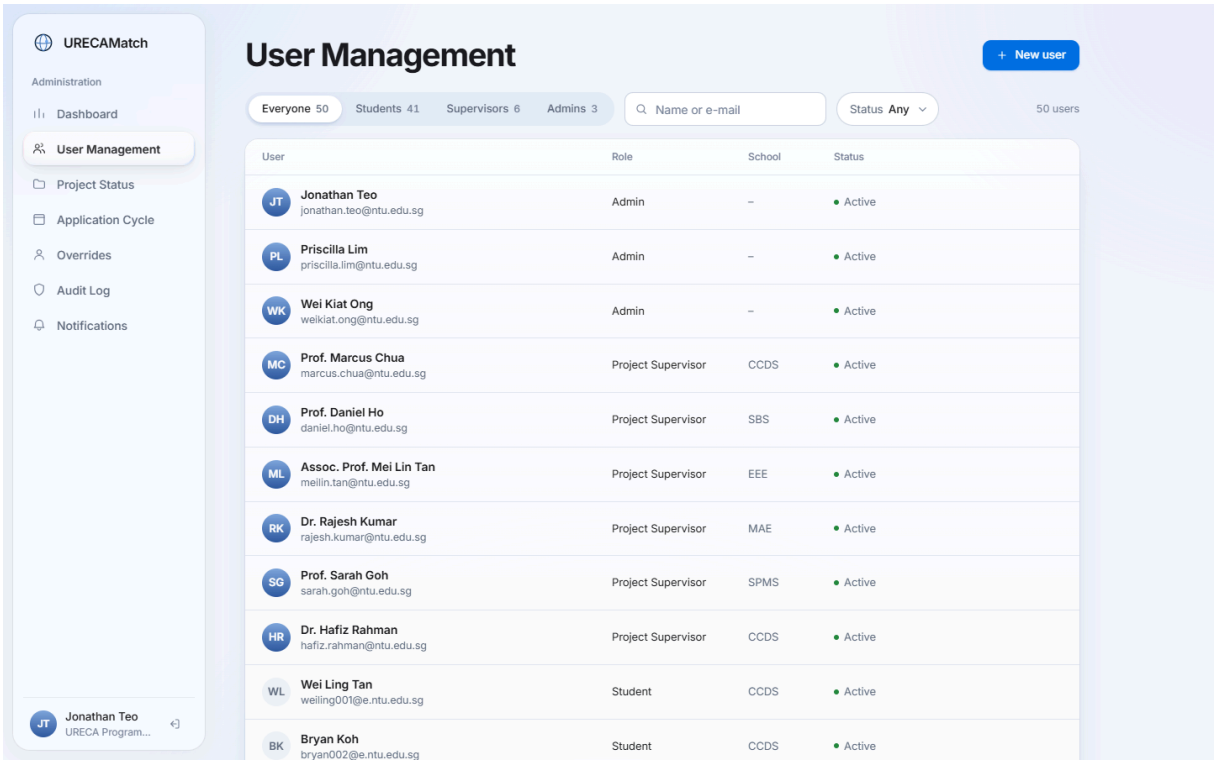


Figure 3.29 ManageUsersPage

Role tabs with counts, search and a status dropdown, 12 rows at a time. Edit and Deactivate appear on hover. Create, edit and deactivate accounts; an Admin cannot change or deactivate their own account.

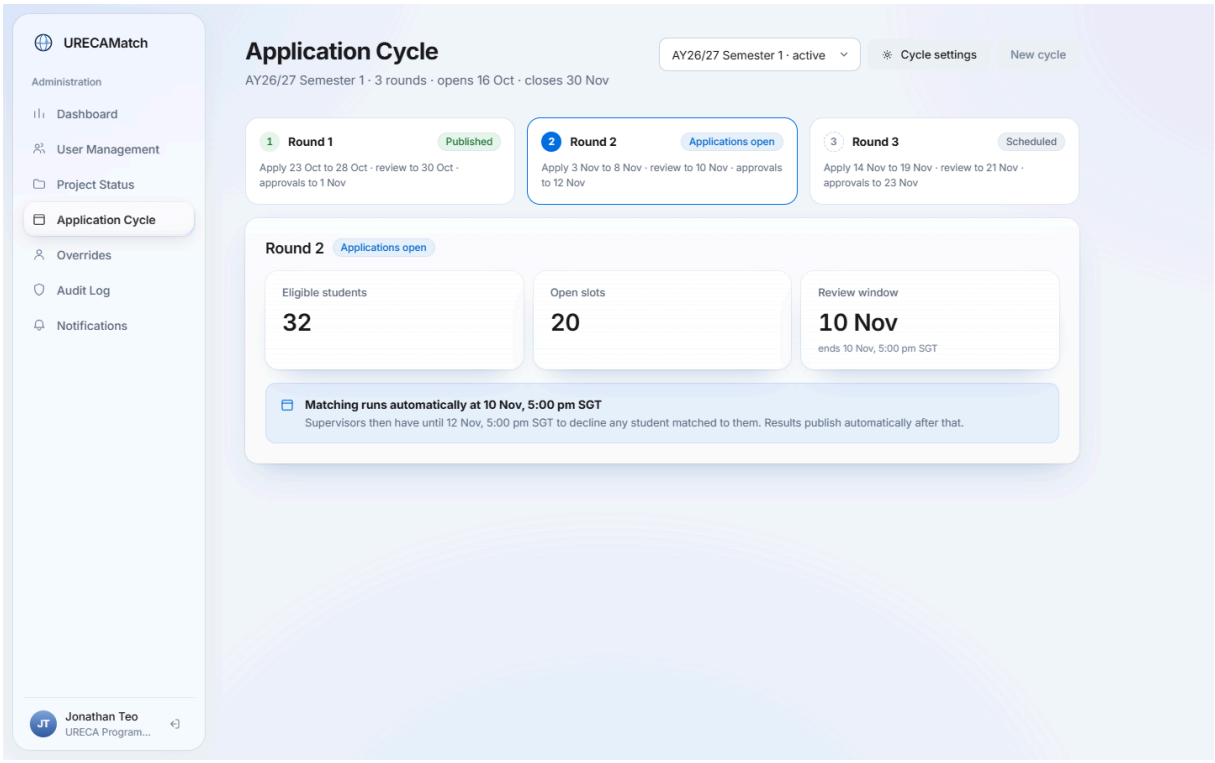


Figure 3.30 MatchingRoundsPage

Cycle selector and settings, the round timeline and the current round summary. Round settings take the length of the application, review and approval windows, optional start dates and the rollover choice. Matching and publishing run by themselves, so there is no Run matching button.

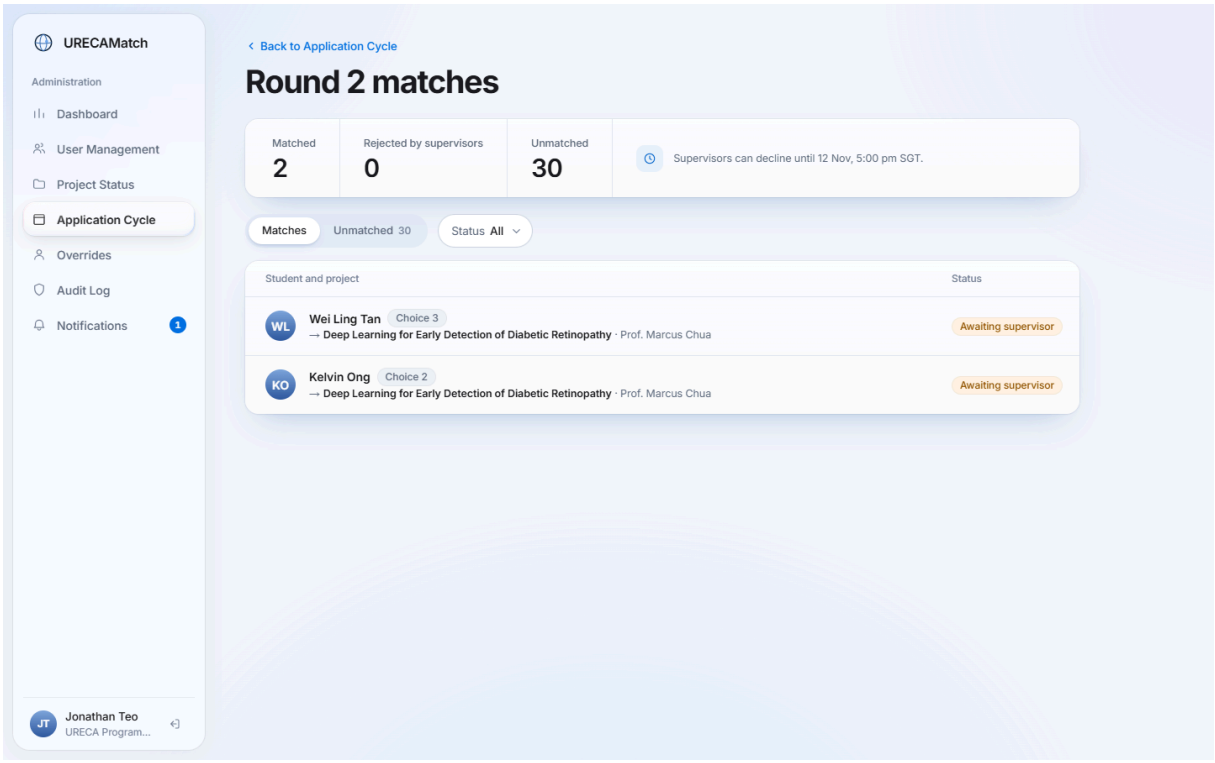


Figure 3.31 MatchingAwaitPublishPage

Read-only matches of one round: a summary, Matches and Unmatched tabs with a status dropdown, declines with their reasons, and an Assign link on each unmatched student. Results publish by themselves when the approval window ends.

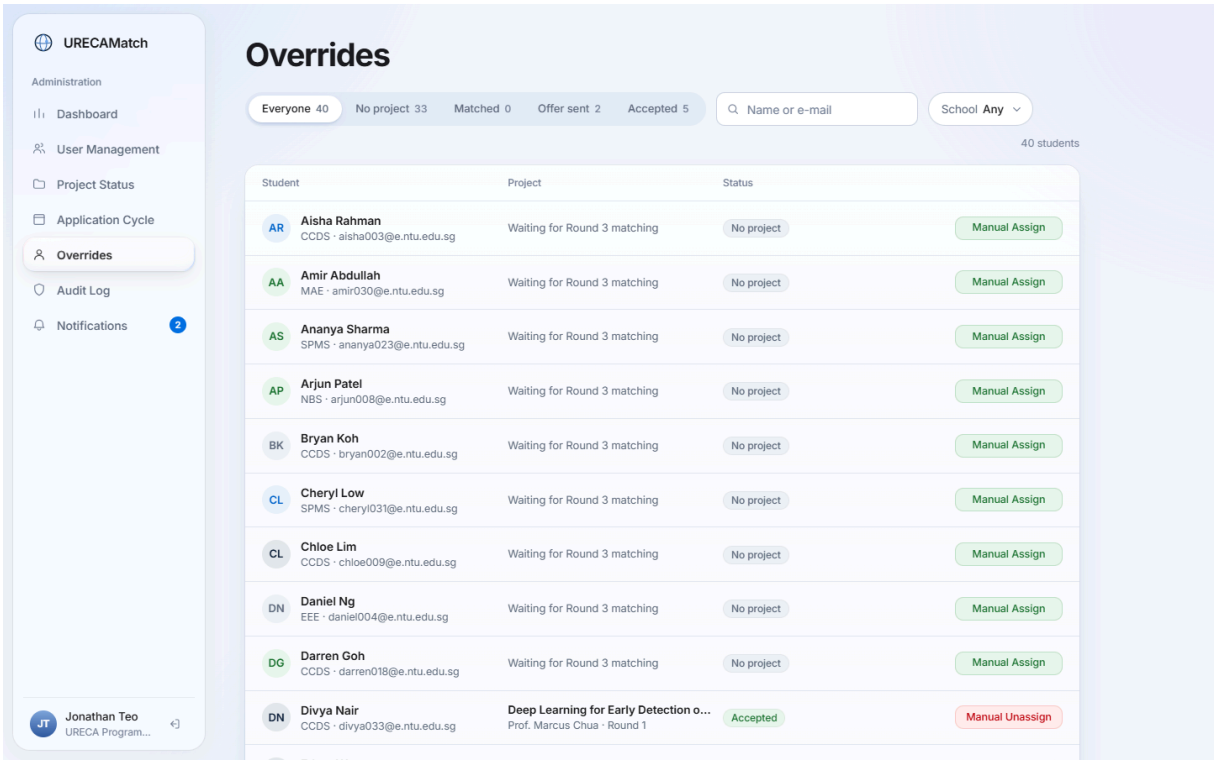


Figure 3.32 AssignmentsPage

Every active student in the cycle with where they stand (accepted, offer sent, matched, or unassigned with the reason), and Assign, Reassign and Unassign for each, each with a justification. The sidebar item and page title read "Overrides".

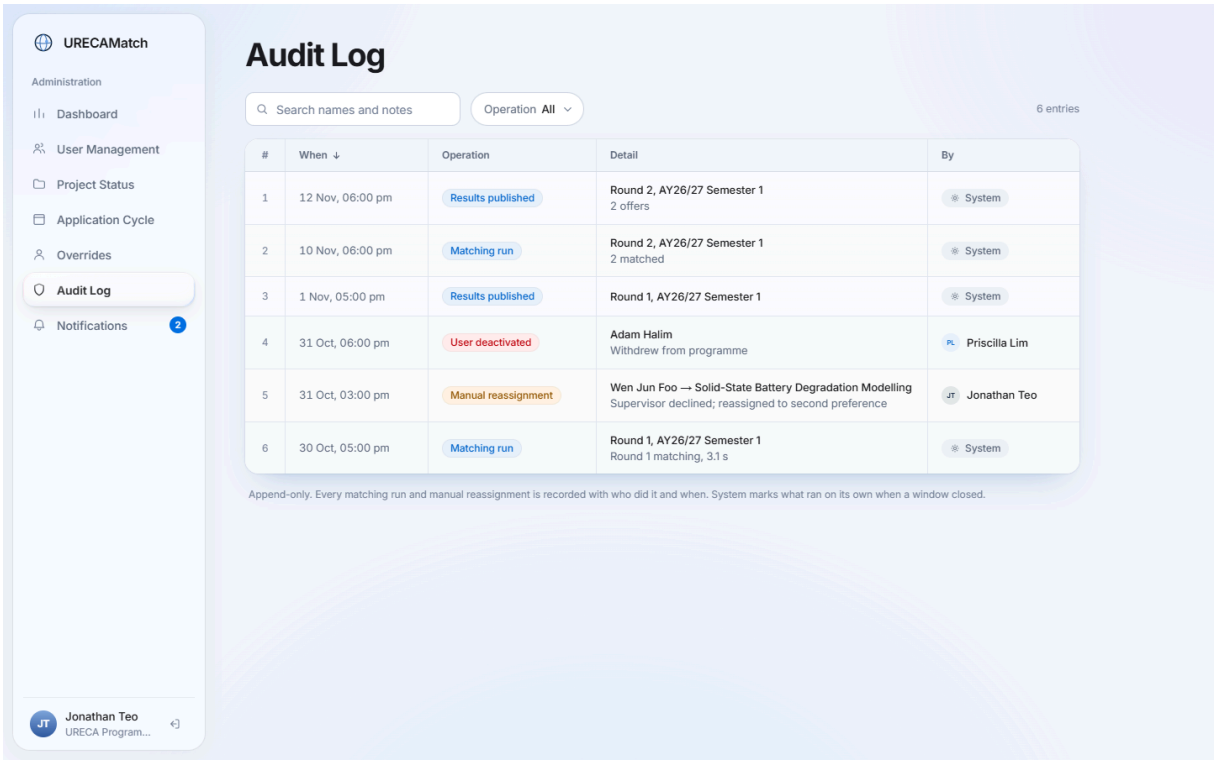


Figure 3.33 AuditOverridesPage

Append-only log of operations, newest first, 50 at a time with Load more, with search and an operation dropdown. Actions taken by the system are shown as System.

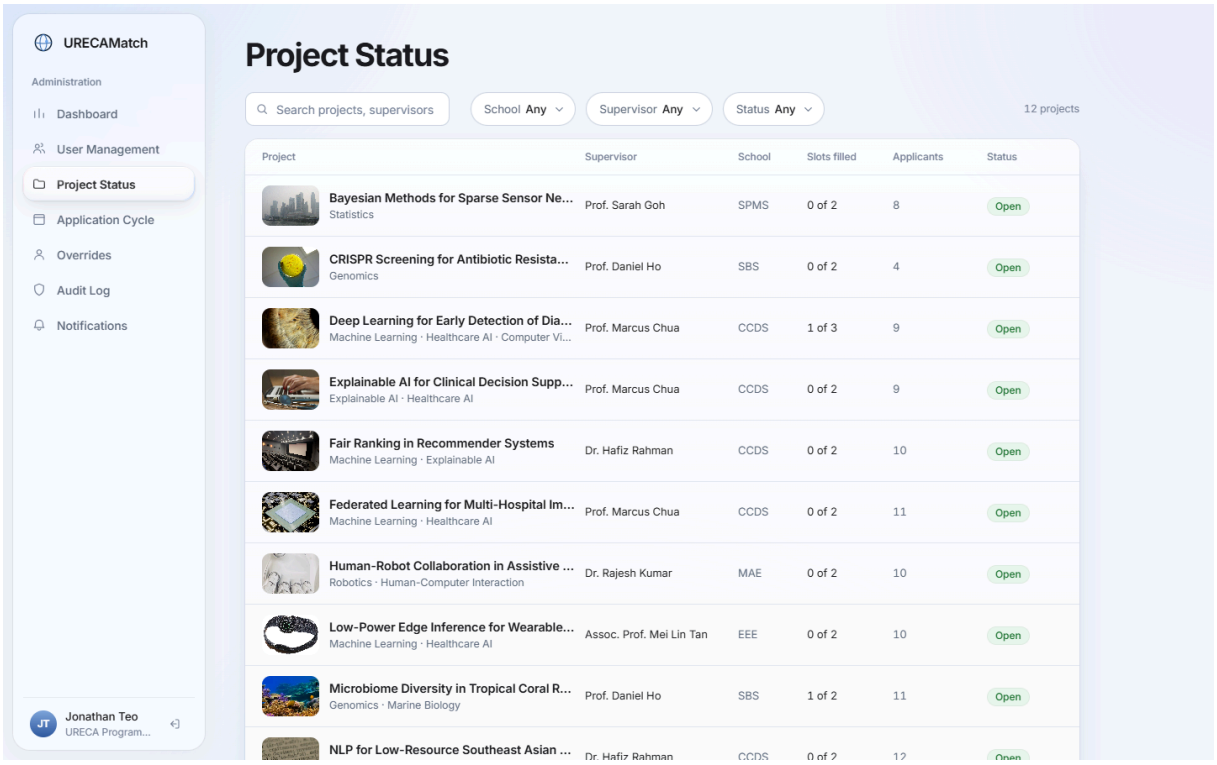


Figure 3.34 ProjectStatusPage

Every project of the active cycle with its supervisor, school, slots filled, applicants and status, with a search box and School, Supervisor and Status filters. Opening a project shows it as students see it, with the students it holds and their status. The page is read only; moving a student is done on the Overrides page.

3.2 Hardware Interfaces

URECAMatch requires no dedicated hardware. It runs in a web browser on the user's own computer and on cloud servers operated by its hosting providers (3.3), and it makes no use of a camera, microphone, location sensor, biometric sensor or printer.

3.2.1 Supported Devices

The system supports desktop and laptop computers running a current version of Chrome, Edge, Safari or Firefox, in a browser window at least 1200 px wide, with a keyboard and a mouse or touchpad. Tablets and smartphones are not supported in this release. The sidebar's refracting glass effect relies on a browser filter that Chrome and Edge support; Safari and Firefox show a plain frosted panel instead, and every function works the same in all four browsers.

3.2.2 Device Storage

The system reads a file from the user's device only when the user selects it in the browser's file picker. Students upload supporting documents, such as academic transcripts, as PDF files; Project Supervisors upload listing cover images and photos as JPEG or PNG files. Each file may be at most 5 MB, and any other file is rejected with a message giving the reason.

Documents are viewed inside the browser, so no other software is needed, and nothing is written to the device unless the user chooses to download a file.

3.3 Software Interfaces

URECAMatch consists of two applications built by the team, a web client and an API server, and depends on the external services shown in Figure 2.1, provided by Supabase (database, authentication and file storage), Vercel, Resend and Google Gemini (Figure 3.35). The web client presents the interface and holds no data of its own. The API server holds every business rule, checks the user's role on every request, and is the only component that reads or writes data. The two applications share one contract, an OpenAPI document generated from the API, from which the web client's data types are also generated, so that both always agree on every request and response.

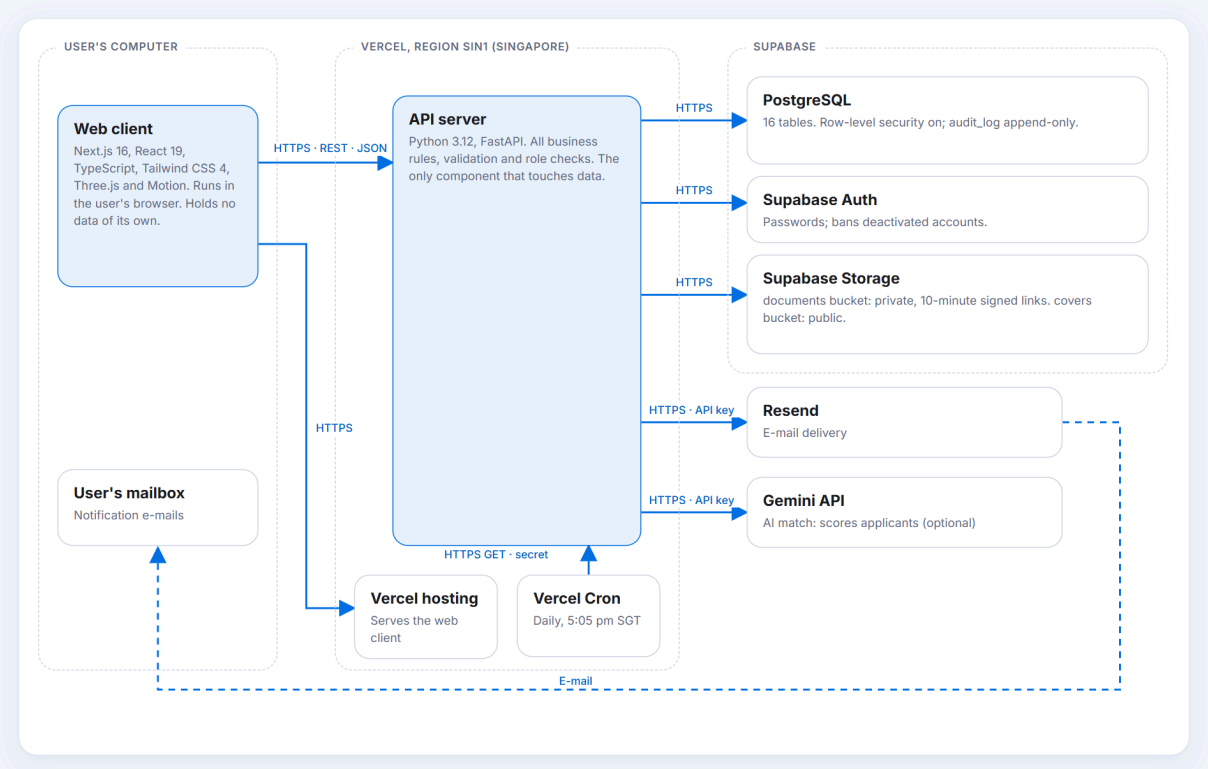


Figure 3.35 Software interfaces

3.3.1 Components and Data Exchanged

Table 3.2 lists each component with its product and version, its purpose and the data exchanged with it.

Table 3.2 Software components and the data they exchange

Component	Product and version	Purpose	Data exchanged
Web client	Next.js 16.3, React 19.2, TypeScript 5, Tailwind CSS 4, Three.js, Motion	Presents screens and navigation	Sends requests to the API server as JSON, as defined by the API contract
API server	Python 3.12, FastAPI 0.115 or later, Pydantic 2	Applies business rules, validation and role-based access control	Reads and writes all records through the Store interface
API contract	OpenAPI 3.1 (shared/openapi.json)	Defines every endpoint and data structure	Source from which the web client types are generated
Database	PostgreSQL, managed by Supabase	Stores persistent records	16 tables, including the topic and course reference tables; the audit log table is append-only
Authentication	Supabase Auth	Manages credentials and account bans	E-mail address, password and account ban status
File storage	Supabase Storage	Stores documents and cover images	Private documents bucket, accessed through 10-minute signed links; public covers bucket
E-mail	Resend API, Python SDK 2.x	Sends notification e-mails	Recipient, subject and body, in batches of up to 100
AI service	Google Gemini API (gemini-3.5-flash-lite)	Scores applicants from 0 to 100 for a Project Supervisor and gives tips on a draft application	Listing and applicant details, and the text of supporting documents with e-mail addresses, phone numbers, matriculation numbers and NRIC or FIN numbers removed, over HTTPS; local scoring is used if the service is unavailable
Hosting	Vercel, Singapore region (sin1)	Hosts both applications and runs the scheduled daily job	One request per day to /internal/tick, which runs matching, publishes results, advances round status and times out unanswered offers

3.3.2 Document Storage

Supporting documents contain personal data and are therefore never publicly accessible (Figure 3.36). When a Student uploads a document, the API checks its type and size, reads a transcript to confirm that it states a Cumulative GPA and lists courses passed (REQ-PROF-4), stores the file in a private storage bucket under the Student's own folder, and records its name, size and location in the database. When a user asks to view a document, the API first confirms that the user is the owner, an Admin, or a Supervisor of a listing the Student applied to, and only then creates a signed link that expires after 10 minutes. The browser uses that link to show the document in the in-app viewer. Listing cover images contain no personal data and are served from a separate public bucket.

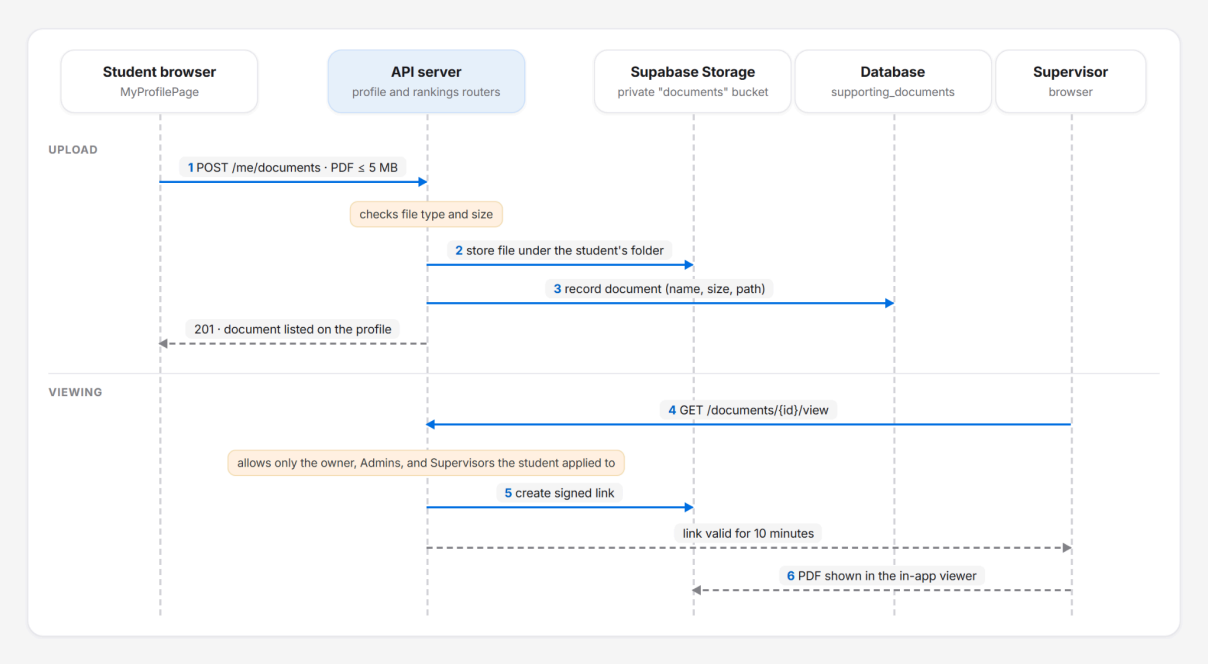


Figure 3.36 Document upload and viewing

3.3.3 Notifications and E-mail

Every notification is created by a single notification service, which no other part of the system bypasses (Figure 3.37). When an event that a user must know about occurs, such as published results, proposed matches awaiting approval, a decision on a proposal or a change to the cycle dates, the service stores one notification for each recipient and then passes them to the e-mail service, which sends them through Resend in batches of up to 100 within 15 minutes (NFR 5.1.2). The two channels are independent: if e-mail delivery fails, the failure is logged and the in-app notification remains, so that no notice is lost.

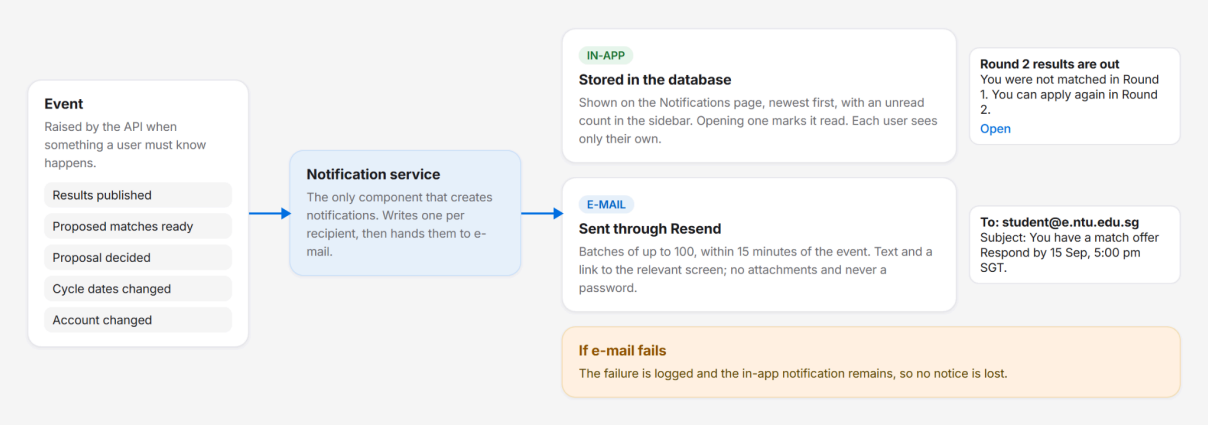


Figure 3.37 Notification flow

3.3.4 Design Constraints

Every software interface follows the same constraints. The API reaches all data through one Store interface, which is implemented on Supabase. Row-level security is enabled on every database table, so that browsers cannot read the database directly, and a database trigger rejects every change to or deletion of an audit log entry, including changes attempted by the API. The API contract and the web client's types are generated from the code, never edited by hand, and checked for differences on every build. Keys for Supabase, Resend, the AI score service and the scheduled job are held in server configuration and are never sent to the browser.

3.4 Communications Interfaces

All communication between the components of URECAMatch, and between the system and its users, uses HTTPS with TLS encryption. Dates and times are exchanged in ISO 8601 format in UTC and converted to Singapore time only for display.

3.4.1 Communication Channels

Table 3.3 lists each channel with its protocol, format and authentication.

Table 3.3 Communication channels

Channel	Protocol and format	Authentication	Notes
Browser to web client	HTTPS on port 443; HTTP on port 80 redirects to HTTPS	Not required	Delivers the application
Web client to API	REST over HTTPS; JSON in UTF-8; files as multipart/form-data	Session token in an Authorization: Bearer header	Accepted only from the URECAMatch web addresses (CORS)
API to Supabase	HTTPS through the Supabase client	Service key, server only	Database, authentication and file storage

Channel	Protocol and format	Authentication	Notes
API to Resend	HTTPS REST	API key, server only	Outgoing e-mail
API to Gemini	HTTPS REST; JSON	API key, server only	AI scores and application tips
Vercel Cron to API	HTTPS GET, daily at 5:05 pm Singapore time	Shared secret	Runs matching after a review window closes, publishes results after an approval window closes, advances round status; times out unanswered offers
E-mail to users	Sent through Resend	Not applicable	Within 15 minutes of the event (NFR 5.1.2); text and a link, no attachments, never a password

3.4.2 Message Format, Sessions and Errors

The web client and the API exchange JSON request and response bodies in UTF-8, as defined by the API contract (Figure 3.38). Signing in returns a session token, which the web client sends with every later request. A session ends after 30 minutes without activity (REQ-AUTH-3), after which the API rejects the token and the user is returned to the sign-in page. A new account is activated with a 6-digit verification code sent to the user's NTU e-mail address, which expires after 10 minutes and can be requested again (REQ-REG-2, REQ-REG-4).

Every failed request returns an HTTP status and a JSON body containing an error code, a plain-English message and, where the problem lies in one input, the name of that field. The status identifies the kind of failure: 401 when the user is not signed in or the session has expired, 403 when the action is not allowed for the user's role or the account is inactive, 404 when a record does not exist or is not visible to the user, 409 when the request conflicts with the current state, such as applying twice to the same project, and 422 when an input is invalid. The web client shows the message to the user (3.1.6), while the error code allows the program to respond to the failure.

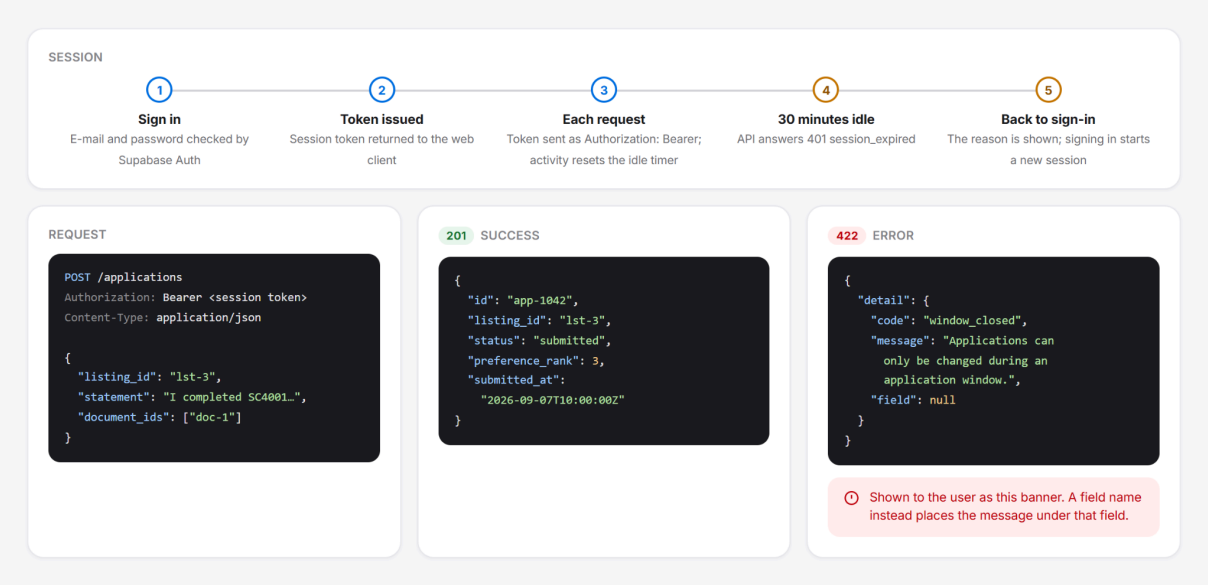


Figure 3.38 Requests, sessions and errors

3.4.3 Security and Performance

Passwords are checked by Supabase Auth and are never stored or handled by URECAMatch itself, and passwords and keys are never included in e-mail. Each channel to an external service is authenticated with a key held only on the server. Uploaded files are limited to 5 MB, and responses are designed to meet the responsiveness targets in NFR 5.1. The web client does not receive pushed updates; it fetches current data, including the unread notification count, each time the user opens or refreshes a screen.

4. System Features

This section describes the system features of URECAMatch, grouped by the user role that initiates them: Admin (4.1 to 4.8, 4.22), Student (4.9 to 4.14), Project Supervisor (4.15 to 4.18) and all users (4.19 to 4.21). For each feature it gives a description and priority, the stimulus/response sequences, and the functional requirements.

Each functional requirement has a unique identifier of the form REQ-AREA-n. The identifier is used to trace the requirement to its use case, screen (Table 3.9), code and test. High priority means the feature is needed for a matching cycle to run; Medium means it is important but a cycle can run without it.

Admin Features (4.1 to 4.8)

4.1 ManageApplicationCycle

4.1.1 Description and Priority

Description

This feature enables an Admin to create a new Application Cycle or modify an existing one, configuring time parameters such as start and end dates. A cycle ends on its closing date. There is no action to close or reopen a cycle; to end it sooner the Admin moves its closing date.

Priority

High; plays a critical role in setting up and adjusting the timeline for URECAMatch application periods.

4.1.2 Stimulus/Response Sequences

1. After the Admin successfully signs in, the Admin navigates to Application Cycle and creates a new cycle, or opens Cycle settings for the selected cycle.

a) Admin selects the option to create a new application cycle.

i) The System displays the Application Cycle configuration where the Admin sets the academic year, the application opening and closing dates, and the pre-application viewing period.

ii) The System validates the provided parameters and saves the newly created Application Cycle upon confirmation. A new cycle can be created only after the current cycle's closing date has passed. The previous cycle then moves to the history, where it can be viewed but not changed.

b) Admin selects the option to modify an existing application cycle.

i) The System displays the configuration details of the selected application cycle.

ii) The System allows the Admin to change the start and end dates for the active or upcoming cycle before those dates have passed.

4.1.3 Functional Requirements

- **REQ-CYCLE-1** The system shall allow an Admin to create an application cycle by setting its academic year, opening date, closing date and pre-application viewing period (0 to 60 days). The system shall reject the cycle if its closing date is not after its opening date, if its dates overlap another cycle, or if the current cycle's closing date has not passed. Creating a cycle shall move the previous cycle to the history, where it can be viewed but not changed, and shall close any of its projects that are still published.
- **REQ-CYCLE-2** The system shall allow an Admin to modify an existing application cycle's start and end dates before those dates have passed. Matching Rounds that have not started shall move with a changed opening date and keep their spacing.
- **REQ-CYCLE-3** When the closing date of an active application cycle is modified, the system shall send a notification to all Students and Project Supervisors.
- **REQ-CYCLE-4** When a new cycle is created, offers of the previous cycle that nobody answered shall become Timed out, proposals still waiting for a Project Supervisor shall become Withdrawn, and every Student shall again be able to apply, propose and be matched. Slots shall be counted, and offers timed out, for the current cycle only.

4.2 ManageMatchingRounds

4.2.1 Description and Priority

Description

This feature enables an Admin to configure multiple sequential matching rounds within the active application cycle, each consisting of an Application Window followed by a Review Window and an Approval Window. The duration of each window is configurable by the Admin.

Priority

High; essential for defining the structured timeline of multi-round student applications and supervisor rankings.

4.2.2 Stimulus/Response Sequences

1. After the Admin navigates to the Application Cycle configuration, the Admin selects the option to manage matching rounds.
    a) Admin selects the option to add or configure matching rounds.
        i) The System generates the interface for up to three sequential matching rounds, where the Admin can set each round's optional start date and the length of its application, review and approval windows.
    b) Admin selects the option to modify existing matching rounds.
        i) The System displays the previously configured matching round dates for editing. A change to a round that has already started is refused with a message.

ii) The System validates that none of the rounds overlap with each other and that all rounds fall within the global Application Cycle start and end dates.

iii) When the dates of a round change, the System notifies all Students and Project Supervisors of the new dates.

4.2.3 Functional Requirements

- **REQ-ROUND-1** The system shall allow an Admin to configure 1 to 3 sequential Matching Rounds in an Application Cycle. For each round the Admin shall set the Application Window length (1 to 14 days), the Review Window length (1 to 7 days), the Approval Window length (REQ-APPRV-5) and, optionally, a start date. A round without a start date shall begin when the pre-application period ends (Round 1) or 2 days after the previous round's Approval Window ends. A round whose start date has already passed shall be refused.
- **REQ-ROUND-2** The system shall allow an Admin to change or remove a Matching Round only before its Application Window starts. A change to a round that has started shall be refused with a message. A round shall not be removed once it holds applications carried over from an earlier round (REQ-EXEC-3) or Students assigned to it by an Admin.
- **REQ-ROUND-3** The system shall reject the configuration if the calculated dates for a matching round overlap with another matching round or fall outside the global Application Cycle dates.
- **REQ-ROUND-4** When the dates of a Matching Round change, the system shall notify all Students and Project Supervisors of the new dates.

4.3 ExecuteMatchingProcess

4.3.1 Description and Priority

Description

This feature runs the stable-matching algorithm automatically when a round's Review Window closes, to assign students to projects based on ranked preferences and supervisor rankings.

Priority

High; essential for processing allocations and rolling over unmatched entities.

4.3.2 Stimulus/Response Sequences

1. When a round's Review Window closes, the System starts the matching process by itself. No Admin action is needed.

2. The System verifies Student eligibility, excluding any Student who already holds a match this cycle or who declined or let an offer time out in an earlier round.

3. The System runs the matching algorithm against rankings and verifies capacity constraints.

4. The System commits match results to storage and logs an immutable entry under the actor "System".

5. The System notifies each Project Supervisor who has a proposed match. Rollover of unmatched Students happens when results are published (4.5).

4.3.3 Functional Requirements

- **REQ-EXEC-1** The system shall only permit the matching process to be initiated after the round review window has passed.
- **REQ-EXEC-2** The system shall prevent the matching process from being initiated more than once per matching round in the application cycle.
- **REQ-EXEC-3** The system shall allow an Admin, when configuring rounds, to choose whether the applications of unmatched eligible Students roll over into the next round. Open slots of published listings, including slots released by rejected, declined or timed-out matches, shall carry over to the next round automatically.
- **REQ-EXEC-4** The matching process shall use student-proposing deferred acceptance with listing capacities. It shall exclude Students who are ineligible or already hold a match this cycle, use only slots no other match holds, and match a Student to a listing only if its Project Supervisor ranked that Student.
- **REQ-EXEC-5** After execution, the system shall set every resulting match to Proposed and notify each affected Project Supervisor.
- **REQ-EXEC-6** The system shall run the matching process automatically when a round's Review Window ends.

4.4 ReviewMatchingResults

4.4.1 Description and Priority

Description

This feature enables an Admin to review a round's matches, the declines made by Project Supervisors and the Students who were not matched. The page is read only. Reassignment is done from the Overrides page (4.6) and publication is automatic (4.5).

Priority

High; critical for finalising and releasing match allocations.

4.4.2 Stimulus/Response Sequences

1. The Admin navigates to the "Application Cycle" page and opens a round's matches.

2. The System displays a summary (matched, rejected, unmatched, and when results publish), the round's matches with their status and, for a decline, its reason, and the Students who applied but were not matched.

3. For an unmatched Student the Admin selects "Assign", which opens the Overrides page for that Student (4.6).

4.4.3 Functional Requirements

- **REQ-REV-1** The system shall allow an Admin to filter a round's matches by status (Proposed, Approved, Rejected, Reassigned, Published, Accepted, Declined, Timed out) and to view the Students who applied in the round but were not matched.
- **REQ-REV-2** The system shall allow an Admin to reassign a match that has not yet been offered to a Student to another published project with an open slot, with a justification of at least 10 characters, from the Overrides page. If the round is already published, the reassigned match shall be offered to the Student immediately.

4.5 PublishResults

4.5.1 Description and Priority

Description

This feature publishes the approved matching results automatically so that students and project supervisors can view their final allocations.

Priority

High; crucial for releasing the outcomes of a matching round.

4.5.2 Stimulus/Response Sequences

1. When a round's Approval Window ends, the System publishes the round's results by itself. Every match that was not declined becomes an offer.

2. The System transitions matching results to the published state and sets the expiry of each offer.

3. The System triggers automated notifications to affected students and supervisors.

4. If rollover is enabled, the System carries the applications of unmatched eligible Students into the next round.

4.5.3 Functional Requirements

- **REQ-PUB-1** The system shall publish a round's results only after matching has run and the round's Approval Window has ended, and only once per round.
- **REQ-PUB-2** When results are published, the system shall notify all affected users.
- **REQ-PUB-3** On publication, each offer shall remain open until the end of the next Application Window that has not closed, or until the cycle's closing date after the last round. The cycle shall close at least 7 days after the last round's Approval Window ends.
- **REQ-PUB-4** The system shall publish a round's results automatically when its Approval Window ends.
- **REQ-PUB-5** When the dates of the next round or the cycle's closing date change, the deadlines of open offers shall move with them and each affected Student shall

be notified. After the cycle's closing date, an open offer can no longer be accepted or declined.

4.6 ManualAssignStudent

4.6.1 Description and Priority

Description

This feature allows an Admin to assign, reassign or unassign any student from the Overrides page, which lists every active student in the cycle with where they stand (accepted, offer sent, matched, or unassigned with the reason). It is reserved for exceptional circumstances mandating a manual override of URECA Project assignments.

Priority

High; essential for handling administrative overrides when standard matching requires manual intervention.

4.6.2 Stimulus/Response Sequences

1. Admin opens the Overrides page and finds the Student.
2. Admin selects "Assign" for an unassigned Student, "Reassign" for a Student whose round has not published, or "Unassign".
3. Admin selects an available URECA Project to be assigned to the Student and enters a justification.
4. Admin clicks "Confirm".

4.6.3 Functional Requirements

- **REQ-ASSIGN-1** The system shall allow an Admin to manually assign a Student to a URECA Project as a manual override.
- **REQ-ASSIGN-2** The system shall require the Admin to input a justification of at least 10 characters before executing a manual assignment override.
- **REQ-ASSIGN-3** If a Student is already assigned to a URECA Project in the current cycle, the system shall display a warning message and prevent assignment until the current project is unassigned. A project from an earlier cycle shall not count as already assigned.
- **REQ-ASSIGN-4** The system shall allow an Admin to unassign a Student from a project with a reason of at least 10 characters, freeing the slot and notifying the Project Supervisor, and the Student if they had been offered the place.
- **REQ-ASSIGN-5** A new assignment shall join the current round. The Student shall receive the offer with that round's results, or at once if those results are already published, and the page shall say which.
- **REQ-ASSIGN-6** The project page shall show an Admin the Students the project holds in the active cycle (matched, offered or accepted), each with its status and round and, for an offer, the answer-by date, and shall link each name to the Overrides page filtered to that Student.

4.7 MaintainAuditLog

4.7.1 Description and Priority

Description

This feature allows an Admin to view immutable system audit records of administrative operations, including matching execution runs and manual allocation overrides, and of actions the system takes by itself.

Priority

Medium; provides traceability and compliance tracking.

4.7.2 Stimulus/Response Sequences

1. The Admin navigates to the "Audit Log" tab.

2. The System displays timestamped records of all administrative operations along with the initiating Admin or the System (Figure 3.33). The newest 50 entries are shown, with "Load more" for the next page.

3. The Admin filters or inspects specific log entries. Search and filters cover the whole log, not only the loaded entries.

4.7.3 Functional Requirements

- **REQ-AUDIT-1** The system shall record, with timestamp and initiating Admin: creating and changing a cycle; configuring rounds; reassigning, assigning and unassigning Students; and creating, updating and deactivating user accounts. Matching runs and the publication of results shall be recorded under REQ-AUDIT-3.
- **REQ-AUDIT-2** The system shall strictly prevent any user, including an Admin, from modifying or deleting audit log entries.
- **REQ-AUDIT-3** Actions the system takes on its own, including running matching and publishing results, shall be recorded under the actor "System".
- **REQ-AUDIT-4** The system shall list audit entries newest first, 50 at a time, or oldest first on request, and shall apply search and filters to the whole log.

4.8 ManageUserAccounts

4.8.1 Description and Priority

Description

This feature allows an Admin to create, update or deactivate user accounts in the system.

Priority

High; critical for user administration and access control.

4.8.2 Stimulus/Response Sequences

1. The Admin navigates to the "User Management" dashboard.
    a) Admin selects the option to manage and update user details.

i) The System displays the full directory of users with role and status filters.

ii) The Admin selects a specific user account to edit, updates their name, role (Student, Project Supervisor, Admin) or status, and clicks "Save" to commit the changes and log the action. Every role change asks for confirmation.

b) Admin selects the option to deactivate a user account.

i) The Admin selects "Deactivate" next to an active user.

ii) The System prompts for confirmation, and upon confirmation, marks the user account as inactive and immediately terminates that user's active sessions. If the change would close active applications, listings, proposals or matches, the System lists them and requires the Admin to acknowledge the warning before the cleanup happens.

4.8.3 Functional Requirements

- **REQ-USER-1** The system shall allow an Admin to create a new user account and assign a role (Student, Project Supervisor, Admin).
- **REQ-USER-2** The system shall restrict the creation of Admin-role accounts to existing Admins.
- **REQ-USER-3** When a user account is deactivated, the system shall terminate that user's active sessions.
- **REQ-USER-4** The system shall prevent the signed-in Admin from changing their own role or deactivating their own account, and shall never leave the system without an active Admin.
- **REQ-USER-5** The system shall ask for confirmation on every role change, and shall require an explicit acknowledgement of an itemised warning before closing active applications, listings, proposals or matches as a result of a role or status change.

Student Features (4.9 to 4.14)

4.9 QueryUrecaListings

4.9.1 Description and Priority

Description

Allows a Student to search and browse available project listings, and recommends the projects that best fit the Student's research interests.

Priority

High.

4.9.2 Stimulus/Response Sequences

1. Student selects "Browse projects" during pre-application or an active matching round.

2. The system displays published project listings filterable by keyword, school, topic tag, and supervisor. Each listing shows its open slots, its number of applicants and, when demand is high, a High demand label. Applicant counts include only the current round's applications.

3. If the Student has research interests and may still apply this cycle, the system also shows up to six recommended projects, each with a match score from 0 to 100 and the reasons for it.

4. The Student may sort the listings by best match, and selects a listing to view its details. While applications are not open, every listing is shown locked, with the reason, and can still be opened.

4.9.3 Functional Requirements

- **REQ-BROWSE-1** The system shall allow a Student to search and browse project listings before the formal application cycle opens.
- **REQ-BROWSE-2** The system shall allow students to filter and search listings by keyword, school, supervisor name, and topic tags.
- **REQ-BROWSE-3** The system shall exclude closed or unpublished project listings from student search results.
- **REQ-BROWSE-4** The system shall recommend to a Student, best first, up to six published projects that share at least one topic with the Student's research interests, each with a match score from 0 to 100 and the reasons for it.
- **REQ-BROWSE-5** The system shall count only the current round's applications in a listing's applicant count, its Applied mark and its High demand label, so that a new cycle and the pre-application period start with none. While applications are not open (before Round 1, between rounds, after the cycle), the system shall show every listing as locked with the reason, and a Student who cannot apply shall be told they are not eligible.
- **REQ-BROWSE-6** The match score shall be the sum of 40 points for one shared topic or 50 for two or more, up to 30 points for the share of prerequisites the Student has passed, and up to 20 points for fewer applicants per open slot.
- **REQ-BROWSE-7** The system shall leave out of the recommendations the projects the Student has applied to and the projects with no open slot, and shall recommend nothing to a Student who has no research interests or may no longer apply this cycle.
- **REQ-BROWSE-8** The system shall not show match scores to Project Supervisors.

4.10 SubmitUrecaApplication

4.10.1 Description and Priority

Description

Allows a Student to submit applications and preference rankings during a round's Application Window.

Priority
High.

4.10.2 Stimulus/Response Sequences

1. Student views project details and clicks "Apply".
2. Student writes a Statement of Interest and selects documents from their profile to attach.
3. Optionally, the Student clicks "Check my application". The system shows tips on the statement and the attached documents. It gives no score and saves nothing.
4. Student clicks "Submit application". The application becomes the Student's lowest preference.
5. The system validates the inputs and checks that an Application Window is open.

4.10.3 Functional Requirements

- **REQ-APPLY-1** The system shall prevent a Student from submitting, withdrawing or re-ranking an application outside a round's Application Window.
- **REQ-APPLY-2** The system shall require a Statement of Interest of at least 50 characters and allow the Student to attach their own uploaded documents.
- **REQ-APPLY-3** The system shall reject an application if the Student's profile is incomplete, the Student is ineligible or already holds a project this cycle, the listing is not published or has no open slots, or the Student already has an active application to that listing.
- **REQ-APPLY-4** The system shall allow a Student to withdraw only an application still under review, and shall renumber the remaining preference ranks.
- **REQ-APPLY-5** The system shall prevent a Student who holds an open match offer from applying to projects.
- **REQ-APPLY-6** The system shall provide a "Check my application" function that returns tips on a draft application before it is submitted.
- **REQ-APPLY-7** The system shall not give a score for the draft when a Student runs the check.
- **REQ-APPLY-8** The system shall not save the draft or the tips when a Student runs the check.
- **REQ-APPLY-9** The system shall allow each Student at most 6 checks in any 10-minute period.

4.11 RankAppliedProjects

4.11.1 Description and Priority

Description
This feature allows a Student to rank and organise their submitted URECA project applications in order of preference during the active application window.

Priority
High; crucial for determining student preference hierarchy during the application process.

4.11.2 Stimulus/Response Sequences

1. The Student navigates to their submitted applications dashboard.
2. The System displays the list of currently submitted URECA project applications.
3. The Student moves an application up or down to change its place in the preference order.
4. The System validates that every active application appears exactly once, with sequential preference numbers.
5. The System saves the new order immediately, with no separate save button, and shows "Order saved".
6. If the Student withdraws an application, the System moves each remaining preference up one place.

4.11.3 Functional Requirements

- **REQ-SRANK-1** The system shall allow a Student to rank their submitted applications in order of preference.
- **REQ-SRANK-2** The system shall require the ranking to contain every active application exactly once.

4.12 SelfProposeUrecaApplication

4.12.1 Description and Priority

Description
This feature allows Students to self-propose a URECA Project and submit it to their desired Project Supervisor for approval, and to withdraw the proposal while the Supervisor has not answered it.

Priority
High; provides an alternative pathway for students to pursue personalised research topics.

4.12.2 Stimulus/Response Sequences

1. Student clicks on "Propose a Project".
2. The System displays the Application page for a Self-Proposed URECA Project, in four steps: Supervisor, Your project, Why this supervisor, and Review and send. Each step enables Continue only when its rules are met, and the last step summarises every answer.
3. Students fill in their proposed URECA Project title, write-up, research interests chosen from the standard topic list, supporting documents, and choice of Project Supervisor.
4. Students click "Send proposal". The System asks the Student to confirm, stating that an accepted proposal cannot be changed and that all the Student's other applications and proposals are then withdrawn.
5. On confirmation, the System validates all required fields, submits the proposal for Project Supervisor consideration, and notifies the Supervisor.

Alternative Flow: Withdraw a proposal

A. On My Applications, the Student selects "Withdraw" on a proposal that the Supervisor has not yet answered, and confirms. The System marks the proposal as Withdrawn and notifies the Supervisor.

Alternative Flow: Read why a proposal was declined

B. On My Applications, a declined proposal shows the reason the Supervisor gave. The Student may use it to send an improved proposal.

4.12.3 Functional Requirements

- **REQ-PROPOSE-1** The system shall allow a Student to create a self-proposed project with a title of 5 to 150 characters, a description of at least 50 characters and at most 500 words, a statement of at least 50 characters and at most 300 words, and at least one research interest chosen from the standard topic list.
- **REQ-PROPOSE-2** The system shall allow a Student to select one active Project Supervisor to receive the proposal, and shall notify that Supervisor.
- **REQ-PROPOSE-3** The system shall accept a proposal only during an Application Window.
- **REQ-PROPOSE-4** The system shall ask the Student to confirm before a proposal is sent, stating that an accepted proposal cannot be changed and that all the Student's other applications and proposals are then withdrawn.
- **REQ-PROPOSE-5** The system shall allow a Student to withdraw a proposal until the Project Supervisor accepts or declines it, and shall notify that Supervisor. An accepted or declined proposal shall be final.
- **REQ-PROPOSE-6** The system shall reject a proposal from a Student who already holds a project this cycle or who has declined or not answered a match offer, and shall allow a Student only one pending proposal with each Project Supervisor.

4.13 ManageMatchingOutcome

4.13.1 Description and Priority

Description

Allows a Student to view published matching outcomes and accept or decline match offers.

Priority

High.

4.13.2 Stimulus/Response Sequences

1) Student navigates to "My Applications" and views the match offer details.

2) Student selects "Accept offer".

3) The System finalises the match assignment, registers the Student to the project, withdraws the Student's other open applications and pending proposals, and notifies the Project Supervisors of those proposals. If the accepted place was the project's last open slot, the

System also withdraws every other Student's application to that project, renumbers their remaining preferences and notifies them.

Alternative Flow for Step 2

A. Manual Decline: Student selects "Decline". The System displays a confirmation warning regarding round disqualification, marks the offer as declined upon confirmation, renders the Student ineligible for future rounds, and withdraws the Student's pending proposals.

B. Auto-Decline: If the Student does not accept or decline the match offer before it expires, the System automatically transitions the match status to "Timed out", renders the Student ineligible for future rounds, and withdraws the Student's pending proposals.

4.13.3 Functional Requirements

- **REQ-OUTCOME-1** When a Student accepts a match offer, the system shall fill one slot of that listing, withdraw the Student's other open applications and pending proposals, and make the Student ineligible for later rounds. A Student who has accepted a project this cycle, through an offer or a proposal, shall not apply, propose or accept another.
- **REQ-OUTCOME-2** When a Student manually declines a match offer, the system shall render that Student ineligible to participate in any subsequent matching rounds for the current application cycle and withdraw the Student's pending proposals.
- **REQ-OUTCOME-3** If a Student does not respond before the offer expires (REQ-PUB-3), the system shall set the match to "Timed out".
- **REQ-OUTCOME-4** A Student whose match offer transitions to "Timed out" shall be rendered ineligible to participate in any subsequent matching rounds for the current application cycle, and the Student's pending proposals shall be withdrawn.
- **REQ-OUTCOME-5** When a Student's pending proposals are withdrawn under REQ-OUTCOME-1, 2 or 4, the system shall notify the Project Supervisor of each proposal.
- **REQ-OUTCOME-6** When the last open slot of a listing is filled, the system shall withdraw every other submitted application to that listing, renumber the remaining preferences of each affected Student, and notify those Students.

4.14 ManageProfile

4.14.1 Description and Priority

Description

This feature allows a Student to complete, view, and update personal academic details, completed modules, research interests, and transcript documents.

Priority

High; required upon first sign-in and for maintaining accurate applicant data.

4.14.2 Stimulus/Response Sequences

1. The Student navigates to "My Profile".

2. The System displays the Student's school, CGPA, year of study, completed modules and research interests. Completed modules are chosen from the NTU course catalogue and research interests from the standard topic list. A Student with no transcript sees only an upload area and the profile fields locked; once a transcript is uploaded the fields are filled from it.

3. The Student uploads or replaces documents at any time. To correct the fields by hand the Student selects "Edit profile", updates them, and clicks "Save changes".

4. The System reads an uploaded transcript before storing it, validates the file and inputs, then saves the profile.

4.14.3 Functional Requirements

- **REQ-PROF-1** The system shall require a Student to complete their profile after first sign-in.
- **REQ-PROF-2** The system shall allow a Student to enter their school, CGPA, year of study, completed modules chosen from the NTU course catalogue, and research interests chosen from the standard topic list.
- **REQ-PROF-3** The system shall accept supporting documents in PDF format up to 5 MB in size and reject invalid formats or sizes.
- **REQ-PROF-4** The system shall accept a file as the Student's transcript only if it states a Cumulative GPA and lists courses passed.
- **REQ-PROF-5** The system shall not store a file rejected as a transcript, and shall leave the profile unchanged.
- **REQ-PROF-6** The system shall warn the Student when a file is rejected as a transcript.
- **REQ-PROF-7** A valid transcript shall replace the previous one and update the school, degree, CGPA, year of study and completed modules, and shall never change research interests.
- **REQ-PROF-8** The system shall not allow a transcript to be deleted.

Project Supervisor Features (4.15 to 4.18)

4.15 ManageProjectDetails

4.15.1 Description and Priority

Description

This feature allows a Project Supervisor to create, update, and close research project listings.

Priority

High; required for maintaining active research offerings and managing project slots.

4.15.2 Stimulus/Response Sequences

1. The system displays the Project Supervisor's existing project listings and prompts them to select CREATE, UPDATE, or CLOSE.

    a) The Project Supervisor selects CREATE.

        i) The system prompts in four guided steps for the project title, description, prerequisites chosen from the NTU course catalogue, topic tags chosen from the standard topic list, number of available slots, offering faculty, research location and cover image, and for optional details: tasks and outputs, skills gained, hours per week, mode, photos and a video link.

        ii) The Project Supervisor inputs the required details and selects PUBLISH or SAVE AS DRAFT.

    b) The Project Supervisor selects UPDATE.

        i) The system displays the selected project's existing details for editing. A closed project cannot be edited.

        ii) The Project Supervisor modifies the details and selects SAVE.

    c) The Project Supervisor selects CLOSE.

        i) The system prompts the Project Supervisor to confirm closing the project.

        ii) On confirmation, the system marks the project listing as closed, removes it from student browsing, withdraws the applications to it that are still under review, renumbers each affected Student's remaining preferences, and notifies those Students.

2. The system validates the entered data and saves the listing as published or as a draft.

4.15.3 Functional Requirements

- **REQ-PROJ-1** The system shall allow a Project Supervisor to save a listing as a draft at any time, and to publish it before or during the application cycle but not after its closing date. Topic tags and prerequisites shall be chosen from the standard topic list and the NTU course catalogue.
- **REQ-PROJ-2** The system shall only allow a project listing to be updated before the application closing date, and shall not allow a closed listing to be updated.
- **REQ-PROJ-3** The system shall prevent a Project Supervisor from reducing a project's slots below the number of students who have accepted a place on it plus the match offers on it that are still open.
- **REQ-PROJ-4** The system shall allow a Project Supervisor to close their own listing, removing it from student search. Applications to it that are still under review shall be withdrawn, the remaining preferences of each affected Student renumbered, and those Students notified.
- **REQ-PROJ-5** The system shall allow a Project Supervisor to add to a listing the optional details tasks and outputs, skills gained, hours per week (1 to 40), mode (on site, hybrid or remote), up to 4 photos, and one YouTube or Vimeo video link.

- **REQ-PROJ-6** The system shall require the offering faculty, and shall limit a listing's title to 150 characters, its description to 5,000 characters and its research location to 100 characters.
- **REQ-PROJ-7** The system shall accept a cover image and photos only as JPEG or PNG files of at most 5 MB each, checked by their content.

4.16 ReviewAndRankStudentApplications

4.16.1 Description and Priority

Description

This feature allows a Project Supervisor to review students who applied to their project, search, filter and sort them, optionally by AI score, and rank them in order of preference.

Priority

High; essential for evaluating applicant suitability and determining placement preferences.

4.16.2 Stimulus/Response Sequences

1. The Project Supervisor selects a project to review.

2. The system displays all applicants sorted by CGPA from highest, with search by name or school, filters for school, year of study and minimum CGPA, and views showing only the applicants who meet the prerequisites or are not yet ranked. The Project Supervisor may instead sort by AI score or name.

3. The Project Supervisor selects an applicant to review their profile and transcript.

4. The Project Supervisor adds applicants to the ranking and reorders or removes them, and the system saves each change immediately. Alternatively, the Project Supervisor uses "Rank top N" to replace the ranking with the first N applicants as currently filtered and sorted, after confirming that the current order will be lost.

5. The Project Supervisor clicks "AI score". The system scores each applicant on four factors (prerequisites, fit, experience and grades) and shows each score with a reason and the quotes it rests on.

6. The Project Supervisor may change how much each factor counts for this project, and runs "AI score" again.

4.16.3 Functional Requirements

- **REQ-VRANK-1** The system shall allow a Project Supervisor to search applicants by name or school, filter them by school, year of study, minimum CGPA and whether prerequisites are met, and sort them by CGPA from highest (the default), AI score or name.
- **REQ-VRANK-2** The system shall allow a Project Supervisor to build and modify an applicant ranking, with each change saved immediately, during the round's Application and Review Windows. Each applicant may appear once. Applicants left unranked shall not be matched to that listing.

- **REQ-VRANK-3** The system shall display applicant documents in the browser, only to the owner, Admins, and Supervisors of listings the Student applied to.
- **REQ-VRANK-4** The system shall show, with each AI score, a reason and the quotes it rests on, each with its source (the statement, the profile, the GitHub profile or an attached document).
- **REQ-VRANK-5** The system shall allow a Project Supervisor to set, for each project, how much each of four factors counts in the AI score: prerequisites, fit, experience and grades.
- **REQ-VRANK-6** The system shall show a quote only if it appears word for word in its source.
- **REQ-VRANK-7** The system shall list an attached document that has no readable text as unread.
- **REQ-VRANK-8** The system shall require the four weights of a project to total 100, with each weight at least 5.
- **REQ-VRANK-9** The system shall not show the weights to Students.
- **REQ-VRANK-10** The system shall remove e-mail addresses, phone numbers, matriculation numbers and NRIC or FIN numbers from document text before it is sent to the AI service.
- **REQ-VRANK-11** The system shall allow a Project Supervisor to replace the ranking with the first N applicants as currently filtered and sorted ("Rank top N"), after confirming that the current order will be lost.

4.17 ApproveMatchingResults

4.17.1 Description and Priority

Description

This feature allows a Project Supervisor to review students proposed to be matched to their project and decline any match during the approval window. Matches that are not declined are accepted automatically.

Priority

High; critical for finalising matches before results are published.

4.17.2 Stimulus/Response Sequences

1. The system displays the students proposed to be matched to the Project Supervisor's project, grouped by project, while the round's Approval Window is open.

2. The Project Supervisor selects a proposed match to review, and the system displays the student's profile and application details.

3. If the Project Supervisor takes no action, the match is accepted when the Approval Window ends and is published with the round's results.

4. Alternatively, the Project Supervisor selects DECLINE before the Approval Window ends. The system prompts for confirmation and an optional reason, records the decline with its reason if

one was given, and returns the affected project slot to available. Admins see the decline and its reason on the round's matches page; they are not notified of each decline.

4.17.3 Functional Requirements

- **REQ-APPRV-1** The system shall allow a Project Supervisor to decline only their own proposed matches, only after the matching process has been executed.
- **REQ-APPRV-2** The system shall record a declined match, and the reason given if any, for Admin review.
- **REQ-APPRV-3** The system shall prevent a Project Supervisor from declining a proposed match after the Approval Window has ended or the matching results have been published.
- **REQ-APPRV-4** The system shall give Project Supervisors an approval window after each review window, during which they may decline students matched to their projects. Matches not declined shall be accepted when it ends.
- **REQ-APPRV-5** The system shall allow an Admin to set the length of each round's approval window (1 to 7 days, default 2).
- **REQ-APPRV-6** The system shall return the slot of a declined match to available.

4.18 ManageSelf-ProposedProjects

4.18.1 Description and Priority

Description

This feature allows a Project Supervisor to review research proposals pitched directly by students and either accept and register or decline them.

Priority

High; enables direct engagement with student-initiated research ideas.

4.18.2 Stimulus/Response Sequences

1. The Project Supervisor navigates to the "Proposals" page.

2. The System displays all proposals submitted by students directed to the supervisor. Proposals the Student has withdrawn are shown as Withdrawn and cannot be acted on.

3. The Project Supervisor reviews the proposal title, write-up, research tags, and supporting documents, then clicks "Accept and register" and confirms.

4. The System registers the student under the supervisor, withdraws the Student's other open applications, offers and pending proposals, notifies the Supervisors concerned, and triggers a confirmation notification to the Student.

5. Alternatively, the Project Supervisor clicks "Decline", enters a reason of at least 10 characters, and the System notifies the Student with that reason.

4.18.3 Functional Requirements

- **REQ-REVPROP-1** The system shall allow a Project Supervisor to view self-proposed project submissions directed to them along with the proposing student's profile and supporting documents.
- **REQ-REVPROP-2** When a proposal is accepted, the system shall register the Student to a new one-slot project, withdraw the Student's other open applications, open offers and pending proposals, notify the Project Supervisors concerned, make the Student ineligible for later rounds, and notify the Student.
- **REQ-REVPROP-3** The system shall allow a Project Supervisor to accept a proposal only while the cycle is open and only if the Student does not already hold a project and has not declined or missed a match offer, and to decline it at any time with a reason of at least 10 characters, which the system shall store, show to the Student with the declined proposal and include in the notification. Accepting needs no reason.

User Features (4.19 to 4.21)

4.19 RegisterAccount

4.19.1 Description and Priority

Description

This feature allows a user to create a new account using an NTU email address and password, followed by an email verification process to activate the account.

Priority

High; essential for onboarding users into the system.

4.19.2 Stimulus/Response Sequences

1. The User navigates to the registration page.

2. The System displays input fields for full name, NTU email address, password, and a choice of Student or Project Supervisor.

3. The User enters details and clicks "Create account".

4. The System validates the email domain and checks for existing accounts, then sends a verification code to the provided email address.

5. The User enters the verification code, and the System verifies the code to activate the account.

4.19.3 Functional Requirements

- **REQ-REG-1** The system shall restrict registration to email addresses ending in an allowed domain, by default @e.ntu.edu.sg and @ntu.edu.sg. The list of allowed domains shall be configurable.

- **REQ-REG-2** The system shall require a user to enter the correct verification code to activate the account.
- **REQ-REG-3** The system shall reject a registration using an email address already associated with an existing account.
- **REQ-REG-4** The verification code shall be 6 digits and expire after 10 minutes, and the user shall be able to request a new one.
- **REQ-REG-5** Self-registration shall offer only the Student and Project Supervisor roles. Admin accounts are created by an Admin (REQ-USER-2).
- **REQ-REG-6** Passwords shall be at least 8 characters.

4.20 SignIn

4.20.1 Description and Priority

Description

This feature allows a registered, verified user to securely access the URECAMatch platform using their email address and password.

Priority

High; critical for authenticating users prior to system access.

4.20.2 Stimulus/Response Sequences

1. The User navigates to the sign-in page.
2. The System displays input fields for NTU email address and password.
3. The User enters their credentials and clicks "Sign in".
4. The System validates the credentials against the user database and redirects valid users to their respective dashboard.

4.20.3 Functional Requirements

- **REQ-AUTH-1** The system shall allow a verified user to sign in with their email address and password, with password input masked and an option to reveal it.
- **REQ-AUTH-2** The system shall prevent sign-in to a deactivated or unverified account.
- **REQ-AUTH-3** A user session shall expire after 30 minutes of inactivity.

4.21 ViewNotifications

4.21.1 Description and Priority

Description

This feature allows an authenticated user to view incoming and historical system notifications regarding status changes, application updates, cycle dates, and matching outcomes.

Priority

Medium; important for keeping users informed of critical updates.

4.21.2 Stimulus/Response Sequences

1. The User clicks on the "Notifications" tab in the navigation menu.
2. The System queries and displays the list of relevant timestamped notifications.
3. The User selects a notification to view its complete message details.
4. The System marks the selected notification as read and updates the unread badge counter.

4.21.3 Functional Requirements

- **REQ-NOTIF-1** The system shall deliver a notification only to the user it concerns and display notifications in reverse chronological order.
- **REQ-NOTIF-2** The system shall mark a notification as read when the user views it.

Admin Features (Continued) (4.22)

4.22 View Project Status

4.22.1 Description and Priority

Description

This feature allows an Admin to view every project of the active cycle and the students each project holds, without changing anything.

Priority

Medium; supports oversight and the choice of project when assigning a student.

4.22.2 Stimulus/Response Sequences

1. The Admin selects "Project Status" in the sidebar.
2. The System displays every project with its supervisor, school, slots filled, applicant count and status, and a search box with School, Supervisor and Status filters.
3. The Admin selects a project, and the System displays its details and the students it holds, each with their status and round.

4.22.3 Functional Requirements

- **REQ-PSTAT-1** The system shall allow an Admin to view every project of the active cycle, whether published, draft or closed, with its supervisor, school, slots filled, applicant count and status, as a read-only list.
- **REQ-PSTAT-2** The system shall allow an Admin to search the project list by project or supervisor name and to filter it by school, supervisor and status.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

5.1.1 Responsiveness

- During an Application Window, all URECA Project Listings (or a subset sufficient for loading a preliminary table display) shall be queried and fetched within 5 seconds.
- Basic visual feedback of interactive elements on the user interface shall be displayed within no more than 2 seconds after user interaction.
- A Supporting Document of up to 5 MB shall be accepted, validated and stored within 15 seconds of upload, with progress shown to the Student throughout.

5.1.2 Notification Latency

- In-app notifications shall appear within a minute of the relevant update, while email notifications shall be dispatched within a maximum of 15 minutes to the receiving users.

5.1.3 Capacity and Load

- The system shall accommodate up to 1000 concurrent active user sessions while continuing to meet the response times stated in 5.1.1.
- The system shall sustain the peak load expected in the final hour before an Application Window closes, without losing a submission or failing to serve the portal webpages to Users.

5.1.4 Matching Execution Time

- The matching process over the submitted Preference Rankings (from applying Students) and Applicant Rankings (from Project Supervisors) shall complete within 10 minutes from the conclusion of a Review Window, and results shall be published at the first scheduled run after the Approval Window ends. The scheduled run happens daily at 5:05 pm SGT. No Admin shall need to be signed in for either to happen.
- While a Matching Run is in progress, the system shall remain available for read-only access.

5.2 Safety Requirements

- All user preference and ranking data must be committed to persistent storage upon submission to ensure zero data loss in the event of a system crash.
- Following a system restart or unexpected crash, all core portal services and API integrations must recover and become accessible within 5 minutes.
- All Application and User data shall be backed up at least daily for the duration of an Application Cycle, and a backup shall be restorable to a working system within 4 hours. On the free hosting tier this is met by a scheduled daily database export; automatic backups require a paid Supabase tier (D-1).

5.3 Security Requirements

5.3.1 Authentication

- Registration shall be restricted to NTU email addresses (@e.ntu.edu.sg or @ntu.edu.sg), or approved external domains.
- A User session shall expire after 30 minutes of inactivity.
- Passwords shall be handled only by the authentication service (Supabase Auth), which stores them as salted one-way hashes; the system shall never store, log or display a password.

5.3.2 Authorization

- Every request shall be authorized against the requesting User's role, and any attempt to access a function outside that role shall be denied.
- Every rule in this document shall be enforced by the API server and not only by the browser.

5.3.3 Personal Data Protection Act

- Personal data collected from Students, including CGPA, completed modules and Supporting Documents, shall be used only for the purpose of matching, and shall not be disclosed to any party other than the Admins and the Project Supervisors entitled to view it for the purposes of URECA applications.
- Supporting Documents sent to the AI service (REQ-VRANK-4) shall be used only to score the applicant for the Project Supervisor who is entitled to view them.
- The handling of student records shall comply with the Singapore Personal Data Protection Act (PDPA) and with NTU's own policy on student data.

5.3.4 Audit Log Immutability

- Audit Log entries shall be append-only, with past records being strictly unmodifiable by a User with any role.
- Every Manual Reassignment shall be attributable to the Admin who performed it and to the time at which it occurred, and every action taken by the system itself shall be attributable to the actor "System". Entries shall be retained for the lifetime of the Application Cycle's records.

5.4 Software Quality Attributes

5.4.1 Availability

- The portal shall maintain 99.9% monthly uptime during the official application and matching periods.

5.4.2 Usability

- The interface shall render without clipping on desktop browser windows 1200 px wide or wider.
- The system must provide visual feedback (e.g., loading spinners, progress bars, or "Success/Error" notifications) for all interactive elements, such as form submissions or ranking updates.

- All User-facing error messages must be written in plain, non-technical language and provide clear instructions on how to resolve the issue.
- Brief confirmations such as "Saved." shall disappear by themselves within 10 seconds. Errors and warnings shall stay until the user acts.

5.4.3 Maintainability, Reusability & Configurability

- The system shall allow for reusability across multiple application cycles, each taking place in its own respective academic year with simple configuration changes and minimal/no code modifications.
- Application Cycle dates, the length of the Pre-Application Window and the duration of each Matching Round's Application Window, Review Window and Approval Window shall be changeable by an Admin at run time, without redeployment.

5.4.4 Documentation

- The system shall be accompanied by maintained code documentation, API specification, and setup guide sufficient for a new developer to build and run it.

5.5 Business Rules

These are the operating principles of the URECA programme that URECAMatch enforces. They are not functional requirements in themselves; the requirements in brackets are the ones that enforce each rule.

5.5.1 Roles and Authority

- One role per user. Every user holds exactly one role (Student, Project Supervisor or Admin) and may use only the functions of that role.
- NTU community only. Only holders of an e-mail address in an allowed domain (by default @e.ntu.edu.sg and @ntu.edu.sg) may register. Self-registration offers only the Student and Project Supervisor roles; only an existing Admin may create an Admin account.
- Admins run the programme. Only an Admin may create or change an Application Cycle, configure Matching Rounds, set approval windows, assign, reassign or unassign a Student, and create or deactivate accounts. Matching and publishing run by themselves.
- Supervisors act only on their own projects. A Project Supervisor may edit, close, rank applicants for, and decline matches of only their own listings, and may see a Student's documents only if that Student applied or proposed to them.
- Admins cannot lock themselves out. An Admin cannot change their own role or deactivate their own account, and the last active Admin is always protected (REQ-USER-4).

5.5.2 The Application Cycle

- One cycle at a time. At most one Application Cycle is active at any time, and cycles may not overlap. A new cycle can be created only after the current cycle's closing date has passed.
- Browse before applying. During the Pre-Application Window, Students may browse listings but may not apply or propose.
- Windows are binding. Students may apply, withdraw, re-rank and propose only during a round's Application Window. Project Supervisors may rank applicants only during that

round's Application and Review Windows, and may decline matches only during its Approval Window.
- Started rounds are fixed. A Matching Round may not be changed or removed once its Application Window has started.
- Listings freeze at closing. A listing may be published or edited only up to the cycle's closing date, and a closed listing may not be edited. Closing a listing withdraws the applications still under review for it.
- A new cycle starts fresh. Unanswered offers of the old cycle time out, waiting proposals are withdrawn, and every Student may apply, propose and be matched again (REQ-CYCLE-4).

5.5.3 Applying and Ranking

- Complete profile first. A Student must have a complete profile and be eligible before applying, and may hold only one active application per listing.
- Every application is ranked. A Student's Preference Ranking contains each of their active applications exactly once.
- Unranked means not matched. A Student whom a Project Supervisor leaves unranked is never matched to that listing.
- Answer your offer first. A Student who holds an open offer cannot apply to other projects (REQ-APPLY-5).

5.5.4 Matching and Offers

- Matching runs once per round. The system runs matching by itself after the round's Review Window has closed, once per round.
- One project per Student per cycle. A Student may hold at most one URECA Project in a cycle. Accepting an offer, or having a self-proposal accepted, withdraws all their other open applications and pending proposals, and when the last slot of a project is taken, every other application to that project is withdrawn.
- Declining is final for the cycle. A Student who declines an offer, or lets it time out, may not take part in later rounds of that cycle or send a new proposal.
- Capacity is never exceeded. A listing is never matched beyond its open slots, and a Project Supervisor may not reduce slots below the places already accepted plus the offers still open.
- Supervisor approval window. After each review window Project Supervisors may decline matches until the approval window ends. Matches not declined are accepted, and the round's results are published automatically. A Project Supervisor may not change a decision after publication.
- Offers have a fair deadline. An offer stays open until the end of the next Application Window that has not closed, or until the cycle's closing date after the last round. After the closing date nothing can be answered.
- Proposals are final once decided. A Student may have only one pending proposal with each Project Supervisor, and an accepted or declined proposal cannot be changed. A declined proposal always comes with a reason.

5.5.5 Overrides and Accountability

- Overrides need a reason. Every manual assignment, reassignment or unassignment carries a written justification of at least 10 characters.
- The audit trail cannot be changed. Every administrative change and every automatic action is recorded with who made it (or "System") and when, and no user of any role may alter or delete a record.
- Deactivation is immediate. A deactivated user is signed out at once and may not sign in again.

6. Other Requirements

6.1 Database Requirements

- All data shall be held in one PostgreSQL database on Supabase and reached only through the API server's Store interface. The web client shall never read the database directly, and row-level security shall be enabled on every table.
- Referential integrity shall be enforced by the database: every application belongs to one Student and one listing, every match to one round, and every round to one cycle.
- Records of closed Application Cycles shall be kept, so that past results and the Audit Log remain available for reporting.
- The database schema shall be kept in version control (supabase/schema.sql), together with seed data for demonstration and testing.

6.2 Legal and Data-Protection Requirements

- The system shall comply with the Singapore Personal Data Protection Act 2012 and NTU's policy on student data.
- Students shall be told what personal data is collected (CGPA, completed modules and Supporting Documents) and that it is used only for URECA matching. This release does not yet show this notice at registration (Appendix F).
- Personal data and Supporting Documents of a closed Application Cycle shall be deleted or anonymised after a retention period set by NTU, except for the Audit Log. This release keeps the data and has no deletion tool (Appendix F).
- Supporting Documents shall be stored in a private bucket and released only through short-lived signed links.

6.3 Internationalisation Requirements

- The user interface shall be in English only.
- Dates and times shall be stored in UTC and displayed in Singapore Time (UTC+8). Window opening and closing times shall be interpreted in Singapore Time.
- Dates shall be displayed in day-month-year order.

6.4 Reuse Objectives

- The system shall be reusable in every future academic year by creating a new Application Cycle, with no code changes.
- The matching algorithm shall be a self-contained module with no database or network access, so that it can be tested on its own and reused or replaced.
- The Store and API-client interfaces shall allow the database or the web client to be replaced without changing the business rules.
- New screens shall reuse the shared component library and the conventions in 3.1.

6.5 Operational Requirements

- Routine transitions (opening and closing windows, and expiring offers) shall run from a daily scheduled job, so that no Admin action is needed for them.
- Every change to the main branch shall pass automated checks (linting, type checks, tests, an API contract check and a production build) before it is deployed.

Appendix A: Glossary / Data Dictionary

Term	Description	Key Attributes	Relationships / Associations
URECA	Undergraduate Research Experience on Campus, a programme in which NTU undergraduate students undertake a research project supervised by a faculty staff or external mentor. Such projects can either be listed by the mentors, or self-proposed by the student.		URECAMatch runs the URECA application and allocation process. URECA Projects are carried out under URECA.
URECAMatch (aka. the "System")	The system specified in this technical requirements document. Its primary goal is to provide a platform to facilitate an improved application process for Students, Project Supervisors, and Admins.		URECAMatch serves Admins, Project Supervisors and Students. URECAMatch hosts URECA Project Listings and Applications. One URECAMatch platform serves many User instances of all three types and hosts many URECA Project Listings. At any one time it runs at most one active Application Cycle.

URECA Project Listing	A URECAMatch descriptive entry of a URECA Project put forward by a Project Supervisor, and made available on the platform for Students to apply for. Comprises a project title, project description, (optional) prerequisites, topic tags, offering faculty/organization and main research location, and optionally tasks and outputs, skills gained, hours per week, mode, with up to 4 photos and one video link.	listingId, title, description, prerequisites, topicTags, availableSlots, filledSlots, offeringFaculty, researchLocation, coverImageUrl, status, createdAt, outcomes, skills, hoursPerWeek, mode, galleryUrls, videoUrl, scoreWeights	A Project Supervisor creates URECA Project Listings. A URECA Project Listing describes a URECA Project and receives Applications. One URECA Project Listing is created by exactly one Project Supervisor, while one Project Supervisor may create many listings. One URECA Project Listing describes exactly one URECA Project, and may receive many Applications or none. One URECA Project Listing may carry zero or more Topic Tags and zero or more Prerequisites. Each accepted Match Result fills one slot in the URECA Project Listing.

URECA Project	A research project carried out by a Student under the mentorship of a Project Supervisor, typically over the span of 11 months in an NTU academic year. Typically, URECA Projects are put forwards by a Project Supervisor, but Students can also initiate a Self-Proposed URECA Project		A Project Supervisor supervises a URECA Project. Students are matched to URECA Projects. One URECA Project is supervised by exactly one Project Supervisor, while one Project Supervisor may supervise many URECA Projects. One URECA Project takes as many Students as its listing has slots, and one Student holds at most one URECA Project per Application Cycle.
Self-Proposed URECA Project (aka. “Self-Initiated URECA Project”)	A URECA Project proposed by a Student instead of being put forward by a Project Supervisor. Students can propose a URECA Project by submitting an Application for a Self-Proposed URECA Project, and subsequently contacting a Project Supervisor to approve the supervision of the project and/or provide additional resources if necessary.		A Self-Proposed URECA Project is a URECA Project. A Student proposes it to a Project Supervisor. One Self-Proposed URECA Project is proposed by exactly one Student and directed to exactly one Project Supervisor. One Student may submit many proposals during an Application Cycle but can be registered to at most one URECA Project

			per cycle, through a listing or a proposal.
User/Actor	A person operating an account on the URECAMatch platform. A User can be classified into 3 different types: Admin, Project Supervisor, and Student.	userId, email, name, role, accountStatus, createdAt, lastLoginAt	A User is an Admin, a Project Supervisor or a Student. A User owns a User Account and receives Notifications. One User holds exactly one role. Many Users of each role may exist in URECAMatch.
Verification Code	A 6-digit code sent to a user's NTU e-mail address to activate the account. It expires after 10 minutes, and a new one can be requested (REQ-REG-4).	code, expiresAt, isUsed	A User is issued Verification Codes. Requesting a new one replaces the old one. Each Verification Code is issued to exactly one User, and one User may be issued zero or more. Requesting a new one replaces the old one. Accounts an Admin creates get none.

Admin (aka. “Administrator”)	One of the 3 primary Users/Actors of the URECAMatch platform. Typically a member of the administrative staff overseeing the URECA Programme, the Admin shall have system-wide administrative privileges to manage Student and Project Supervisor accounts, configure application opening and closing dates, manually match projects to Students/Supervisors, notify Users, and modify various other records within the URECAMatch system.	Inherits the User Attributes, none of its own	Many Admins may exist. One Admin initiates zero or more Audit Log entries.
Project Supervisor	One of the 3 primary Users/Actors of the URECAMatch platform. A Project Supervisor acts as the mentor in a URECA Project, and is able to supervise multiple URECA Projects and put forward these projects as URECA Project Listings on the platform. Project Supervisors are chiefly NTU faculty teaching staff, but can hail from external organizations as well.	title, faculty, organisation (plus the User attributes)	A Project Supervisor is a User. A Project Supervisor owns URECA Project Listings and supervises URECA Projects. One Project Supervisor may supervise many URECA Projects and Self-Proposed URECA Projects. One Project Supervisor may create multiple URECA Project Listings per Application Cycle. One Project Supervisor may own one Applicant Ranking for each URECA Project Listing they list

			during each Matching Round. One Project Supervisor receives zero or more Self-Proposed URECA Applications.
Student	One of the 3 primary Users/Actors of the URECAMatch platform. A Student is a full-time NTU undergraduate who meets all eligibility requirements to be invited for the URECA application window, and can be accepted to at most one concurrent URECA Project.	school, cgpa, yearOfStudy, completedModules, researchInterests, isProfileComplete, isEligibleForMatching (plus the User attributes)	A Student is a User. A Student submits Applications during an Application Window and is matched to at most one URECA Project. One Student can be registered to at most one URECA Project per Application Cycle. One Student may submit multiple Applications to different URECA Project Listings during an Application Window, but only at most one Application to any given URECA Project Listing. One Student owns at most one Preference Ranking per Matching Round. A Student has at most one transcript and up to 5 other Supporting Documents.

Application	A Student's request to be considered for a URECA Project.	applicationId, statementOfInterest, submittedAt	An Application is either a Listing Application or a Self-Proposed URECA Application, and carries Supporting Documents. One Application is submitted by exactly one Student, and has exactly one Statement of Interest and zero or more Supporting Documents.
Match Result	The allocation of a Student to a URECA Project Listing for a given Matching Round. It is created as Proposed when the system runs matching. The Project Supervisor may decline it during the Approval Window, with an optional reason (Rejected). When that window ends, the system publishes the rest as offers (Published). The Student accepts or declines, or the system marks the offer Timed out at its deadline (REQ-EXEC-5, REQ-EXEC-6, REQ-APPRV-4, REQ-PUB-4, REQ-OUTCOME-3).	resultId, status, preferenceRank, supervisorDecision At, publishedAt, offerExpiresAt, studentResponseAt, supervisorNote, justification	A Match Result pairs one Student with one URECA Project Listing, is published by the system when the Approval Window ends, unless the Project Supervisor declined it. Accepting a Self-Proposed URECA Application creates an Accepted Match Result. Each Match Result is associated with exactly one Student and one URECA Project Listing within exactly one Matching Round. A Student receives at most one Match Result per round, or

	An Admin can also move a match not yet offered to another project (Reassigned), assign a Student who has none, or unassign a Student, which deletes the Match Result. Each needs a justification of at least 10 characters and is logged. If the round's results are already out, the new match is offered at once (REQ-REV-2, REQ-ASSIGN-2, 4, 5).		none if their Application was unsuccessful. A Match Result may record the URECA Project Listing it was reassigned from.
Listing Application	An Application to a URECA Project Listing, as opposed to a Self-Proposed URECA Application.	preferenceRank, status (plus the Application attributes)	A Listing Application is an Application. One Listing Application is submitted to exactly one URECA Project Listing in exactly one Matching Round. One URECA Project Listing may receive many Listing Applications. Each Listing Application is ordered by at most one Applicant Ranking.

NTU	Nanyang Technological University, a tertiary higher education institution in Singapore. It hosts the URECA programme for its eligible undergraduates.		NTU hosts URECA. NTU enrols the Students and employs most Project Supervisors. NTU hosts exactly one URECA programme. It enrols many Students and employs many of the Project Supervisors, the remainder coming from external organisations.
CGPA	Cumulative Grade Point Average (CGPA), the aggregate measure of a student's academic performance across all enrolled semesters on a scale from 0.00 to 5.00, with 5.00 being the highest numerical grade point possible.	value	A CGPA is an attribute of a Student. A Project Supervisor views a Student's CGPA when ranking applicants. Each Student has exactly one CGPA.
Prerequisite	An NTU course a Student should have passed before applying, stated on a URECA Project Listing by its course code. This is merely a guideline: Students who do not fulfil a Prerequisite are still eligible to apply for a Project Listing.		A URECA Project Listing states Prerequisites. A Prerequisite is assessed against a Student's completed modules. One URECA Project Listing states zero or more Prerequisites, and one course may be a Prerequisite on many listings.

Topic Tags	A short, key-word label attached to a listing (eg. “Artificial Intelligence” or “Finance”) that highlights the topics involved in the URECA Project. May be used for filtering search results.		A URECA Project Listing carries Topic Tags. A Student filters listings by Topic Tag. One URECA Project Listing carries zero or more Topic Tags, and one Topic Tag may appear on many listings.
Statement of Interest	A brief free-text personal statement written by a Student when submitting an Application for a URECA Project Listing or a Self-Proposed URECA Project. Usually used to explain their motivation in applying for the project, as well as other relevant details not captured by the Supporting Documents.		An Application carries a Statement of Interest. A Student writes it and the Project Supervisor reads it. Each Application carries exactly one Statement of Interest, written by exactly one Student.
Supporting Document	A file uploaded by a Student in support of their Application, e.g. academic transcripts, resume, or CV.	documentId, fileName, fileFormat, fileSize, storagePath, uploadedAt	A Student uploads Supporting Documents. A Supporting Document may be attached to zero or more Applications. One Student may upload zero or more Supporting Documents, and each Supporting Document belongs to exactly one Student.

Application Cycle	A defined period set by Admins, in which Students can apply for projects listed by Project Supervisors, and Project Supervisors can consider and accept Students into their projects. During this period, Project Supervisors can also list new URECA Project Listings, and Students can similarly submit proposals for Self-Proposed URECA Projects. An Application Cycle is subdivided into a Pre-Application Window followed by one to three Matching Rounds; Students may submit Applications and proposals only during the Application Window of a Matching Round.	cycleId, academicYear, openingDate, closingDate, preApplicationDays, rolloverUnmatched, status	An Application Cycle opens with a Pre-Application Window and then runs up to three Matching Rounds. Listing Applications belong to it through their Matching Round. Self-Proposed applications belong to it by their submission date. One Cycle is active at a time, and Cycles cannot overlap. One Application Cycle is created by exactly one Admin. It contains exactly one Pre-Application Window and up to three Matching Rounds.
Pre-Application Window	The opening period of an Application Cycle, during which Students may sign in and browse URECA Project Listings but may not submit Applications or Self-Proposed URECA Projects. Its length is configured by an Admin.		An Application Cycle opens with a Pre-Application Window. A Pre-Application Window precedes the first Matching Round. A Student may view URECA Project Listings during it but may not apply. Each Application Cycle has at most one Pre-Application Window, and each

			Pre-Application Window belongs to exactly one Application Cycle.
Matching Round	One complete pass of application, review, matching, approval and publication within an Application Cycle. A Matching Round runs an Application Window, a Review Window and an Approval Window. The system runs the Matching Process when the Review Window closes and publishes the results when the Approval Window ends	roundId, roundNumber, applicationWindowStart, applicationWindowEnd, reviewWindowStart, reviewWindowEnd, status, executedAt, publishedAt, approvalWindowEnd	An Application Cycle contains Matching Rounds. A Matching Round contains an Application Window, a Review Window and an Approval Window, and runs the Matching Process once. A Student who holds a project, or who declined or let an offer lapse, is left out of later rounds. One Application Cycle may contain up to 3 Matching Rounds, and each Matching Round belongs to exactly one Application Cycle. Each Matching Round has exactly one Application Window, one Review Window and one Approval Window. A Matching Round accepts Listing Applications, collects Applicant Rankings and produces Match Results.

Application Window	The first period of a Matching Round, during which Students may submit Applications and Self-Proposed URECA Projects and set their Preference Ranking. Its length in days is configured by an Admin. Supervisors may also start to rank applicants as they apply during this round.	applicationWindowStart, applicationWindowEnd	A Matching Round begins with an Application Window. A Student submits Applications and sets a Preference Ranking during it. An Admin configures its duration. Each Matching Round has exactly one Application Window, and each Application Window belongs to exactly one Matching Round.
Review Window	The second period of a Matching Round, during which no new Applications may be submitted, and Project Supervisors review applicants and build an Applicant Ranking.	reviewWindowStart, reviewWindowEnd	An Application Window is followed by a Review Window. A Project Supervisor builds an Applicant Ranking during a Review Window or the Application Window. An Admin configures its duration. Each Matching Round has exactly one Review Window. Many Applicant Rankings may be submitted during one Review Window, one for each URECA Project Listing that received Applications.

Preference Ranking	The ordering a Student gives to their own submitted Applications, from most to least preferred.		A Student owns a Preference Ranking, set during an Application Window. A Preference Ranking orders the Student's Listing Applications in the round and feeds the Matching Process. One Student owns at most one Preference Ranking per Matching Round, and each Preference Ranking orders that Student's Applications in that round by choice preference.
Applicant Ranking	The ordering a Project Supervisor gives to the Students who applied to one of their listings. Serves as the Supervisor-side input to the Matching Process.	rankedApplicantsId, submittedAt	A Project Supervisor produces an Applicant Ranking for a URECA Project Listing during the round's Application and Review Windows. An Applicant Ranking feeds the Matching Process. At most one Applicant Ranking is produced per URECA Project Listing in each Matching Round. It orders zero or more of the Listing Applications that listing received in that round, and applicants left

			unranked are not matched to it.
Self-Proposed URECA Application	A special type of Application, detailing a Student's request to be considered for a self-proposed URECA Project, instead of a normal Application to a URECA Project Listing.	title, description, researchInterests, status, decidedAt, decisionNote	A Student submits Self-Proposed URECA Application during an Application Window. One Self-Proposed URECA Application is submitted by submitted by exactly one Student to exactly one Project Supervisor, and has exactly one Statement of Interest and zero or more Supporting Documents, along with other details of the Self-Proposed URECA Project.
Audit Log	An append-only record of every administrative operation in URECAMatch	entryId, operation, actorId, timestamp, affectedRecords, note	Admins may view the Audit Log. No Users of any role are allowed to modify existing records on the Audit Log. Each administrative operation listed in REQ-AUDIT-1 creates one Audit Log entry, initiated by exactly one Admin.

Notification	A short summary of a System event sent to a specific User, meant to inform them of relevant changes on the URECAMatch portal. Some notifications may also be sent via email.	notificationId, event, title, message, createdAt, isRead, link	A Notification is raised by a triggering event and delivered only to the User it concerns; a User views their Notifications in reverse chronological order, and each is marked read when its recipient views it. Each Notification is addressed to exactly one User; one User receives zero or more Notifications; each Notification arises from exactly one triggering event.
Approval Window	The third period of a Matching Round, after the system runs matching. A Project Supervisor may decline a proposed match, with an optional reason. Matches not declined are published when it ends. An Admin sets its duration.	approvalWindowEnd	Each Matching Round has exactly one Approval Window. Project Supervisors decline Match Results during it.

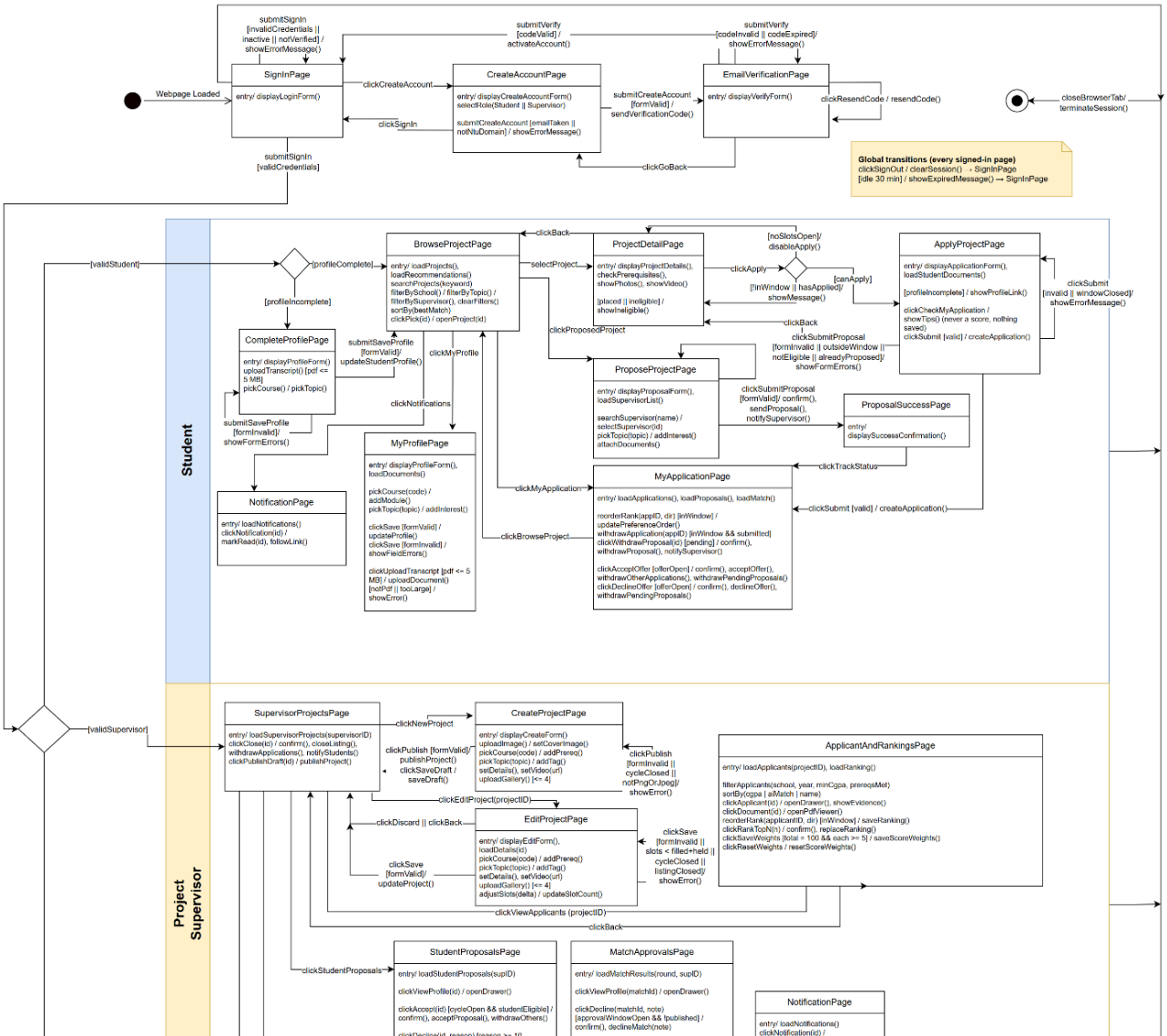
Offer	A published Match Result waiting for the Student's answer. It stays open until the end of the next Application Window that has not closed, or until the cycle's closing date after the last round	offerExpiresAt	Each Offer is one Match Result. A Student holds at most one open Offer.
AI Score	A 0 to 100 score of how well an applicant fits a listing, shown only to its Project Supervisor with a reason and the quotes it rests on. The AI service (Google Gemini) produces it. If Gemini is unavailable, the system's own formula is used.	scoreWeights	Each Student that applied for a Project Listing has at most one AI Score.

Topic	A Topic describes a field of study that a URECA Project Listing will require, or explore	name, topicGroup	A Student may add Topics to their research interest, and Project Supervisors may add Topics to their URECA Project Listing. One Topic may be used by many URECA Project Listings and Students.
Course	An individual, self-contained unit of study that Undergraduates take during their candidature at NTU. A Course is uniquely identified by their course code.	code, title, au, dept, lastOffered	Each Prerequisite and completed module must be a Course code One Course may be used by many URECA Project Listings (as Prerequisites) or Students (in their Profile).

System (actor)	The reserved account recorded as the actor for what URECAMatch does on its own, such as running matching and publishing results.	Refer to “User”	System initiates zero or more Audit Log Entries.
Score Weights	Score Weights describe how much each factor counts in a URECA Project Listing’s AI Match Score, set by its Project Supervisor. Four percentages representing “Prerequisites”, “Fit”, “Experience”, and “CGPA” that total 100, each at least 5. The default is 25 each.	prerequisites, fit, experience, cgpa	A Supervisor may adjust Score Weights on a URECA Project Listing. One URECA Project Listing has at most one set of Score Weights. They combine the factor ratings into the AI Match Score.

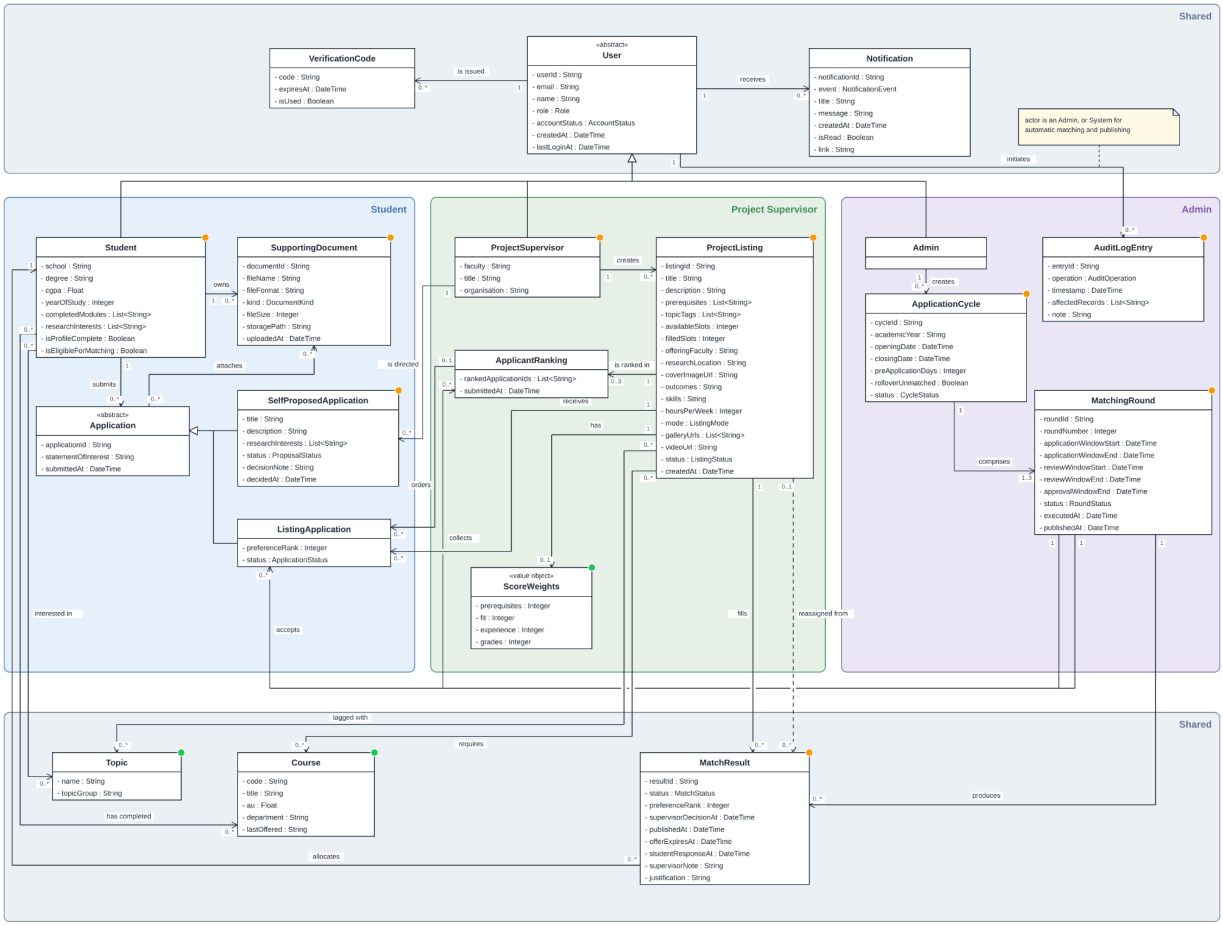
Application Tips	AI-generated advice on a draft application for a Student before it is submitted. Generated if a Student clicks on the “Check my application” button. Application Tips are not indicative of actual matching results in any way.	tips, source	A Student may generate one or more Application Tips per Application to a URECA Project Listing. Each generated Application Tips replaces the last.

Appendix B: Dialog Map



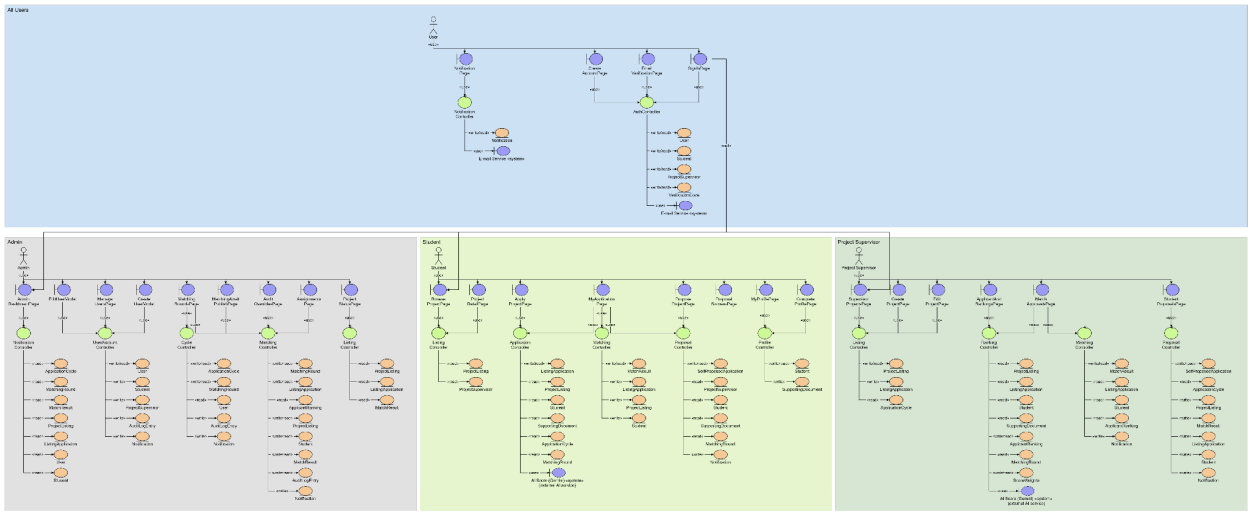
Appendix C: Class / Boundary Diagram

Figure 2.4 URECAMatch class diagram



Legend new class · class with changes · Student area · Project Supervisor area · Admin area · Shared area · Hollow triangle = inheritance · filled diamond = composition · dashed = reassigned from · gaps = line crossings · enumeration values: see table below

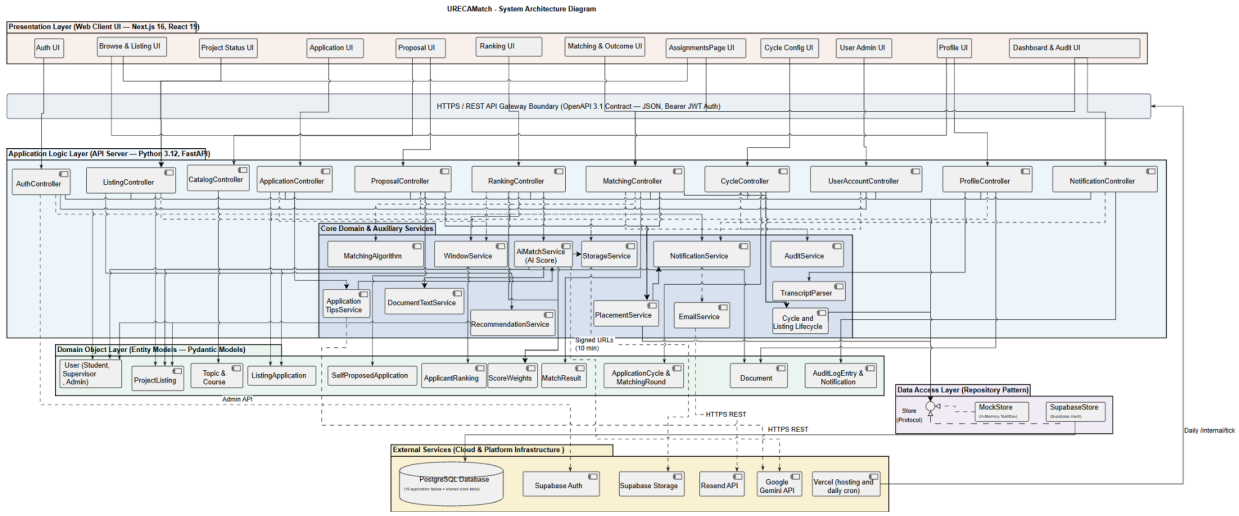
Enumeration	Used by	Values
Role	User.role	Student / Supervisor / Admin
AccountStatus	User.accountStatus	Pending verification / Active / Inactive
DocumentKind	SupportingDocument.kind	Transcript / Supporting
CycleStatus	ApplicationCycle.status	Draft / Pre-application / In progress / Closed
RoundStatus	MatchingRound.status	Scheduled / Application window / Review window / Approval window / Awaiting publication / Published / Completed
ListingStatus	ProjectListing.status	Draft / Published / Closed
ListingMode	ProjectListing.mode	On site / Hybrid / Remote
ApplicationStatus	ListingApplication.status	Submitted / Withdrawn / Matched / Unmatched
ProposalStatus	SelfProposedApplication.status	Submitted / Accepted / Declined / Withdrawn
MatchStatus	MatchResult.status	Proposed / Approved / Rejected / Reassigned / Published / Accepted / Declined / Timed out
NotificationEvent	Notification.event	Cycle changed / Round opened / Results published / Offer received / Proposal decided / Matches proposed / Match approved / Match rejected / Account changed
AuditOperation	AuditLogEntry.operation	Matching executed / Manual reassignment / Results published / User updated / User deactivated / Cycle updated


All Users
Admin
Student
Project Supervisor

Appendix D: System Architecture Diagram

Simplified:

- main entity per controller
- Other helper services (account cleanup, cycle scope, titles) are omitted for clarity.



Appendix E: Starting Skeleton Architecture (Refer: Lab 3 Supplementary AI Report)

```
URECAMatch/
├── README.md
├── .github/workflows/ci.yml   checks on every pull request
└── ureca-match-app/
    ├── api/
    │   ├── app/
    │   │   ├── main.py        entry point / driver
    │   │   ├── deps.py
    │   │   ├── errors.py      error functions/formatting
    │   │   ├── settings.py    configuration
    │   │   ├── routers/       10 routers (control)
    │   │   ├── services/      6 services (control)
    │   │   ├── models/        9 enums + 14 models (entity)
    │   │   └── db/            Store, seed data, export
    │   ├── api/index.py       Vercel entry point
    │   ├── tests/             28 test modules by requirement
    │   ├── pyproject.toml     Python dependencies
    │   ├── vercel.json        hosting and daily cron
    │   └── .env.example       environment settings
    ├── shared/openapi.json    generated API contract
```

```
├── supabase/              schema
└── web/
    ├── app/               22 pages + 5 layouts (boundary)
    ├── components/        app shell, UI library, listing form
    ├── lib/
    │   ├── api/           ApiClient, live client, types
    │   ├── mock/          mock client
    │   ├── session.ts     session expiry
    │   └── use-api.ts     data hook for pages
    ├── e2e/               end-to-end test
    ├── package.json       web dependencies
    ├── next.config.ts     Next.js settings
    └── .env.example       environment settings
```

Appendix F: To Be Determined List

To be determined at the time of writing:

- Daily backups (5.2): needs a paid Supabase tier.
- Personal data notice at registration (6.2): not yet shown.
- Retention period and deletion of closed-cycle data (6.2): the period is for NTU to set, and no deletion tool exists yet.

Source:
http://www.frontiernet.net/~kwiegers/process_assets/srs_template.doc